

LibMusicXML Reference Manual

1.03

Generated by Doxygen 1.5.1

Wed Nov 21 15:10:14 2007

Contents

1	LibMusicXML Overview	1
1.1	Introduction	1
1.2	Conventions	1
1.3	Common entities representation	2
1.4	Common types definition	3
1.5	Browsing the music tree	4
1.6	Reading and writing MusicXML files	4
1.7	MusicXML Library specific elements	4
1.8	MusicXML Elements to be implemented	4
2	LibMusicXML Module Index	7
2.1	LibMusicXML Modules	7
3	LibMusicXML Directory Hierarchy	9
3.1	LibMusicXML Directories	9
4	LibMusicXML Hierarchical Index	11
4.1	LibMusicXML Class Hierarchy	11
5	LibMusicXML Class Index	19
5.1	LibMusicXML Class List	19
6	LibMusicXML Page Index	23
6.1	LibMusicXML Related Pages	23
7	LibMusicXML Module Documentation	25
7.1	Guido	25
7.2	Visitors	27
8	LibMusicXML Directory Documentation	29
8.1	/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/core/ Directory Reference	29

8.2	global/ Directory Reference	31
8.3	/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/include/ Directory Reference	32
8.4	/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/ Directory Reference	34
8.5	/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/visitors/ Directory Reference	35
9	LibMusicXML Class Documentation	37
9.1	Bendable Class Reference	37
9.2	bimap Class Template Reference	38
9.3	Editorialable Class Reference	39
9.4	Fontable Class Reference	40
9.5	FullCue Class Reference	41
9.6	guidochecker Class Reference	42
9.7	guidochord Class Reference	43
9.8	guidoelement Class Reference	44
9.9	guidonote Class Reference	46
9.10	guidonoteduration Class Reference	47
9.11	guidonotestatus Class Reference	48
9.12	guidoparam Class Reference	49
9.13	guidoseq Class Reference	50
9.14	guidotag Class Reference	51
9.15	guidovisitable Class Reference	52
9.16	guidovisitor Class Reference	53
9.17	LineType Class Reference	54
9.18	NoteType Class Reference	55
9.19	Orientable Class Reference	56
9.20	pairmap Class Template Reference	57
9.21	Placementable Class Reference	58
9.22	Positionable Class Reference	59
9.23	smartable Class Reference	60
9.24	SMARTP Class Template Reference	62
9.25	StartStop Class Reference	64
9.26	TAccidental Class Reference	65
9.27	TAccidentalMark Class Reference	67
9.28	TArpeggiate Class Reference	68
9.29	TArticulationElement Class Reference	69
9.30	TArticulations Class Reference	71

9.31 TAttributes Class Reference	72
9.32 TBackup Class Reference	73
9.33 TBarline Class Reference	74
9.34 TBass Class Reference	76
9.35 TBeam Class Reference	77
9.36 TBeatRepeat Class Reference	79
9.37 TBend Class Reference	80
9.38 TBendSound Class Reference	81
9.39 TBendWithBar Class Reference	82
9.40 TBezier Class Reference	83
9.41 TBracket Class Reference	84
9.42 TChord Class Reference	86
9.43 TClef Class Reference	87
9.44 TCoda Class Reference	89
9.45 TCreator Class Reference	90
9.46 TDashes Class Reference	91
9.47 TDegree Class Reference	92
9.48 TDirection Class Reference	93
9.49 TDirectionType Class Reference	94
9.50 TDirectionTypeElement Class Reference	95
9.51 TDirective Class Reference	96
9.52 TDynamic Class Reference	97
9.53 TEditorial Class Reference	98
9.54 TEncoding Class Reference	99
9.55 TEnding Class Reference	100
9.56 TExpression Class Reference	102
9.57 TFeature Class Reference	103
9.58 TFermata Class Reference	104
9.59 TFigure Class Reference	105
9.60 TFiguredBass Class Reference	106
9.61 TFingering Class Reference	107
9.62 TFont Class Reference	108
9.63 TFootnote Class Reference	109
9.64 TForward Class Reference	110
9.65 TFrame Class Reference	111
9.66 TFrameNote Class Reference	112

9.67 TFunction Class Reference	113
9.68 TGlissando Class Reference	114
9.69 TGraceNote Class Reference	116
9.70 TGraphNode Class Reference	117
9.71 TGrouping Class Reference	119
9.72 THammerPull Class Reference	120
9.73 THarmonic Class Reference	121
9.74 THarmony Class Reference	123
9.75 THarmonyChord Class Reference	124
9.76 TIdentification Class Reference	125
9.77 TKey Class Reference	126
9.78 TLevel Class Reference	128
9.79 TLyric Class Reference	129
9.80 TMeasure Class Reference	131
9.81 TMeasureRepeat Class Reference	132
9.82 TMeasureStyle Class Reference	133
9.83 TMeasureStyleElement Class Reference	134
9.84 TMetronome Class Reference	135
9.85 TMidiContextVisitor Class Reference	136
9.86 TMidiDevice Class Reference	139
9.87 TMidiInstrument Class Reference	140
9.88 TMiscField Class Reference	142
9.89 TMiscId Class Reference	143
9.90 TMordentOrnament Class Reference	144
9.91 TMultipleLyricPart Class Reference	145
9.92 TMultipleRest Class Reference	146
9.93 TMusicData Class Reference	147
9.94 TMusicXMLFile Class Reference	148
9.95 TNodeVisitor Class Reference	149
9.96 TNonArpeggiate Class Reference	154
9.97 TNotation Class Reference	155
9.98 TNotationElement Class Reference	156
9.99 TNote Class Reference	157
9.100 TNoteHead Class Reference	159
9.101 TOctaveShift Class Reference	161
9.102 TOrientation Class Reference	163

9.103TOrnament Class Reference	164
9.104TOrnaments Class Reference	166
9.105TOtherArticulation Class Reference	167
9.106TOtherDirection Class Reference	168
9.107TOtherDirections Class Reference	169
9.108TOtherNotation Class Reference	170
9.109TOtherOrnament Class Reference	171
9.110TPart Class Reference	172
9.111TPartGroup Class Reference	173
9.112TPartList Class Reference	175
9.113TPartListElement Class Reference	176
9.114TPartSummary Class Reference	177
9.115TPCData Class Reference	179
9.116TPedal Class Reference	180
9.117TPitch Class Reference	181
9.118TPlacement Class Reference	183
9.119TPosition Class Reference	184
9.120TPrint Class Reference	185
9.121TPrintout Class Reference	186
9.122TPWMeasure Class Reference	187
9.123TPWPart Class Reference	188
9.124TRational Class Reference	189
9.125TRehearsal Class Reference	191
9.126TRepeat Class Reference	192
9.127TRest Class Reference	193
9.128TRights Class Reference	194
9.129Trillable Class Reference	195
9.130TrillStart Class Reference	196
9.131TrillStep Class Reference	197
9.132TRolledVisitor Class Reference	198
9.133TRoot Class Reference	201
9.134TRootFunction Class Reference	202
9.135TRoutedVisitor Class Reference	203
9.136TScanVisitor Class Reference	205
9.137TScore Class Reference	206
9.138TScoreHeader Class Reference	207

9.139TScoreInstrument Class Reference	208
9.140TScorePart Class Reference	209
9.141TScorePartwise Class Reference	210
9.142TScoreTimewise Class Reference	211
9.143TScoreVisitor Class Reference	212
9.144TSegno Class Reference	215
9.145TSlash Class Reference	216
9.146TSlide Class Reference	217
9.147TSlur Class Reference	218
9.148TSound Class Reference	219
9.149TStaffDetails Class Reference	222
9.150TStaffTuning Class Reference	224
9.151TStrongAccent Class Reference	225
9.152TSupport Class Reference	226
9.153TSymbolicNoteDuration Class Reference	227
9.154TTechnical Class Reference	228
9.155TTechnicals Class Reference	230
9.156TTie Class Reference	231
9.157TTieSlur Class Reference	232
9.158TTimeModification Class Reference	233
9.159TTimeSign Class Reference	234
9.160TTranspose Class Reference	236
9.161TTransposeVisitor Class Reference	237
9.162TTrillOrnament Class Reference	241
9.163TTrillSound Class Reference	242
9.164TTuplet Class Reference	243
9.165TTupletActual Class Reference	245
9.166TTupletDesc Class Reference	246
9.167TTupletNormal Class Reference	247
9.168TTWMeasure Class Reference	248
9.169TTWPart Class Reference	249
9.170TUnpitched Class Reference	250
9.171TUnrolledVisitor Class Reference	251
9.172TVoice Class Reference	252
9.173TWavyLine Class Reference	253
9.174TWedge Class Reference	254

9.175TWords Class Reference	256
9.176TWork Class Reference	257
9.177TXML2GuidoVisitor Class Reference	258
9.178visitable Class Reference	261
9.179vvector Class Template Reference	262
9.180xmlattribute Class Reference	263
9.181xmlelement Class Reference	264
9.182xmlheader Class Reference	266
9.183YesNo Class Reference	268
10 LibMusicXML Page Documentation	269
10.1 The MusicXML format	269
10.2 Sample code	275
10.3 Todo List	281

Chapter 1

LibMusicXML Overview

1.1 Introduction

The MusicXML library provides a set of classes that covers the elements defined by the MusicXML 1.0 dtds. The library has been developed in C++, with a great care of preserving platform independence.

Connection between the classes and the xml elements is not a one-to-one relation: for simplification, a class may contain several MusicXML elements, provided that these elements are not reused anywhere else. For example, MusicXML defines `dynamics` as separate elements (see `common.dtd`): using a single object to represent them clarifies the representation. Apart a few exceptions detailed below, all of the MusicXML elements have their counterpart in the library representation.

A good knowledge of MusicXML is required for a good comprehension of the library.

1.2 Conventions

1.2.1 Memory management

Each class that describes a MusicXML element is handled using *smart pointers*. A smart pointer maintains a reference count for an object and takes in charge the automatic deletion of this object when its reference count drops to zero.

Objects that are handled using smart pointers derive from the `smartable` class. Their implementation prevents direct call to the constructor. Instead of constructor call, they provide a function to create a new object directly embedded within a smart pointer.

Smart pointers don't work when a loop in the reference occurs. However, it doesn't matter in our case since MusicXML is strictly hierarchical.

1.2.2 Class and type names.

All the classes that corresponds to a MusicXML element have a name in the form of `Txxx` where `xxx` relates to the MusicXML element name.

For each `smartable` class, a typedef defines the corresponding smart pointer as follow:

```
typedef Txxx Sxxx where xxx is the class name.
```

More generally:

- a type name that starts with **T** denotes a class name,
- a type name that starts with **S** denotes a class smart pointer.

1.2.3 Constructors.

As mentioned above, *smartable* classes prevent direct call to their constructor but a *new* function is provided which conforms to the following rule:

for a class named `Txxx` where `xxx` is the class name, the *new* function name is `newxxx`.

A *new* function interface is similar to the class constructor. Each MusicXML object provides a unique constructor. Arguments of a class constructor are limited to the **required** MusicXML element attributes.

1.2.4 Fields access.

Fields of the defined objects are generally private or at least protected. Semantic of a field may be deduced from the field name and from the object corresponding MusicXML description, which is provided as the class documentation. Access methods are designed according to the following rules:

- methods starting with `set` are write only methods,
- methods starting with `get` are read only methods,
- methods that designates a field name (like `transpose()` to access a `fTranspose` field) are read/write methods,
- `add` methods are polymorphic methods used to push new elements into internal lists.

Warning:

Read/write methods are always used with embedded smart pointers. These pointers are initialized by default and therefore points to *null*. This design is intended to denote undefined elements.

1.2.5 Files organization.

The files organization is similar to the MusicXML dtd files organization: for example, most of the elements or entities defined in `common.dtd` are handled by **common.h** (p. ??). Elements described within MusicXML `attributes.dtd` or `direction.dtd` are respectively located in **TAttributes.h** (p. ??) or **TDirection.h** (p. ??). When no correspondence exists between a header file and a MusicXML dtd, the header file name relates to a MusicXML element and declares this element and possibly other related elements. Except for **common.h** (p. ??), all the files concerned by the above rules have a name starting with **T**.

1.3 Common entities representation

MusicXML defines entities that are common across multiple component DTDs. Many of them are intended to describe graphic layout like position, placement or orientation.

```
<!ENTITY % position
"default-x    %tenths;    #IMPLIED
"default-y    %tenths;    #IMPLIED
"relative-x   %tenths;    #IMPLIED
"relative-y   %tenths;    #IMPLIED">
```

```
<!ENTITY % placement
"placement (above | below) #IMPLIED">

<!ENTITY % orientation
"orientation (over | under) #IMPLIED">
```

These entities are defined as separate objects. A class is defined to aggregate the corresponding properties to objects that require them. For example, a `TOrientation` class is defined to describe the `orientation` entity; next an `Orientable` class is intended to provide the corresponding properties to derived classes. MusicXML elements that carry the orientation entity have to derive the `Orientable` class.

MusicXML also makes use of entities to enumerate values like the `orientation` defined above or the `start-stop` or `yes-no` examples below.

```
<!ENTITY % start-stop "(start | stop)">

<!ENTITY % yes-no "(yes | no)">
```

For all these entities, the library provides conversion classes (like the `YesNo` class) that:

- define a type for the corresponding enumeration
- provides conversion methods from/to integer and textual representation. These methods are generally named *xml* and overloaded to access both the integer and string representation.

```
class EXP YesNo {
public:
    enum type { undefined, yes, no, last=no };

    static const string xml (type d);
    static          type xml (const string str);
};
```

1.4 Common types definition

Many of the MusicXML elements are defined as an alternative between a set of elements. These alternative may be viewed as an inheritance relationship. Each time it has been convenient to do so, this relationship has been made explicit using a specific type, defined to cover the corresponding elements:

- `TMusicData` is an abstract class which purpose is to define a common type for all the elements covered by the `music-data` entity. It also provides the *smartability* and *visitability* properties to the derived objects.

```
class TMusicData : public virtual visitable, public virtual smartable {
protected:
    TMusicData () {}
    virtual ~TMusicData() {}
};
```

Similarly:

- `TDirectionTypeElement` is an abstract class which purpose is to define a common type for all the elements of a `direction-type` element, `TNotationElement` is an abstract class which purpose is to define a common type for all the elements of a `notation` element,
- `TPartListElement` is an abstract class which purpose is to define a common type for all the elements of a `part-list` element.

1.5 Browsing the music tree

The music representation is a tree which root is a *timewise* or *partwise* score. All the elements of the score support the *visitor* design pattern and accept a `TScoreVisitor` as the base class of the visitor design.

To preserve the choice of different strategies for the traversing the music representation structure, traversing has not been implemented in the tree components: it is assumed that it's the visitor responsibility. A visitor that implements a basic traversing of the music structure is included in the library: the `TRoutedVisitor`.

A `TRoutedVisitor` provides a path for traversing of a score tree. Each implemented *visit* method calls *accept()* for its subclasses. Order of these calls is the following:

- subclasses representing attributes are called first,
- subclasses representing elements are called in the MusicXML defined order.

1.6 Reading and writing MusicXML files

The `TMusicXMLFile` object provides a simple API to read and write MusicXML files.

```
class EXP TMusicXMLFile {
public:
    TMusicXMLFile() {}
    virtual ~TMusicXMLFile() {}

    SScore read (const string& file);

    bool write (SScore score, const string& file);
    bool write (SScore score, ostream& os);
};
```

It provides a `read` method that takes a file name as argument and returns smart pointer to a music score as result. The returned pointer is null in case of failure.

It provides `write` methods that take a music score and a file name or an `ostream` as arguments and returns a boolean value as result. The returned value is false in case of failure.

1.7 MusicXML Library specific elements

The library design is very close to the MusicXML definition. A music score memory representation tree matches the corresponding MusicXML tree with just a few exceptions:

- there is no chord element in the MusicXML specification but a chord object is part of the MusicXML library.
- not all the MusicXML elements are represented as individual objects: it has often been more convenient to group a collection of similar elements into a single object. The `TArticulation` object is an example of such a grouping.

1.8 MusicXML Elements to be implemented

The main part of the MusicXML elements is covered by the current library. Only a few elements are not yet supported. Below is the list of these elements.

From link.dtd:

- link
- bookmark

From opus.dtd:

- opus

Chapter 2

LibMusicXML Module Index

2.1 LibMusicXML Modules

Here is a list of all modules:

Guido	25
Visitors	27

Chapter 3

LibMusicXML Directory Hierarchy

3.1 LibMusicXML Directories

This directory hierarchy is sorted roughly, but not completely, alphabetically:

global	31
src	34
core	29
include	32
visitors	35

Chapter 4

LibMusicXML Hierarchical Index

4.1 LibMusicXML Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Bendable	37
TBend	80
TSlide	217
bimap	38
Editorialable	39
TAttributes	72
TBackup	73
TBarline	74
TDirection	93
TFiguredBass	106
TForward	110
TGraphNode	117
THarmony	123
TLyric	129
TNotation	155
TPartGroup	173
Fontable	40
TDirective	96
TDynamic	97
TFingering	107
THammerPull	120
TLyric	129
TMultipleLyricPart	145
TRehearsal	191
TWords	256
FullCue	41
guidonoteduration	47
guidonotestatus	48
guidovisitable	52
guidoelement	44
guidochord	43
guidonote	46

guidoseq	50
guidotag	51
guidoparam	49
guidovisitor	53
guidochecker	42
LineType	54
NoteType	55
Orientable	56
TTieSlur	232
TSlur	218
TTie	231
pairmap	57
Placementable	58
TAccidentalMark	67
TArpeggiate	68
TArticulationElement	69
TOtherArticulation	167
TStrongAccent	225
TBendWithBar	82
TDirection	93
TDynamic	97
TGlissando	114
TSlide	217
THarmony	123
TNonArpeggiate	154
TOrnamant	164
TOtherOrnament	171
TTrillOrnament	241
TMordentOrnament	144
TWavyLine	253
TOtherNotation	170
TEchnical	228
TBend	80
TFingering	107
THammerPull	120
THarmonic	121
TTieSlur	232
TTuplet	243
Positionable	59
TAccidentalMark	67
TArpeggiate	68
TArticulationElement	69
TBendWithBar	82
TBracket	84
TCoda	89
TDashes	91
TDynamic	97
TFermata	104
TFiguredBass	106
TGlissando	114
THarmony	123
TMetronome	135

TNonArpeggiate	154
TNote	157
TOctaveShift	161
TOrnament	164
TOtherDirection	168
TOtherDirections	169
TOtherNotation	170
TPedal	180
TRehearsal	191
TSegno	215
TEchnical	228
TTieSlur	232
TTuplet	243
TWedge	254
TWords	256
smartable	60
guidoelement	44
guidoparam	49
TAccidental	65
TArticulationElement	69
TBeam	77
TBendWithBar	82
TClef	87
TCreator	90
TDegree	92
TDirectionType	94
TDirectionTypeElement	95
TBracket	84
TCoda	89
TDashes	91
TDynamic	97
TMetronome	135
TOctaveShift	161
TOtherDirection	168
TOtherDirections	169
TPedal	180
TRehearsal	191
TSegno	215
TWedge	254
TWords	256
TDirective	96
TEncoding	99
TEnding	100
TExpression	102
TFeature	103
TFigure	105
TFrame	111
TFrameNote	112
TGraceNote	116
TGraphNode	117
THarmonyChord	124
TIdentification	125
TKey	126

TLyric	129
TMeasure	131
TPWMeasure	187
TTWMeasure	248
TMeasureStyle	133
TMeasureStyleElement	134
TBeatRepeat	79
TMeasureRepeat	132
TMultipleRest	146
TSlash	216
TMidiDevice	139
TMidiInstrument	140
TMiscField	142
TMiscId	143
TMultipleLyricPart	145
TMusicData	147
TAttributes	72
TBackup	73
TBarline	74
TChord	86
TDirection	93
TFiguredBass	106
TForward	110
TGrouping	119
THarmony	123
TNote	157
TPrint	185
TSound	219
TNotation	155
TNotationElement	156
TAccidentalMark	67
TArpeggiate	68
TArticulations	71
TDynamic	97
TFermata	104
TGlissando	114
TNonArpeggiate	154
TOrnaments	166
TOtherNotation	170
TEchnicals	230
TTieSlur	232
TTuplet	243
TNoteHead	159
TOrnamant	164
TPart	172
TPWPart	188
TTWPart	249
TPartList	175
TPartListElement	176
TPartGroup	173
TScorePart	209
TPCData	179
TFootnote	109

TLevel	128
TVoice	252
TPitch	181
TRest	193
TUnpitched	250
TRepeat	192
TRights	194
TRootFunction	202
TFunction	113
TRoot	201
TBass	76
TScore	206
TScorePartwise	210
TScoreTimewise	211
TScoreHeader	207
TScoreInstrument	208
TStaffDetails	222
TStaffTuning	224
TSupport	226
TSymbolicNoteDuration	227
TMetronome	135
TTechnical	228
TTimeModification	233
TTimeSign	234
TTranspose	236
TTupletDesc	246
TTupletActual	245
TTupletNormal	247
TWork	257
xmlattribute	263
xmlelement	264
xmlheader	266
TPartSummary	177
SMARTP	62
StartStop	64
TBendSound	81
TEditorial	98
TFont	108
TMusicXMLFile	148
TNodeVisitor	149
TTransposeVisitor	237
TMidiContextVisitor	136
TOrientation	163
TPlacement	183
TPosition	184
TRational	189
Trillable	195
TTrillOrnament	241
TrillStart	196
TrillStep	197
TScoreVisitor	212
TRoutedVisitor	203

TRolledVisitor	198
TUnrolledVisitor	251
TScanVisitor	205
TXML2GuidoVisitor	258
TTrillSound	242
vector	
visitable	261
TAccidental	65
TArticulationElement	69
TBeam	77
TBendWithBar	82
TBezier	83
TClef	87
TCreator	90
TDegree	92
TDirectionType	94
TDirectionTypeElement	95
TDirective	96
TEncoding	99
TEnding	100
TExpression	102
TFeature	103
TFigure	105
TFrame	111
TFrameNote	112
TGraceNote	116
TGraphNote	117
THarmonyChord	124
TIdentification	125
TKey	126
TLyric	129
TMeasure	131
TMeasureStyle	133
TMeasureStyleElement	134
TMidiDevice	139
TMidiInstrument	140
TMiscField	142
TMiscId	143
TMultipleLyricPart	145
TMusicData	147
TNotation	155
TNotationElement	156
TNoteHead	159
TOrnamant	164
TPart	172
TPartList	175
TPartListElement	176
TPCData	179
TPitch	181
TPrintout	186
TRepeat	192
TRights	194
TRootFunction	202

TScore	206
TScoreHeader	207
TScoreInstrument	208
TStaffDetails	222
TStaffTuning	224
TSupport	226
TEchnical	228
TTimeModification	233
TTimeSign	234
TTranspose	236
TTupletDesc	246
TWork	257
vvector	262
YesNo	268

Chapter 5

LibMusicXML Class Index

5.1 LibMusicXML Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Bendable (Base class for all elements that carry the trill-sound entity)	37
bimap (Implements a bijective map)	38
Editorialable (Base class for all elements that have editorial information)	39
Fontable (Base class for all elements that have a font)	40
FullCue (Conversions between numeric note size types and strings)	41
guidochecker (Guidochecker is intended to ensure a correct ordering a part tags)	42
guidochord (The guido chord element)	43
guidoelement (A generic guido element representation)	44
guidonote (A guido note representation)	46
guidonoteduration (A guido note duration representation)	47
guidonotestatus (Represents the current status of notes duration and octave)	48
guidoparam (A guidotag parameter representation)	49
guidoseq (The guido sequence element)	50
guidotag (A guido tag representation)	51
guidovisitable (Interface for visitable guido objects)	52
guidovisitor (Abstract generic guido visitor)	53
LineType (Conversions between numeric line types and strings)	54
NoteType (Conversions between numeric note types and strings)	55
Orientable (Base class for all elements that have an orientation)	56
pairmap (Implements a multimap where <key, value> pairs are unique)	57
Placementable (Base class for all elements that have a placement)	58
Positionable (Base class for all elements that have a position)	59
smartable (Base class for smart pointers implementation)	60
SMARTP (Smart pointer implementation)	62
StartStop (Conversions between numeric start-stop types and strings)	64
TAccidental (Representation of an accidental)	65
TAccidentalMark (Accidental-mark)	67
TArpeggiate (MusicXML <i>arpeggiate</i> representation)	68
TArticulationElement (Articulations and accents definition)	69
TArticulations (The MusicXML <i>articulations</i> element)	71
TAttributes (Contains information that typically changes on measure boundaries)	72
TBackup (Represents the MusicXML backup element)	73
TBarline (Represents a MusicXML barline element)	74

TBass (Bass element of the <i>harmony-chord</i> entity)	76
TBeam (Beaming representation)	77
TBeatRepeat (Represents beat repeat (for measure style))	79
TBend (MusicXML bend element)	80
TBendSound (Represents the bend-sound entity)	81
TBendWithBar (MusicXML with-bar (from bend) element)	82
TBezier (Represents the MusicXML <i>bezier</i> entity)	83
TBracket (The MusicXML <i>bracket</i> element)	84
TChord (Notes container that denotes a chord)	86
TClef (Represents a clef)	87
TCoda (Represents the MusicXML <i>coda</i> element)	89
TCreator (Represents the author(s) of the score)	90
TDashes (Dashes, used for instance with cresc. and dim. marks)	91
TDegree (<i>degree</i> element of the <i>harmony-chord</i> entity)	92
TDirection (The MusicXML <i>direction</i> element)	93
TDirectionType (Represents textual direction indications)	94
TDirectionTypeElement (Base class for all the elements of the <i>direction-type</i> element)	95
TDirective (Represents musical directives)	96
TDynamic (Represents the MusicXML <i>dynamics</i> element)	97
TEditorial (Represents the MusicXML <i>editorial</i> and editorial-voice entities)	98
TEncoding (Contains information about encoding)	99
TEnding (Endings refers to multiple (e.g. first and second) endings)	100
TExpression (Represents the expression context of a note)	102
TFeature (<i>feature</i> element (used by the <i>grouping</i> element))	103
TFermata (Represents the fermata sign)	104
TFigure (Figure for figured bass)	105
TFiguredBass (Figured bass)	106
TFingering	107
TFont (Represents the MusicXML <i>font</i> entity)	108
TFootnote (Footnote elements)	109
TForward (Represents the MusicXML forward element)	110
TFrame	111
TFrameNote	112
TFunction (Function element of the <i>harmony-chord</i> entity)	113
TGlissando (Glissando representation)	114
TGraceNote (Grace note representation)	116
TGraphNote (Represents the graphic elements of a note)	117
TGrouping (<i>grouping</i> element)	119
THammerPull (MusicXML <i>hammer-on</i> and <i>pull-off</i> elements)	120
THarmonic (MusicXML harmonic element)	121
THarmony	123
THarmonyChord	124
TIdentification (Contains metadata about the score)	125
TKey (Represents a key signature)	126
TLevel (Level elements)	128
TLyric (Representation of lyrics)	129
TMeasure (The MusicXML base class for partwise and timewise <i>measure</i> elements)	131
TMeasureRepeat (Represents measure repeat (for measure style))	132
TMeasureStyle (Represents measure style)	133
TMeasureStyleElement (Base class for all the elements of the <i>measure-style</i>)	134
TMetronome (Standard metronome marks)	135
TMidiContextVisitor (A Visitor that maintains a context for MIDI generation)	136
TMidiDevice (Corresponds to the DeviceName meta event in Standard MIDI Files)	139
TMidiInstrument (Represents a MIDI instrument)	140

TMiscField (Fields of miscellaneous identification)	142
TMiscId (Miscellaneous identification)	143
TMordentOrnament (Mordent and inverted-mordent ornaments)	144
TMultipleLyricPart (Representation of multiple lyrics on a single note)	145
TMultipleRest (Represents multiple rests (for measure style))	146
TMusicData (Base class for all the elements of the <i>music-data</i> entity)	147
TMusicXMLFile (MusicXML files reading and writing)	148
TNodeVisitor (A node Visitor of MusicXML scores)	149
TNonArpeggiate (MusicXML representation for <i>non-arpeggiate</i>)	154
TNotation (Represents musical notations)	155
TNotationElement (Base class for all the elements of the <i>notation</i> element)	156
TNote (TNote (p. 157) is the main common type of the music representation)	157
TNoteHead (Shape of the note head)	159
TOctaveShift	161
TOrientation (Represents the MusicXml <i>orientation</i> entity)	163
TOrnamant (Basic ornament representation)	164
TOrnament (Basic ornament representation)	166
TOtherArticulation (Other articulations)	167
TOtherDirection	168
TOtherDirections	169
TOtherNotation (Other notations representation)	170
TOtherOrnament (Other ornaments)	171
TPart (The base class for the MusicXML timewise and partwise <i>part</i> elements)	172
TPartGroup (Indicates groupings of parts in a score)	173
TPartList (Identifies the different musical parts in a movement)	175
TPartListElement (Base class for the elements of a <i>part-list</i>)	176
TPartSummary (Produces a summary of a MusicXML part)	177
TPCData (Base class for all the PCData simple elements)	179
TPedal	180
TPitch (The MusicXML representation of the pitch)	181
TPlacement (Represents the MusicXml <i>placement</i> entity)	183
TPosition (Represents a MusicXML <i>position</i> entity)	184
TPrint	185
TPrintout (Represents the MusicXML <i>printout</i> entity)	186
TPWMeasure (The MusicXML partwise <i>measure</i> element)	187
TPWPart (The MusicXML partwise <i>part</i> element)	188
TRational (Rational number representation)	189
TRehearsal	191
TRepeat (Represents repeat marks)	192
TRest (The MusicXML representation of a rest)	193
TRights (Contains copyright and other intellectual property notices)	194
Trillable (Base class for all elements that carry the trill-sound entity)	195
TrillStart (Conversions between numeric trill start notes and strings)	196
TrillStep (Conversions between numeric trill step, turn and strings)	197
TRolledVisitor (A visitor that transport a TNodeVisitor (p. 149) along a score)	198
TRoot (Root element of the <i>harmony-chord</i> entity)	201
TRootFunction (Base class for the root and function elements of the <i>harmony-chord</i> entity)	202
TRoutedVisitor (A visitor that implements a traversing of a score)	203
TScanVisitor (A visitor that only scans a score to gather information)	205
TScore (MusicXML score)	206
TScoreHeader (Contains basic score meta-data plus the part list)	207
TScoreInstrument (Represents an instrument on the score)	208
TScorePart (Identifies a musical part in a score)	209
TScorePartwise (MusicXML partwise score)	210

TScoreTimewise (MusicXML timewise score)	211
TScoreVisitor (A base class to implement MusicXML score visitors)	212
TSegno (Represents the MusicXML <i>segno</i> element)	215
TSlash (Slash element (for measure style))	216
TSlide (Slide representation)	217
TSlur (Slur representation)	218
TSound (Represents the MusicXML <i>sound</i> element)	219
TStaffDetails (Represents staff details)	222
TStaffTuning (Staff tuning element (for staff details))	224
TStrongAccent (Strong accent representation)	225
TSupport (MusciXML <i>support</i> element)	226
TSymbolicNoteDuration (Base class for symbolic note duration)	227
TTechnical (Basic technical representation)	228
TTTechnical (MusicXML <i>technical</i> element)	230
TTie (Ties representation)	231
TTieSlur (Base class for ties and slurs)	232
TTIMEModification (Represents tuplets and other durational changes)	233
TTIMESign (Represents time signatures)	234
TTTranspose (Used for transposing instruments)	236
TTTransposeVisitor (A visitor that operates a diatonic transposition of a score)	237
TTTrillOrnament (Ornaments that carry a trill-sound entity)	241
TTTrillSound (Represents the trill-sound entity)	242
TTuplet (Graphical representation of a tuplet)	243
TTupletActual (Actual tuplet description)	245
TTupletDesc (Tuplet description)	246
TTupletNormal (Normal tuplet description)	247
TTWMeasure (The MusicXML timewise <i>measure</i> element)	248
TTWPart (The MusicXML timewise <i>part</i> element)	249
TUnpitched (The MusicXML representation of an unpitched note)	250
TUnrolledVisitor (A visitor that "unroll" the score)	251
TVoice (Voice elements)	252
TWavyLine (Represents a MusicXML <i>wavy-line</i> element)	253
TWedge (MusicXML <i>wedge</i> element)	254
TWords (The MusicXML <i>words</i> element)	256
TWork (Represents works and movements)	257
TXML2GuidoVisitor (A score visitor to produce a generic Guido representation)	258
visitable (Interface for visitable objects)	261
vvector (A visitable vector)	262
xmlattribute (A generic xml attribute representation)	263
xmlelement (A generic xml element representation)	264
xmlheader (Class for generating the MusicXML header)	266
YesNo (Conversions between numeric yes-no types and strings)	268

Chapter 6

LibMusicXML Page Index

6.1 LibMusicXML Related Pages

Here is a list of all related documentation pages:

The MusicXML format	269
Sample code	275
Todo List	281

Chapter 7

LibMusicXML Module Documentation

7.1 Guido

Classes

- class **guidoparam**
A guidotag parameter representation.
- class **guidoelement**
A generic guido element representation.
- class **guidonoteduration**
A guido note duration representation.
- class **guidonote**
A guido note representation.
- class **guidonotestatus**
Represents the current status of notes duration and octave.
- class **guidoseq**
The guido sequence element.
- class **guidochord**
The guido chord element.
- class **guidotag**
A guido tag representation.
- class **guidovisitor**
an abstract generic guido visitor
- class **guidovisible**
interface for visitable guido objects.

- class **TXML2GuidoVisitor**

A score visitor to produce a generic Guido representation.

Typedefs

- typedef SMARTP< **guidonote** > **Sguidonote**
- typedef SMARTP< **guidoseq** > **Sguidoseq**
- typedef SMARTP< **guidochord** > **Sguidochord**
- typedef SMARTP< **guidotag** > **Sguidotag**

7.2 Visitors

Classes

- class **TMidiContextVisitor**
A Visitor that maintains a context for MIDI generation.
- class **TNodeVisitor**
A node Visitor of MusicXML scores.
- class **TRolledVisitor**
A visitor that transport a TNodeVisitor (p. 149) along a score.
- class **TRoutedVisitor**
A visitor that implements a traversing of a score.
- class **TPartSummary**
Produces a summary of a MusicXML part.
- class **TScanVisitor**
A visitor that only scans a score to gather information.
- class **TScoreVisitor**
A base class to implement MusicXML score visitors.
- class **visitable**
interface for visitable objects.
- class **vvector**
A visitable vector.
- class **TTransposeVisitor**
A visitor that operates a diatonic transposition of a score.
- class **TUnrolledVisitor**
A visitor that "unroll" the score.
- class **TXML2GuidoVisitor**
A score visitor to produce a generic Guido representation.

Typedefs

- typedef SMARTP< TRoutedVisitor > **SRoutedVisitor**
- typedef SMARTP< TPartSummary > **SPartSummary**
- typedef SMARTP< visitable > **Svisitable**
- typedef int **Chromatic**
- typedef int **Diatonic**
- typedef SMARTP< TTransposeVisitor > **STransposeVisitor**

- `typedef pair< long, long > TSection`

A section is a pair (beginning measure, end measure).

Chapter 8

LibMusicXML Directory Documentation

8.1 /Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/core/ Directory Reference

Files

- file **common.cpp**
- file **conversions.cpp**
- file **reader.h**
- file **smartpointer.cpp**
- file **TArpeggiate.cpp**
- file **TAttributes.cpp**
- file **TBackupForward.cpp**
- file **TBarline.cpp**
- file **TChord.cpp**
- file **TClef.cpp**
- file **TDirection.cpp**
- file **TFiguredBass.cpp**
- file **THarmony.cpp**
- file **TIdentification.cpp**
- file **TKey.cpp**
- file **TMeasure.cpp**
- file **TMeasureStyle.cpp**
- file **TMidiDevice.cpp**
- file **TMidiInstrument.cpp**
- file **TMusicXMLFile.cpp**
- file **TNotation.cpp**
- file **TNote.cpp**
- file **TOrnaments.cpp**
- file **TOtherDirections.cpp**
- file **TPart.cpp**
- file **TPartGroup.cpp**
- file **TPartList.cpp**

- file **TPitch.cpp**
- file **TRational.cpp**
- file **TRoutedVisitor.cpp**
- file **TScore.cpp**
- file **TScoreHeader.cpp**
- file **TScoreInstrument.cpp**
- file **TScorePart.cpp**
- file **TScorePartwise.cpp**
- file **TScoreTimewise.cpp**
- file **TSound.cpp**
- file **TStaffDetails.cpp**
- file **TEchnical.cpp**
- file **TTimeModification.cpp**
- file **TTimeSign.cpp**
- file **TTranspose.cpp**
- file **TWork.cpp**
- file **xml.cpp**
- file **xmllex.cpp**
- file **xmlparse.cpp**
- file **xmlparse.h**

8.2 global/ Directory Reference

Files

- file **libdoc.doxygen**
- file **libxml.doxygen**
- file **samples.doxygen**

8.3 /Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/include/ Directory Reference

Files

- file **bimap.h**
- file **common.h**
- file **conversions.h**
- file **exports.h**
- file **guido.h**
- file **guidovisitor.h**
- file **LibMusicXML.h**
- file **smartclasses.h**
- file **smartpointer.h**
- file **TArpeggiate.h**
- file **TAttributes.h**
- file **TBackupForward.h**
- file **TBarline.h**
- file **TChord.h**
- file **TClef.h**
- file **TDirection.h**
- file **TFiguredBass.h**
- file **THarmony.h**
- file **TIdentification.h**
- file **TKey.h**
- file **TMeasure.h**
- file **TMeasureStyle.h**
- file **TMidiContextVisitor.h**
- file **TMidiDevice.h**
- file **TMidiInstrument.h**
- file **TMusicXMLFile.h**
- file **TNodeVisitor.h**
- file **TNotation.h**
- file **TNote.h**
- file **TOrnaments.h**
- file **TOtherDirections.h**
- file **TPart.h**
- file **TPartGroup.h**
- file **TPartList.h**
- file **TPitch.h**
- file **TRational.h**
- file **TRolledVisitor.h**
- file **TRoutedVisitor.h**
- file **TScanVisitor.h**
- file **TScore.h**
- file **TScoreHeader.h**
- file **TScoreInstrument.h**
- file **TScorePart.h**
- file **TScorePartwise.h**
- file **TScoreTimewise.h**

- file **TScoreVisitor.h**
- file **TSound.h**
- file **TStaffDetails.h**
- file **TEchnical.h**
- file **TTimeModification.h**
- file **TTimeSign.h**
- file **TTranspose.h**
- file **TTransposeVisitor.h**
- file **TUnrolledVisitor.h**
- file **TWork.h**
- file **TXML2GuidoVisitor.h**
- file **UTools.h**
- file **xml.h**

8.4 /Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/ Directory Reference

Directories

- directory **core**
- directory **include**
- directory **visitors**

8.5 /Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/src/visitors/ Directory Reference

Files

- file `guido.cpp`
- file `GuidoChecker.cpp`
- file `GuidoChecker.h`
- file `TMidiContextVisitor.cpp`
- file `TRolledVisitor.cpp`
- file `TScanVisitor.cpp`
- file `TTransposeVisitor.cpp`
- file `TUnrolledVisitor.cpp`
- file `TXML2GuidoVisitor.cpp`

Chapter 9

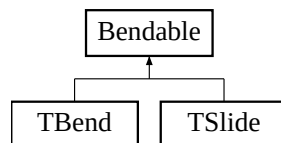
LibMusicXML Class Documentation

9.1 Bendable Class Reference

base class for all elements that carry the trill-sound entity.

```
#include <common.h>
```

Inheritance diagram for Bendable::



Public Member Functions

- **TBendSound** & **bendSound** ()

9.2 bimap Class Template Reference

implements a bijective map

```
#include <bimap.h>
```

Public Member Functions

- **bimap** (const T1 tbl1[], const T2 tbl2[], int n)
 - const T2 **operator**[] (const T1 key)
returns the second type value indexed by the first type
- const T1 **operator**[] (const T2 key)
returns the first type value indexed by the second type
- long **size** ()
returns the map size
- **bimap & add** (const T1 &v1, const T2 &v2)
adds a pair of values

9.2.1 Detailed Description

```
template<typename T1, typename T2> class bimap< T1, T2 >
```

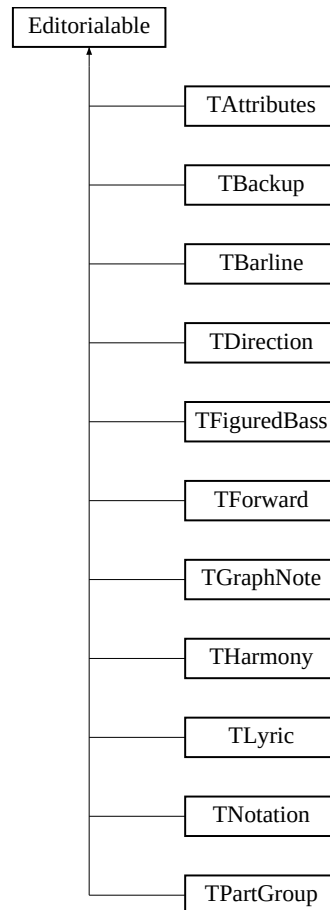
A bijective map is a map where values and keys are interchangeable. To operate correctly, typenames T1 and T2 must be different.

9.3 Editorialable Class Reference

base class for all elements that have editorial information.

```
#include <common.h>
```

Inheritance diagram for Editorialable::



Public Member Functions

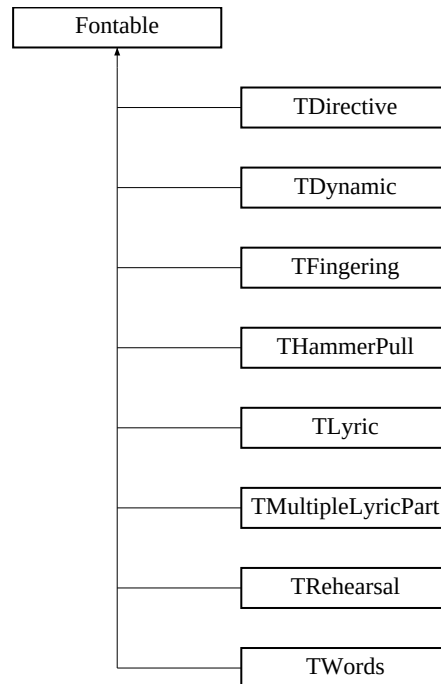
- TEditorial & editorial ()
- TEditorial * editorialPtr ()

9.4 Fontable Class Reference

base class for all elements that have a font.

```
#include <common.h>
```

Inheritance diagram for Fontable::



Public Member Functions

- `TFont & font ()`

9.5 FullCue Class Reference

provides conversions between numeric note size types and strings

```
#include <conversions.h>
```

Public Types

- enum **type** { **undefined**, **full**, **cue**, **last** = cue }

Static Public Member Functions

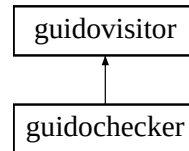
- static const string **xml** (type d)
convert a numeric size value to a MusicXML string
- static type **xml** (const string str)
convert a MusicXML string to a numeric size value

9.6 guidochecker Class Reference

guidochecker is intended to ensure a correct ordering a part tags

```
#include <GuidoChecker.h>
```

Inheritance diagram for guidochecker::



Public Member Functions

- virtual void **visite** (**Sguidoelement** &elt)
- virtual void **visite** (**Sguidonote** &elt)
- virtual void **visite** (**Sguidoseq** &elt)
- virtual void **visite** (**Sguidochord** &elt)
- virtual void **visite** (**Sguidotag** &elt)
- virtual void **visite** (**Sguidoparam** &elt)

9.6.1 Detailed Description

guidochecker is intended to ensure a correct ordering of the clef, meter key and repeatbegin tags at the beginning of a staff. It enforces the the visited guidoelements to the following order:

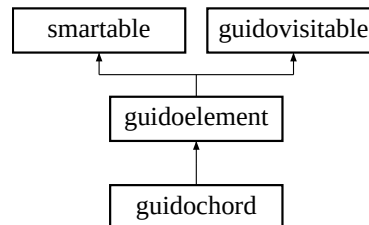
-
-
-
- The visitor stops checking at the first note or chord of a part.

9.7 guidochord Class Reference

The guido chord element.

```
#include <guido.h>
```

Inheritance diagram for guidochord::



Public Member Functions

- void **accept** (**guidovisitor** &visitor)

Friends

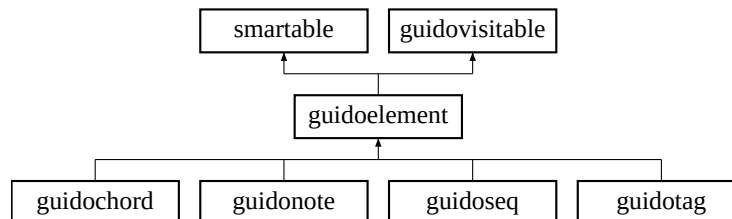
- VEXP friend **SMARTP**< **guidochord** > **new_guidochord** ()

9.8 guidoelement Class Reference

A generic guido element representation.

```
#include <guido.h>
```

Inheritance diagram for guidoelement::



Public Member Functions

- long **add** (Sguidoelement &elt)
- long **add** (Sguidoparam ¶m)
- long **add** (Sguidoparam param)
- void **print** (std::ostream &os)
- void **setName** (string name)
the element name
- string **getName** () const
- string **getStart** () const
- string **getEnd** () const
- string **getSep** () const
- vector< Sguidoelement > & **elements** ()
- const vector< Sguidoelement > & **elements** () const
- const vector< Sguidoparam > & **parameters** () const
- bool **empty** () const
- virtual void **accept** (guidovisitor &visitor)

Protected Member Functions

- **guidoelement** (string name, string sep=" ")

Protected Attributes

- string **fName**
- string **fStartList**
the contained element start marker (default to empty)
- string **fEndList**
the contained element end marker (default to empty)
- string **fSep**

the element separator (default to space)

- vector< **Sguidoelement** > **fElements**

list of the enclosed elements

- vector< **Sguidoparam** > **fParams**

list of optional parameters

Friends

- VEXP friend **SMARTP**< **guidoelement** > **new_guidoelement** (string name, string sep=" ")

9.8.1 Detailed Description

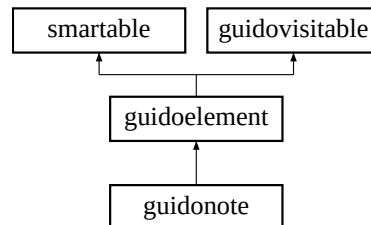
An element is represented by its name and the list of its enclosed elements plus optional parameters.

9.9 guidonote Class Reference

A guido note representation.

```
#include <guido.h>
```

Inheritance diagram for guidonote::



Public Member Functions

- void **set** (unsigned short voice, string name, char octave, **guidonoteduration** &dur, string acc)
- void **accept** (**guidovisitor** &visitor)
- const char * **name** () const
- const char * **accidental** () const
- char **octave** () const
- const **guidonoteduration** & **duration** () const

Protected Member Functions

- **guidonote** (unsigned short voice, string name, char octave, **guidonoteduration** &dur, string acc="")

Protected Attributes

- string **fNote**
- string **fAccidental**
- char **fOctave**
- **guidonoteduration** **fDuration**

Friends

- VEXP friend **SMARTP**< **guidonote** > **new_guidonote** (unsigned short voice, string name, char octave, **guidonoteduration** &dur, string acc="")

9.9.1 Detailed Description

A note is represented by its name, optional accidentals, duration (in the form of numerator/denominator) and optional dots.

9.10 guidonoteduration Class Reference

A guido note duration representation.

```
#include <guido.h>
```

Public Member Functions

- **guidonoteduration** (long num, long denom, long dots=0)
- void **set** (long num, long denom, long dots=0)
- **guidonoteduration** & **operator=** (const **guidonoteduration** &dur)
- bool **operator!=** (const **guidonoteduration** &dur) const

Public Attributes

- long **fNum**
- long **fDenom**
- long **fDots**

9.10.1 Detailed Description

A note duration is represented by a numerator (denotes the number of beats), a denominator (denotes the beat value) and optional dots. Triplets are represented as 1/3, 1/6, ... quintuplets, septuplets and so on are handled analogously.

9.11 **guidonotestatus** Class Reference

Represents the current status of notes duration and octave.

```
#include <guido.h>
```

Public Types

- enum { **kMaxInstances** = 128 }
- enum { **defoctave** = 1, **defnum** = 1, **defdenom** = 4 }

Public Member Functions

- void **reset** ()
- **guidonotestatus** & **operator=** (const **guidonoteduration** &dur)
- bool **operator!=** (const **guidonoteduration** &dur) const

Static Public Member Functions

- static **guidonotestatus** * **get** (unsigned short voice)
- static void **resetall** ()
- static void **freeall** ()

Public Attributes

- char **fOctave**
- **guidonoteduration** **fDur**

9.11.1 Detailed Description

Octave and duration may be omitted for guido notes. If so, they are inferred from preceeding notes (or rest), within the same sequence or chord, or assumed to have standard values.

The object is defined as a multi-voices singleton: a single object is allocated for a specific voice and thus it will not operate correctly on a same voice parallel formatting operations.

Todo

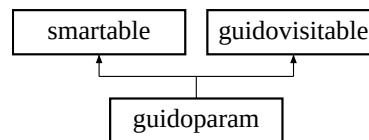
handling the current beat value for **num* duration form.

9.12 guidoparam Class Reference

A guidotag parameter representation.

```
#include <guido.h>
```

Inheritance diagram for guidoparam::



Public Member Functions

- void **set** (string value, bool quote=true)
the parameter value
- void **set** (long value, bool quote=true)
- string **get** () const
- bool **quote** () const
- void **accept** (guidovisitor &visitor)

Protected Member Functions

- **guidoparam** (string value, bool quote)
- **guidoparam** (long value, bool quote)

Friends

- VEXP friend **SMARTP**< **guidoparam** > **new_guidoparam** (string value, bool quote=true)
- VEXP friend **SMARTP**< **guidoparam** > **new_guidoparam** (long value, bool quote=true)

9.12.1 Detailed Description

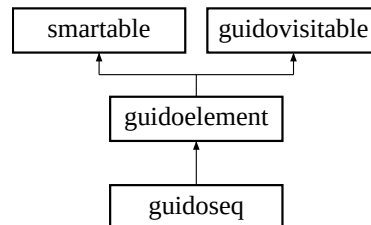
A parameter is represented by its value.

9.13 guidoseq Class Reference

The guido sequence element.

```
#include <guido.h>
```

Inheritance diagram for guidoseq::



Public Member Functions

- void **accept** (**guidovisitor** &visitor)

Friends

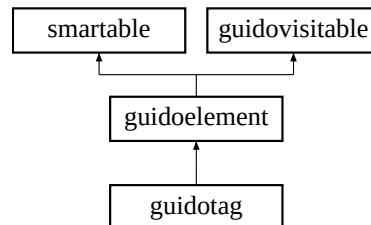
- VEXP friend **SMARTP**< **guidoseq** > **new_guidoseq** ()

9.14 guidotag Class Reference

A guido tag representation.

```
#include <guido.h>
```

Inheritance diagram for guidotag::



Public Member Functions

- void **accept** (`guidovisitor` &visitor)

Protected Member Functions

- **guidotag** (string name)

Friends

- VEXP friend **SMARTP**< **guidotag** > **new_guidotag** (string name)

9.14.1 Detailed Description

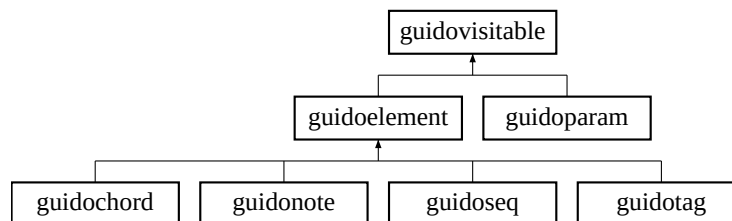
A tag is represented by its name and optional parameters. A range tag contains enclosed elements.

9.15 guidovisible Class Reference

interface for visitable guido objects.

```
#include <guidovisitor.h>
```

Inheritance diagram for guidovisible::



Public Member Functions

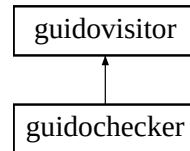
- virtual void **accept** (**guidovisitor** &visitor)=0

9.16 guidovisitor Class Reference

an abstract generic guido visitor

```
#include <guidovisitor.h>
```

Inheritance diagram for guidovisitor::



Public Member Functions

- virtual void **visite** (Sguidoelement &elt)=0
- virtual void **visite** (Sguidonote &elt)=0
- virtual void **visite** (Sguidoseq &elt)=0
- virtual void **visite** (Sguidochord &elt)=0
- virtual void **visite** (Sguidotag &elt)=0
- virtual void **visite** (Sguidoparam &elt)=0

9.17 LineType Class Reference

provides conversions between numeric line types and strings

```
#include <conversions.h>
```

Public Types

- enum **type** {
 undefined, **solid**, **dashed**, **dotted**,
 wavy, **last** = wavy }

Static Public Member Functions

- static const string **xml** (type d)
 convert a numeric start-stop value to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric start-stop value

9.18 NoteType Class Reference

provides conversions between numeric note types and strings

```
#include <conversions.h>
```

Public Types

- enum **type** {
 undefined, **t256th** = 1, **t128th** = 1<<1, **t64th** = 1<<2,
 t32nd = 1<<3, **t16th** = 1<<4, **eighth** = 1<<5, **quarter** = 1<<6,
 half = 1<<7, **whole** = 1<<8, **breve** = 1<<9, **tlong** = 1<<10,
 count = 11 }

Static Public Member Functions

- static **TRational** & **rational** (type d, **TRational** &r)
 convert an integer note to a rational representation
- static const string **xml** (type d)
 convert an integer note type to a MusicXML string
- static type **xml** (const string str)
 convert an MusicXML string to an integer note type

9.18.1 Detailed Description

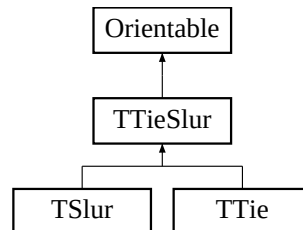
Type indicates the graphic note type. Valid values (from shortest to longest) are 256th, 128th, 64th, 32nd, 16th, eighth, quarter, half, whole, breve, and long.

9.19 Orientable Class Reference

base class for all elements that have an orientation.

```
#include <common.h>
```

Inheritance diagram for Orientable::



Public Member Functions

- `TOrientation & orientation ()`

9.20 pairmap Class Template Reference

implements a multimap where <key, value> pairs are unique

```
#include <bimap.h>
```

Public Types

- typedef pairmap< T1, T2 >::iterator **iterator**
- typedef pairmap< T1, T2 >::const_iterator **const_iterator**

Public Member Functions

- iterator **insert** (const pair< T1, T2 > &x)
- iterator **insert** (iterator position, const pair< T1, T2 > &x)

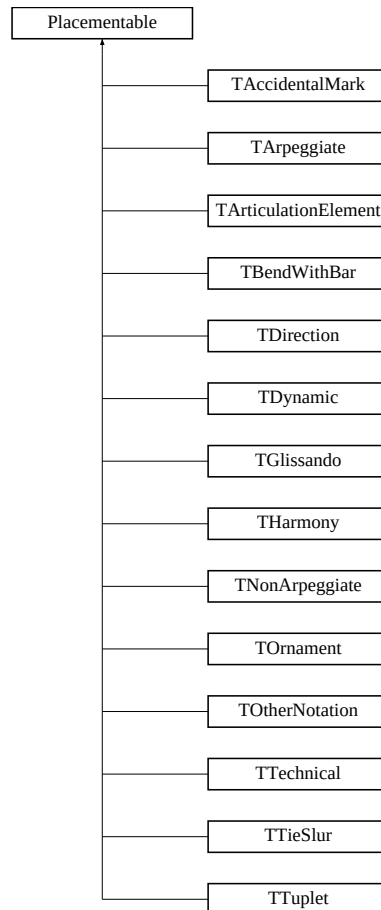
```
template<typename T1, typename T2> class pairmap< T1, T2 >
```

9.21 Placementable Class Reference

base class for all elements that have a placement.

```
#include <common.h>
```

Inheritance diagram for Placementable::



Public Member Functions

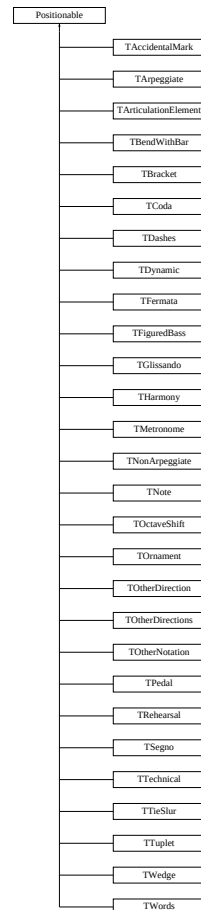
- **TPlacement** & **placement** ()

9.22 Positionable Class Reference

base class for all elements that have a position.

```
#include <common.h>
```

Inheritance diagram for Positionable::



Public Member Functions

- `TPosition & position ()`

Protected Member Functions

- **smartable** (const **smartable** &)
- virtual **~smartable** ()
destructor checks for non-zero refCount
- **smartable** & **operator=** (const **smartable** &)

9.23.1 Detailed Description

Any object that want to support smart pointers should inherit from the smartable class which provides reference counting and automatic delete when the reference count drops to zero.

9.24 SMARTP Class Template Reference

the smart pointer implementation

```
#include <smartpointer.h>
```

Public Member Functions

- **SMARTP ()**
an empty constructor - points to null
- **SMARTP (T *rawptr)**
build a smart pointer from a class pointer
- **template<class T2> SMARTP (const SMARTP< T2 > &ptr)**
build a smart pointer from an convertible class reference
- **SMARTP (const SMARTP &ptr)**
build a smart pointer from another smart pointer reference
- **~SMARTP ()**
the smart pointer destructor: simply removes one reference count
- **operator T * () const**
cast operator to retrieve the actual class pointer
- **T & operator * () const**
'' operator to access the actual class pointer*
- **T * operator → () const**
operator → overloading to access the actual class pointer
- **template<class T2> SMARTP & operator= (T2 p1_)**
operator = that moves the actual class pointer
- **SMARTP & operator= (T *p_)**
operator = that moves the actual class pointer
- **SMARTP & operator= (const SMARTP< T > &p_)**
operator = to support inherited class reference
- **template<class T2> SMARTP & cast (T2 *p_)**
dynamic cast support
- **template<class T2> SMARTP & cast (const SMARTP< T2 > &p_)**
dynamic cast support

9.24.1 Detailed Description

template<class T> class SMARTP< T >

A smart pointer is in charge of maintaining the objects reference count by the way of pointers operators overloading. It supports class inheritance and conversion whenever possible.

Instances of the **SMARTP** (p. 62) class are supposed to use *smartable* types (or at least objects that implements the *addReference* and *removeReference* methods in a consistent way).

9.25 StartStop Class Reference

provides conversions between numeric start-stop types and strings

```
#include <conversions.h>
```

Public Types

- enum **type** {
 undefined, **start**, **stop**, **cont**,
 last = **cont** }

Static Public Member Functions

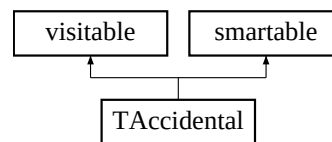
- static const string **xml** (type d)
 convert a numeric start-stop value to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric start-stop value

9.26 TAccidental Class Reference

Representation of an accidental.

```
#include <TNote.h>
```

Inheritance diagram for TAccidental::



Public Types

- enum **accident** {
 undefined = -1, **sharp** = 1, **natural**, **flat**,
 double_sharp, **sharp_sharp**, **flat_flat**, **natural_sharp**,
 natural_flat, **quarter_flat**, **quarter_sharp**, **three_quarters_flat**,
 three_quarters_sharp, **last** = three_quarters_sharp }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- accident **getAccident** () const
- YesNo::type **getCautionary** () const
- YesNo::type **getEditorial** () const
- FullCue::type **getSize** () const
- void **setAccident** (accident acc)
- void **setCautionary** (YesNo::type yn)
- void **setEditorial** (YesNo::type yn)
- void **setSize** (FullCue::type size)

Static Public Member Functions

- static const string **xmlaccidental** (accident d)
 convert a numeric accidental type to a MusicXML string
- static accident **xmlaccidental** (const string str)
 convert an MusicXML string to a numeric accidental type

Friends

- EXP friend SMARTP< TAccidental > **newAccidental** ()

9.26.1 Detailed Description

Actual notated accidentals. Valid values include: sharp, natural, flat, double-sharp, sharp-sharp, flat-flat, natural-sharp, natural-flat, quarter-flat, quarter-sharp, three-quarters-flat, and three-quarters-sharp. Editorial and cautionary indications, are indicated by attributes. Values for these attributes are "no" if not present.

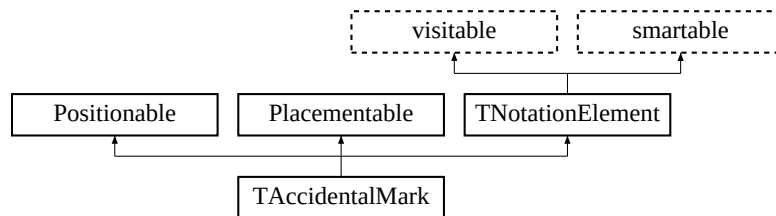
The MusicXML *accidental* element is defined in note.dtd.

9.27 TAccidentalMark Class Reference

an accidental-mark

```
#include <TOrnaments.h>
```

Inheritance diagram for TAccidentalMark::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- const string & **getData** () const

Friends

- EXP friend SMARTP< TAccidentalMark > newAccidentalMark ()

9.27.1 Detailed Description

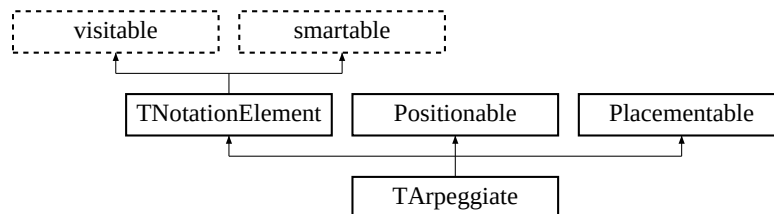
An accidental-mark can be used as a separate notation or as part of an ornament. When used in an ornament, position and placement are relative to the ornament, not relative to the note.

9.28 TArpeggiate Class Reference

MusicXML *arpeggiate* representation.

```
#include <TArpeggiate.h>
```

Inheritance diagram for TArpeggiate::



Public Types

- enum **direction** { **undefined**, **up**, **down** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setNumber** (int n)
- void **setDirection** (direction dir)
- int **getNumber** () const
- direction **getDirection** () const

Friends

- EXP friend SMARTP< TArpeggiate > newArpeggiate ()

9.28.1 Detailed Description

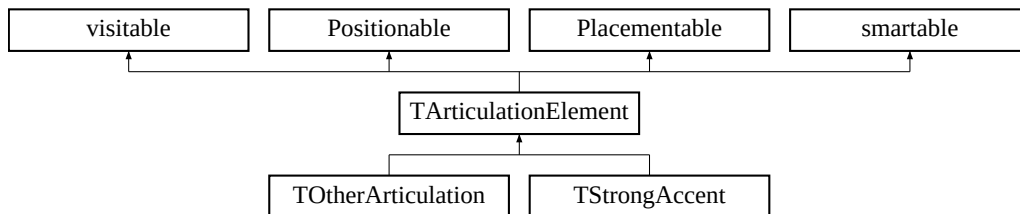
Indicates that this note is part of an arpeggiated chord. The number attribute can be used to distinguish between two simultaneous chords arpeggiated separately (different numbers) or together (same number). The up-down attribute is used if there is an arrow on the arpeggio sign. By default, arpeggios go from the lowest to highest note.

9.29 TArticulationElement Class Reference

articulations and accents definition.

```
#include <TNotation.h>
```

Inheritance diagram for TArticulationElement::



Public Types

- enum **articulation** {
first = 1, **accent** = first, **strong_accent**, **staccato**,
tenuto, **detached_legato**, **staccatissimo**, **spiccato**,
scoop, **plop**, **doit**, **falloff**,
breath_mark, **caesura**, **other_articulation**, **last** = other_articulation }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- articulation **getArtType** () const
- void **setArtType** (articulation a)

Static Public Member Functions

- static const string **xmlname** (articulation d)
convert a numeric articulation name to a MusicXML string
- static articulation **xmlname** (const string str)
convert an MusicXML string to a numeric articulation name
- static const string * **getArticulationStrings** ()

Protected Member Functions

- TArticulationElement** (articulation type)

Friends

- EXP friend SMARTP< TArticulationElement > **newArticulationElement** (articulation type)

9.29.1 Detailed Description

Articulations elements are all handled by a single object with a type field used to specify the articulation type. All the articulations have a position and placement attributes. Apart the *other-articulation* element, all are empty elements.

TArticulation handles a set of elements defined in note.dtd.

Todo

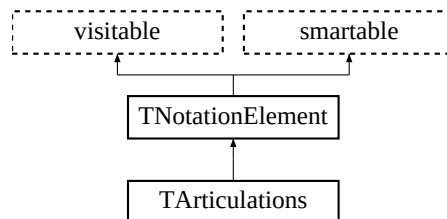
- support of the line-shape and line-type attributes

9.30 TArticulations Class Reference

The MusicXML *articulations* element.

```
#include <TNotation.h>
```

Inheritance diagram for TArticulations::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SArticulationElement &art)
- vvector< SArticulationElement > & **articulations** ()

Friends

- EXP friend SMARTP< TArticulations > **newArticulations** ()

9.30.1 Detailed Description

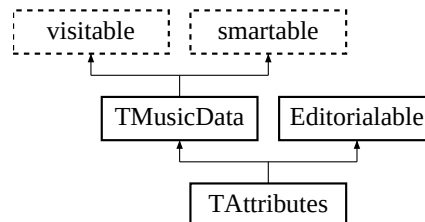
Defined in note.dtd.

9.31 TAttributes Class Reference

Contains information that typically changes on measure boundaries.

```
#include <TAttributes.h>
```

Inheritance diagram for TAttributes::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDivisions** (long div)
- void **setStaves** (long staves)
- void **setInstrument** (long inst)
- long **getDivision** () const
- long **getStaves** () const
- long **getInstrument** () const
- SKey & **key** ()
- STimeSign & **timeSign** ()
- STranspose & **transpose** ()
- SDirective & **directive** ()
- SStaffDetails & **staffDetails** ()
- SMeasureStyle & **measureStyle** ()
- vvector< SClef > & **clefs** ()
- long **add** (const SClef &clef)

Friends

- EXP friend SMARTP< TAttributes > **newAttributes** ()

9.31.1 Detailed Description

It includes key and time signatures, clefs, transpositions, and staving. **TAttributes** (p. 72) corresponds to the MusicXML *attribute* element as defined in attributes.dtd.

Todo

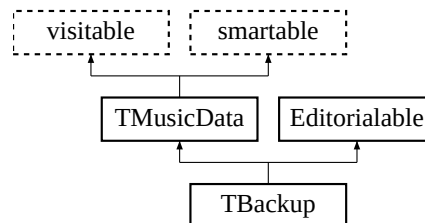
- clarify the Instrument type and semantic

9.32 TBackup Class Reference

Represents the MusicXML backup element.

```
#include <TBackupForward.h>
```

Inheritance diagram for TBackup::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDuration** (long dur)
Duration is a required element and is set to zero by the constructor.
- long **getDuration** () const

Friends

- EXP friend SMARTP< TBackup > newBackup ()

9.32.1 Detailed Description

Backup and forward are required to coordinate multiple voices in one part, including music on a grand staff. Backup is generally used to move between voices and staves. Thus the backup element does not include voice or staff elements. Duration values should always be positive, and should not cross measure boundaries.

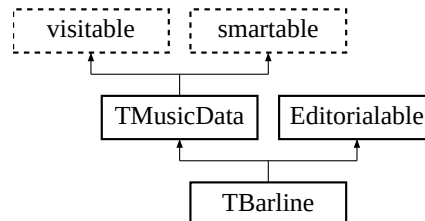
The MusicXML *backup* element is defined in note.dtd.

9.33 TBarline Class Reference

Represents a MusicXML barline element.

```
#include <TBarline.h>
```

Inheritance diagram for TBarline::



Public Types

- enum {
 undefined = 0, **right** = 1, **left**, **middle**,
 lastloc = middle }
- enum {
 none = 1, **regular**, **dotted**, **heavy**,
 light_light, **light_heavy**, **heavy_light**, **heavy_heavy**,
 last = heavy_heavy }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setLocation** (int loc)
- void **setBarStyle** (int style)
- int **getLocation** () const
- int **getBarStyle** () const
- SWavyLine & wavyLine ()
- SSegno & segno ()
- SCoda & coda ()
- SFermata & fermata ()
- SEnding & ending ()
- SRepeat & repeat ()

Static Public Member Functions

- static const string **xmllocation** (int d)
 convert an integer location to a MusicXML string
- static int **xmllocation** (const string str)
 convert an MusicXML string to an integer location

- static const string **xmlstyle** (int d)
convert an integer bar style to a MusicXML string
- static int **xmlstyle** (const string str)
convert an MusicXML string to an integer bar style

Friends

- EXP friend SMARTP< TBarline > newBarline ()

9.33.1 Detailed Description

If a barline is other than a normal single barline, it should be represented by a barline element that describes it. This includes information about repeats and multiple endings, as well as line style. The two fermata elements allow for fermatas on both sides of the barline (the lower one inverted).

Barlines have a location attribute. It must match where the barline element occurs within the rest of the musical data in the score. If location is left, it should be the first element in the measure; if location is right, it should be the last element. If no location is specified, the right barline is the default.

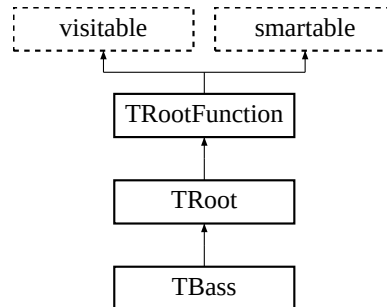
The MusicXML *barline* element is defined in barline.dtd.

9.34 TBass Class Reference

the bass element of the *harmony-chord* entity.

```
#include <THarmony.h>
```

Inheritance diagram for TBass::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

- EXP friend SMARTP< TBass > newBass ()

9.34.1 Detailed Description

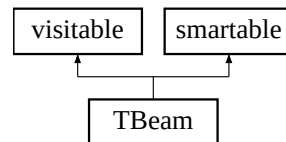
Bass is used to indicate a bass note in popular music chord symbols, e.g. G/C. It is generally not used in functional harmony, as inversion is generally not used in pop chord symbols. As with root, it is divided into step and alter elements, similar to pitches.

9.35 TBeam Class Reference

beaming representation.

```
#include <TNote.h>
```

Inheritance diagram for TBeam::



Public Types

- enum **beamtype** {
 undefined = -1, **beambegin** = 1, **beamcontinue**, **beamend**,
 beamforward, **beambackward**, **last** = beambackward }

Public Member Functions

- beamtype **getType** () const
- int **getNumber** () const
- YesNo::type **getRepeater** () const
- void **setType** (beamtype type)
- void **setNumber** (int num)
- void **setRepeater** (YesNo::type r)
- virtual void **accept** (TScoreVisitor &visitor)

Static Public Member Functions

- static const string **xmlbeamtype** (beamtype d)
 convert a numeric beam type to a MusicXML string
- static beamtype **xmlbeamtype** (const string str)
 convert an MusicXML string to a numeric beam type

Friends

- EXP friend **SMARTP< TBeam > newBeam** ()

9.35.1 Detailed Description

Beam types include begin, continue, end, forward hook, and backward hook. In MuseData, up to six concurrent beams are available to cover up to 256th notes. This seems sufficient so we use an enumerated

type defined in common.dtd. The repeater attribute needs to be specified with a "yes" value for each beam using it.

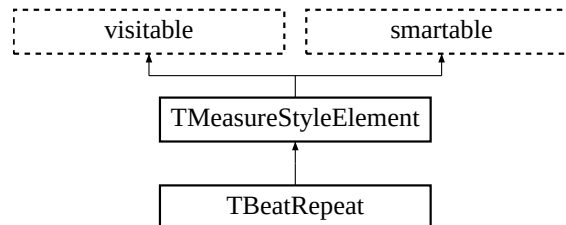
The MusicXML *beam* element as defined in note.dtd.

9.36 TBeatRepeat Class Reference

Represents beat repeat (for measure style).

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TBeatRepeat::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setUseDots** (YesNo::type t)
- void **setSlashes** (int n)
- void **setStartStop** (StartStop::type t)
- int **getSlashes** () const
- YesNo::type **getUseDots** () const
- StartStop::type **getStartStop** () const

Protected Member Functions

- **TBeatRepeat** (StartStop::type t)

Friends

- EXP friend **SMARTP**< **TBeatRepeat** > **newBeatRepeat** (StartStop::type type=Start-Stop::undefined)

9.36.1 Detailed Description

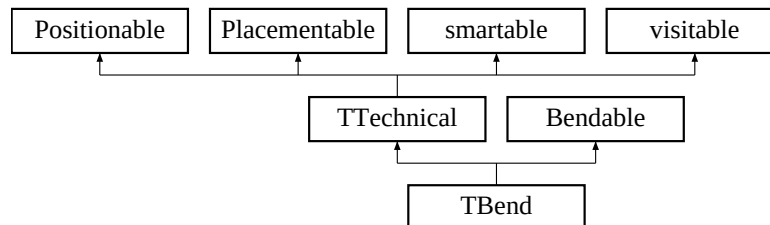
The beat-repeat element is used to indicate that a single beat (but possibly many notes) is repeated. Both the start and stop of the beat being repeated should be specified. The slashes attribute specifies the number of slashes to use in the symbol. The use-dots attribute indicates whether or not to use dots as well (for instance, with mixed rhythm patterns). By default, the value for slashes is 1 and the value for use-dots is no.

9.37 TBend Class Reference

MusicXML bend element.

```
#include <TTechnical.h>
```

Inheritance diagram for TBend::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setAlter** (float alter)
- void **setPreBend** (bool v)
- void **setRelease** (bool v)
- void **setWithBar** (SBendWithBar wb)
- float **getAlter** () const
- bool **getPreBend** () const
- bool **getRelease** () const
- SBendWithBar **getWithBar** ()

Friends

- EXP friend SMARTP< TBend > **newBend** ()

9.37.1 Detailed Description

The bend element is used in guitar and tablature. The bend-alter element indicates the number of steps in the bend, similar to the alter element. As with the alter element, numbers like 0.5 can be used to indicate microtones. Negative numbers indicate pre-bends or releases; the pre-bend and release elements are used to distinguish what is intended. A with-bar element indicates that the bend is to be done at the bridge with a whammy or vibrato bar. The content of the element indicates how this should be notated.

9.38 TBendSound Class Reference

Represents the bend-sound entity.

```
#include <common.h>
```

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setAccelerate** (YesNo::type yn)
- void **setBeat** (int beat)
- void **setFirstBeat** (int beat)
- void **setLastBeat** (int beat)
- YesNo::type **getAccelerate** () const
- int **getBeat** () const
- int **getFirstBeat** () const
- int **getLastBeat** () const

9.38.1 Detailed Description

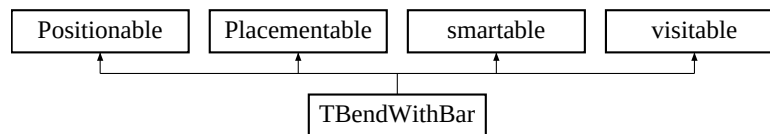
The MusicXML *bend-sound* entity is defined in common.dtd

9.39 TBendWithBar Class Reference

MusicXML with-bar (from bend) element.

```
#include <TTechnical.h>
```

Inheritance diagram for TBendWithBar::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const std::string &data)
- const std::string & **getData** () const

Friends

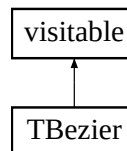
- EXP friend SMARTP< TBendWithBar > newBendWithBar ()

9.40 TBezier Class Reference

Represents the MusicXML *bezier* entity.

```
#include <common.h>
```

Inheritance diagram for TBezier::



Public Types

- enum { **undefined** = 0x7fff }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setOffset** (float offset)
- void **setOffset2** (float offset)
- void **setX** (float x)
- void **setY** (float y)
- void **setX2** (float x)
- void **setY2** (float y)
- float **getOffset** () const
- float **getOffset2** () const
- float **getX** () const
- float **getY** () const
- float **getX2** () const
- float **getY2** () const

9.40.1 Detailed Description

The bezier entity is used to indicate the curvature of slurs. Normal slurs and S-shaped slurs need only two bezier points: one associated with the start of the slur, the other with the stop. Complex slurs and slurs divided over system breaks can specify bezier data at slur elements with a continue type.

The bezier-offset, bezier-x, and bezier-y attributes describe the outgoing bezier point for slurs with a start type, and the incoming bezier point for slurs with types of continue or stop. The attributes bezier-offset2, bezier-x2, and bezier-y2 are only valid with slurs of type continue, and describe the outgoing bezier point.

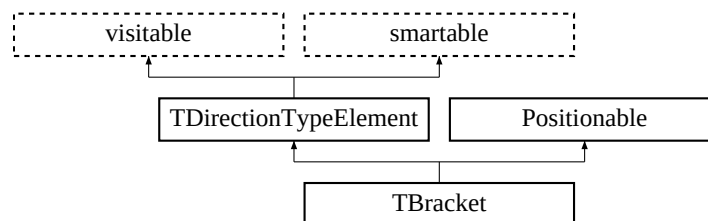
The bezier-offset and bezier-offset2 attributes are measured in terms of musical divisions, like the offset element. These are the recommended attributes for specifying horizontal position. The other attributes are specified in tenths, relative to any position settings associated with the slur element. It is defined in common.dtd.

9.41 TBracket Class Reference

The MusicXML *bracket* element.

```
#include <TDirection.h>
```

Inheritance diagram for TBracket::



Public Types

- enum {
undefined = -1, **up** = 1, **down**, **both**,
arrow, **none**, **last** = none }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
- void **setNumber** (int num)
- void **setLineEnd** (int lend)
- void **setLineType** (LineType::type t)
- StartStop::type **getType** () const
- int **getNumber** () const
- int **getLineEnd** () const
- LineType::type **getLineType** () const

Static Public Member Functions

- static const string **xmllineend** (int d)
convert an numeric line end value to a MusicXML string
- static int **xmllineend** (const string str)
convert an MusicXML string to a numeric line end value

Protected Member Functions

- TBracket** (StartStop::type t)

Friends

- EXP friend **SMARTP**< **TBracket** > **newBracket** (StartStop::type t=StartStop::undefined)

9.41.1 Detailed Description

Brackets are combined with words in a variety of modern directions. The line-end attribute specifies if there is a jog up or down (or both), an arrow, or nothing at the start or end of the bracket. The line-type is solid by default.

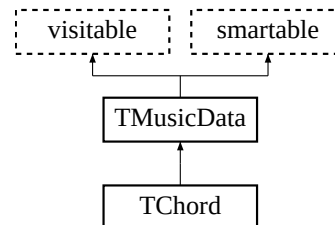
The MusicXML *bracket* element is defined in direction.dtd.

9.42 TChord Class Reference

a notes container that denotes a chord.

```
#include <TChord.h>
```

Inheritance diagram for TChord::



Public Types

- enum **sorttype** { **pitch**, **duration** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SNote ¬e)
- **vvector**< SNote > & **notes** ()
- unsigned long **getDuration** () const
returns the chord duration that is the shortest note duration (or 0 for an empty chord)
- long **getVoice** () const
returns the voice of the chord notes
- long **getStaff** () const
returns the staff of the first chord note
- TNote::style **getStyle** () const
returns the style of the chord notes (normal, cue, grace)
- void **sort** (sorttype type)
sorts the notes of the chord

Friends

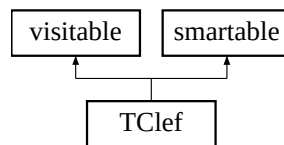
- EXP friend **SMARTP**< TChord > **newChord** ()

9.43 TClef Class Reference

Represents a clef.

```
#include <TClef.h>
```

Inheritance diagram for TClef::



Public Types

- enum {
 undefined = -1, **none** = 1, **G**, **F**,
 C, **percussion**, **TAB**, **last** = TAB }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setStaffNum** (long num)
- void **setSign** (long sign)
- void **setLine** (long line)
- void **setOctave** (long oct)
- long **getStaffNum** () const
- long **getSign** () const
- long **getLine** () const
- long **getOctave** () const

Static Public Member Functions

- static const string **xmlsign** (int d)
convert an integer clef sign to a MusicXML string
- static int **xmlsign** (const string str)
convert an MusicXML string to an integer clef sign

Friends

- EXP friend SMARTP< TClef > **newClef** ()

9.43.1 Detailed Description

Clefs are represented by the sign, line, and clef-octave-change elements. Sign values include G, F, C, percussion, TAB, and none. Line numbers are counted from the bottom of the staff. Standard values are 2 for the G sign (treble clef), 4 for the F sign (bass clef), 3 for the C sign (alto clef) and 5 for TAB (on a 6-line staff). The clef-octave-change element is used for transposing clefs (e.g., a treble clef for tenors would have a clef-octave-change value of -1). The optional number attribute refers to staff numbers, from top to bottom on the system. A value of 1 is assumed if not present.

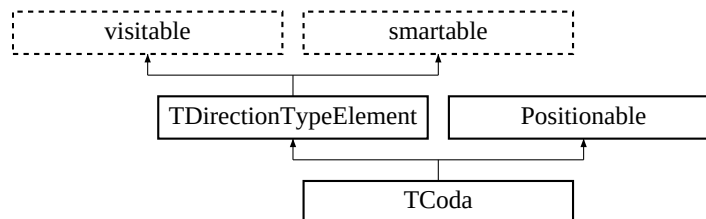
TClef (p. 87) corresponds to the MusicXML *clef* element as defined in attributes.dtd

9.44 TCoda Class Reference

Represents the MusicXML *coda* element.

```
#include <common.h>
```

Inheritance diagram for TCoda::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

- EXP friend SMARTP< TCoda > newCoda ()

9.44.1 Detailed Description

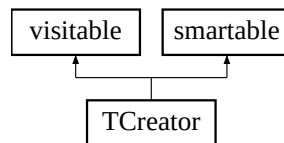
TCoda (p. 89) is a visual indicator only. It is defined in common.dtd.

9.45 TCreator Class Reference

Represents the author(s) of the score .

```
#include <TIdentification.h>
```

Inheritance diagram for TCreator::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (const std::string &type)
- void **setName** (const std::string &name)
- const std::string & **getType** () const
- const std::string & **getName** () const

Friends

- EXP friend SMARTP< TCreator > newCreator ()

9.45.1 Detailed Description

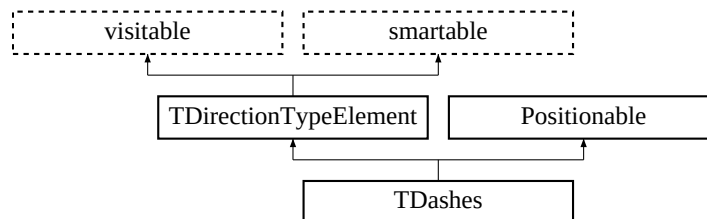
The type attribute can be used to distinguish words and music so as to represent lyricists and librettists as well as composers. Thus, there can be multiple creators in an identification. It corresponds to the MusicXML *creator* element as defined in identity.dtd.

9.46 TDashes Class Reference

Dashes, used for instance with cresc. and dim. marks.

```
#include <TDirection.h>
```

Inheritance diagram for TDashes::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type t)
- void **setNumber** (int num)
- StartStop::type **getType** () const
- int **getNumber** () const

Protected Member Functions

- **TDashes** (StartStop::type t)

Friends

- EXP friend SMARTP< TDashes > **newDashes** (StartStop::type t=StartStop::undefined)

9.46.1 Detailed Description

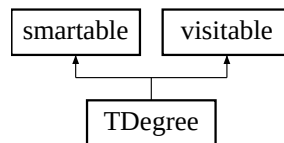
The MusicXML *dashes* element is defined in direction.dtd.

9.47 TDegree Class Reference

the *degree* element of the *harmony-chord* entity.

```
#include <THarmony.h>
```

Inheritance diagram for TDegree::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setValue** (int val)
- void **setAlter** (int val)
- void **setType** (const std::string &alter)
- int **getValue** () const
- int **getAlter** () const
- const std::string & **getType** () const

Friends

- EXP friend SMARTP< TDegree > newDegree ()

9.47.1 Detailed Description

The degree element is used to add, alter, or subtract individual notes in the chord. The degree-value element is a number indicating the degree of the chord (1 for the root, 3 for third, etc). The degree-alter element is like the alter element in notes: 1 for sharp, -1 for flat, etc. The degree-type element can be add, alter, or subtract. If the degree-type is alter or subtract, the degree-alter is relative to the degree already in the chord based on its kind element. If the degree-type is add, the degree-alter is relative to a dominant chord (major and perfect intervals except for a minor seventh).

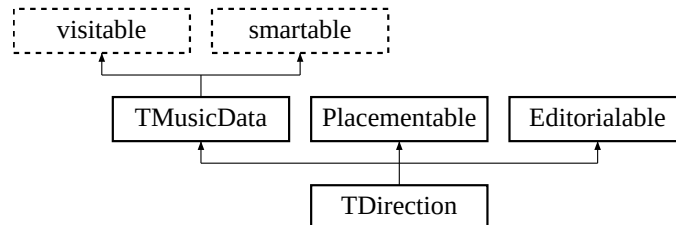
A harmony of kind "other" can be spelled explicitly by using a series of degree elements together with a root.

9.48 TDirection Class Reference

The MusicXML *direction* element.

```
#include <TDirection.h>
```

Inheritance diagram for TDirection::



Public Types

- enum { **undefined** = -1 }
an undefined constant, used for offset, voice and staff

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SDirectionType &dir)
- void **setOffset** (long offset)
- void **setVoice** (long voice)
- void **setStaff** (long staff)
- SSound & **sound** ()
- long **getOffset** () const
- long **getVoice** () const
- long **getStaff** () const
- vvector< SDirectionType > & **types** ()

Friends

- EXP friend SMARTP< TDirection > **newDirection** ()

9.48.1 Detailed Description

A direction is a musical indication that is not attached to a specific note. Two or more may be combined to indicate starts and stops of wedges, dashes, etc.

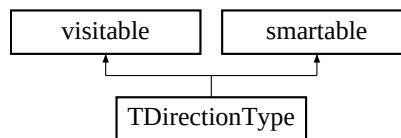
Defined in direction.dtd.

9.49 TDirectionType Class Reference

Represents textual direction indications.

```
#include <TDirection.h>
```

Inheritance diagram for TDirectionType::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SDirectionTypeElement &elt)
- vvector< SDirectionTypeElement > & **elements** ()

Friends

- EXP friend SMARTP< TDirectionType > **newDirectionType** ()

9.49.1 Detailed Description

Textual direction types may have more than 1 component due to multiple font numbers.

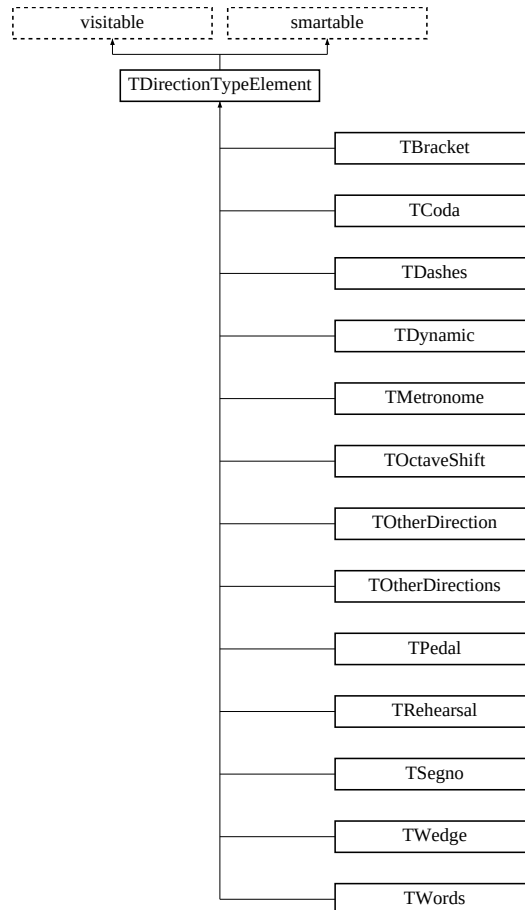
TDirectionType (p. 94) corresponds to the MusicXML *direction-type* element as defined in direction.dtd.

9.50 TDirectionTypeElement Class Reference

base class for all the elements of the *direction-type* element.

```
#include <common.h>
```

Inheritance diagram for TDirectionTypeElement::



9.50.1 Detailed Description

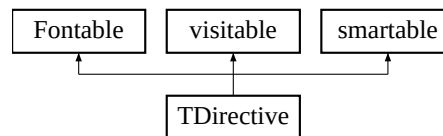
TDirectionTypeElement (p. 95) is an abstract class which purpose is to define a common type for all the elements of a *direction-type* element.

9.51 TDirective Class Reference

Represents musical directives.

```
#include <TAttributes.h>
```

Inheritance diagram for TDirective::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setLang** (const string &lang)
the xml:lang attribute
- void **setValue** (const string &val)
textual directive
- const string & **getLang** () const
- const string & **getValue** () const

Friends

- EXP friend SMARTP< TDirective > newDirective ()

9.51.1 Detailed Description

Directives are like directions, but can be grouped together with attributes for convenience. This is typically used for tempo markings at the beginning of a piece of music. The language is Italian ("it") by default.

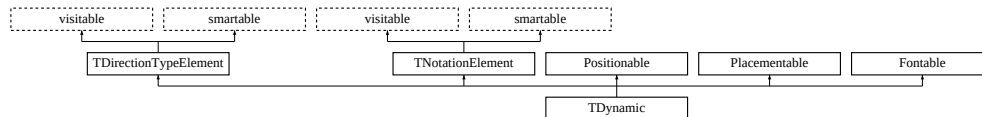
TDirective (p. 96) corresponds to the MusicXML *directive* element.

9.52 TDynamic Class Reference

Represents the MusicXML *dynamics* element.

```
#include <common.h>
```

Inheritance diagram for TDynamic::



Public Types

- enum **dynamic** {
 pppppp = 1, pppppp, ppppp, pppp,
 pp, p, mp, mf,
 f, ff, fff, ffff,
 fffff, fffffff, sf, sfp,
 sfpp, fp, rf, rfz,
 sfz, sffz, fz, last = fz }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- vector< dynamic > & **dynamics** ()
- long **add** (dynamic d)

Static Public Member Functions

- static const string **xmldynamic** (dynamic d)
convert an integer dynamic to a MusicXML string
- static dynamic **xmldynamic** (const string str)
convert an MusicXML string to an integer dynamic
- static const string * **getDynamicStrings** ()

Friends

- EXP friend SMARTP< TDynamic > **newDynamic** ()

9.52.1 Detailed Description

It is defined in common.dtd.

9.53 TEditorial Class Reference

Represents the MusicXml *editorial* and editorial-voice entities.

```
#include <common.h>
```

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- **SFootnote** & **footnote** ()
- **SLevel** & **level** ()
- **SVoice** & **voice** ()

9.53.1 Detailed Description

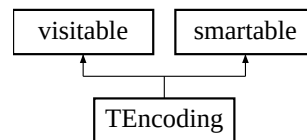
Footnote and level are used to specify editorial information. These elements are used throughout the component MusicXML DTDs. The attributes allow specification of three common ways to indicate the editorial difference: putting parentheses or square brackets around a symbol, or making the symbol cue-size vs. full-size. If not specified, they are left to application defaults. It is defined in common.dtd.

9.54 TEncoding Class Reference

Contains information about encoding.

```
#include <TIdentification.h>
```

Inheritance diagram for TEncoding::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDate** (const std::string &date)
- void **setEncoder** (const std::string &encoder)
- void **setEncoderType** (const std::string &etype)
- void **setSoftware** (const std::string &soft)
- void **setDescription** (const std::string &desc)
- const std::string & **getDate** () const
- const std::string & **getEncoder** () const
- const std::string & **getEncoderType** () const
- const std::string & **getSoftware** () const
- const std::string & **getDescription** () const
- void **add** (SSupport support)
- **vvector**< SSupport > & **support** ()

Friends

- EXP friend SMARTP< TEncoding > **newEncoding** ()

9.54.1 Detailed Description

Encoding contains information about who did the digital encoding, when, with what software, and in what aspects. The supports element indicates if the encoding supports a particular MusicXML element. This is recommended for elements like beam, stem, and accidental, where the absence of an element is ambiguous if you do not know if the encoding supports that element.

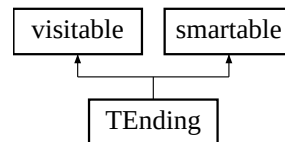
It corresponds to the MusicXML *encoding* element as defined in identity.dtd.

9.55 TEnding Class Reference

Endings refers to multiple (e.g. first and second) endings.

```
#include <TBarline.h>
```

Inheritance diagram for TEnding::



Public Types

- enum { **start** = 1, **stop**, **discontinue**, **last** = discontinue }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setNum** (long num)
- void **setType** (int type)
- long **getNum** () const
- int **getType** () const

Static Public Member Functions

- static const string **xmltype** (int d)
convert an integer ending type to a MusicXML string
- static int **xmltype** (const string str)
convert an MusicXML string to an integer ending type

Protected Member Functions

- **TEnding** (long num, int type)

Friends

- EXP friend SMARTP< TEnding > **newEnding** (long num, int type)

9.55.1 Detailed Description

Typically, the start type is associated with the left barline of the first measure in an ending. The stop and discontinue types are associated with the right barline of the last measure in an ending. Stop is used when

the ending mark concludes with a downward jog, as is the case for first endings. Discontinue is used when there is no downward jog, as in the final ending.

The MusicXML *ending* element is defined in barline.dtd.

Todo

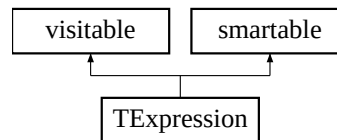
- support of multiple numbers

9.56 TExpression Class Reference

Represents the expression context of a note.

```
#include <TNote.h>
```

Inheritance diagram for TExpression::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDynamic** (long dyn)
reflects the MIDI key on velocity (expressed in terms of percentages of a standard MIDI forte volume of 90)
- void **setEndDynamic** (long dyn)
reflects the MIDI key off velocity (expressed in terms of percentages of a standard MIDI forte volume of 90)
- void **setAttack** (long attack)
used to alter the starting time of the note (expressed in term of divisions)
- void **setRelease** (long rel)
used to alter the stopping time of the note (expressed in term of divisions)
- void **setPizzicato** (YesNo::type pizz)
used when just this note is sounded pizzicato
- long **getDynamic** () const
- long **getEndDynamic** () const
- long **getAttack** () const
- long **getRelease** () const
- YesNo::type **getPizzicato** () const

Friends

- EXP friend SMARTP< TExpression > **newExpression** ()

9.56.1 Detailed Description

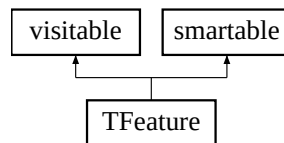
TExpression (p. 102) corresponds to a subset of the *note* attributes as defined in notes.dtd

9.57 TFeature Class Reference

the feature element (used by the grouping element).

```
#include <TScore.h>
```

Inheritance diagram for TFeature::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (const string &type)
- void **setDate** (const string &data)
- const string & **getType** () const
- const string & **getData** () const

Friends

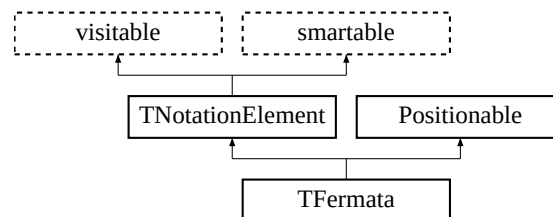
- EXP friend SMARTP< TFeature > newFeature ()

9.58 TFermata Class Reference

Represents the fermata sign.

```
#include <common.h>
```

Inheritance diagram for TFermata::



Public Types

- enum { **undefined**, **upright**, **inverted**, **last** = inverted }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (int type)
- int **getType** () const

Static Public Member Functions

- static const string **xmltype** (int d)
convert an integer type to a MusicXML string
- static int **xmltype** (const string str)
convert an MusicXML string to an integer type

Friends

- EXP friend SMARTP< TFermata > **newFermata** ()

9.58.1 Detailed Description

Fermata elements can be applied both to notes and to measures. Fermata type is upright if not specified.

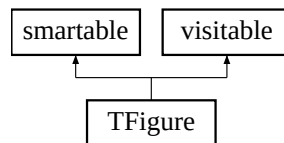
The MusicXML *fermata* element is defined in common.dtd

9.59 TFigure Class Reference

figure for figured bass

```
#include <TFiguredBass.h>
```

Inheritance diagram for TFigure::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setPrefix** (const std::string &data)
- void **setNumber** (const std::string &data)
- void **setSuffix** (const std::string &data)
- const std::string & **getPrefix** () const
- const std::string & **getNumber** () const
- const std::string & **getSuffix** () const

Friends

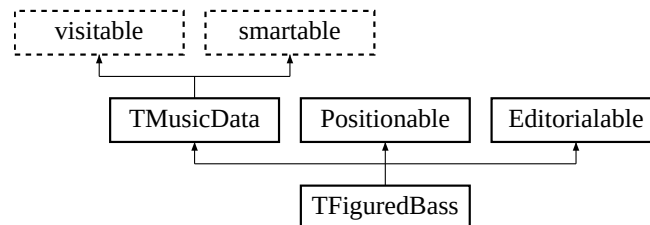
- EXP friend SMARTP< TFigure > **newFigure** ()

9.60 TFiguredBass Class Reference

figured bass

```
#include <TFiguredBass.h>
```

Inheritance diagram for TFiguredBass::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDuration** (unsigned long d)
- unsigned long **getDuration** () const
- long **add** (const SFigure &fig)
- **vvector**< SFigure > & **figures** ()
- TPrintout & **printout** ()

Friends

- EXP friend SMARTP< TFiguredBass > **newFiguredBass** ()

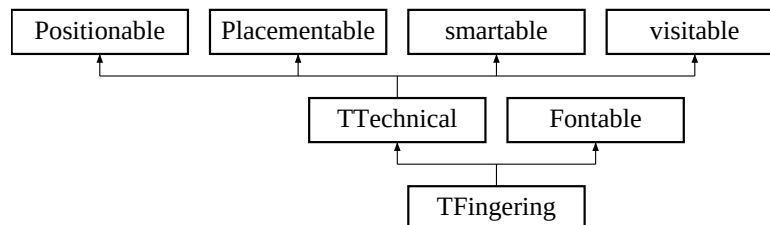
9.60.1 Detailed Description

Figured bass elements take their position from the first regular note that follows. We will need to clarify these definitions over time.

9.61 TFingering Class Reference

```
#include <TEchnical.h>
```

Inheritance diagram for TFingering::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setSubstitution** (YesNo::type sub)
- void **setAlternate** (YesNo::type sub)
- void **setData** (const std::string &data)
- YesNo::type **getSubstitution** () const
- YesNo::type **getAlternate** () const
- const std::string **getData** () const

Friends

- EXP friend SMARTP< TFingering > newFingering ()

9.61.1 Detailed Description

the *fingering* element.

Fingering is typically indicated 1,2,3,4,5. Multiple fingerings may be given, typically to substitute fingerings in the middle of a note. The substitution and alternate values are "no" if the attribute is not present.

9.62 TFont Class Reference

Represents the MusicXml *font* entity.

```
#include <common.h>
```

Public Member Functions

- void **accept** (TScoreVisitor &visitor)
- void **setFamily** (const std::string &family)
- void **setStyle** (const std::string &style)
- void **setSize** (const std::string &size)
- void **setWeight** (const std::string &weight)
- const std::string & **getFamily** () const
- const std::string & **getStyle** () const
- const std::string & **getSize** () const
- const std::string & **getWeight** () const

9.62.1 Detailed Description

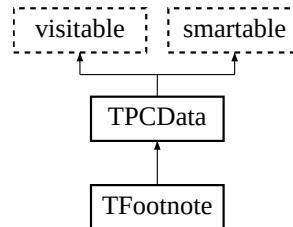
It is defined in common.dtd.

9.63 TFootnote Class Reference

the footnote elements.

```
#include <common.h>
```

Inheritance diagram for TFootnote::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

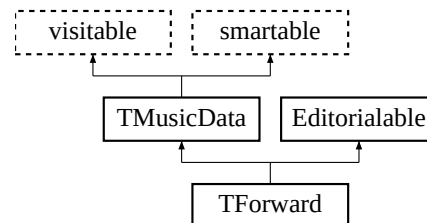
- EXP friend SMARTP< TFootnote > newFootnote ()

9.64 TForward Class Reference

Represents the MusicXML forward element.

```
#include <TBackupForward.h>
```

Inheritance diagram for TForward::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- **TForward** & **setDuration** (long dur)
Duration is a required element and is set to zero by the constructor.
- **TForward** & **setVoice** (long voice)
- **TForward** & **setStaff** (long staff)
- long **getDuration** () const
- long **getVoice** () const
- long **getStaff** () const

Friends

- EXP friend SMARTP< **TForward** > **newForward** ()

9.64.1 Detailed Description

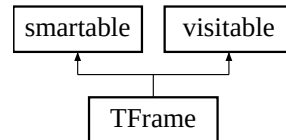
Backup and forward are required to coordinate multiple voices in one part, including music on a grand staff. Forward is generally used within voices and staves. Duration values should always be positive, and should not cross measure boundaries.

The MusicXML *forward* element is defined in note.dtd.

9.65 TFrame Class Reference

```
#include <THarmony.h>
```

Inheritance diagram for TFrame::



Public Types

- enum { **undefined** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setStrings** (int n)
- void **setFrets** (int n)
- void **setFirstFret** (int n)
- int **getStrings** () const
- int **getFrets** () const
- int **getFirstFret** () const
- void **add** (SFrameNote fn)
- **vvector**< SFrameNote > & **frameNotes** ()

Friends

- EXP friend SMARTP< TFrame > **newFrame** ()

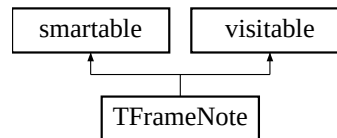
9.65.1 Detailed Description

a frame element used in the *harmony* element.

9.66 TFrameNote Class Reference

```
#include <THarmony.h>
```

Inheritance diagram for TFrameNote::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setString** (const std::string str)
- void **setFret** (const std::string str)
- const std::string & **getString** () const
- const std::string & **getFret** () const
- **SFingering** & **fingering** ()

Friends

- EXP friend SMARTP< TFrameNote > newFrameNote ()

9.66.1 Detailed Description

a frame-note used in the *harmony* element.

Todo

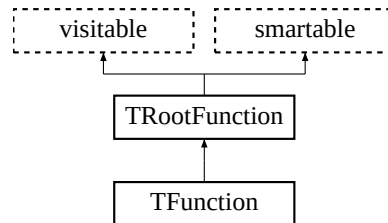
: use the TString and TFret classes (**TTechnical.h** (p. ??))

9.67 TFunction Class Reference

the function element of the *harmony-chord* entity.

```
#include <THarmony.h>
```

Inheritance diagram for TFunction::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setFunction** (const std::string &function)
- const std::string & **getFunction** () const

Friends

- EXP friend SMARTP< TFunction > **newFunction** ()

9.67.1 Detailed Description

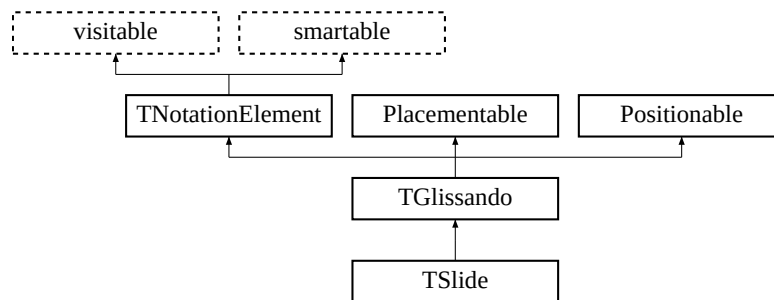
A function is an indication like I, II, III. Function is generally used with classical functional harmony. . Function requires that the key be specified in the encoding.

9.68 TGlissando Class Reference

glissando representation

```
#include <TNotation.h>
```

Inheritance diagram for TGlissando::



Public Types

- enum {
 undefined = -1, **actual** = 1, **both**, **none**,
 last = none }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
 the required type attribute
- void **setNumber** (int number)
- void **setLineType** (LineType::type type)
- StartStop::type **getType** () const
- int **getNumber** () const
- LineType::type **getLineType** () const

Protected Member Functions

- TGlissando (StartStop::type type)

Friends

- EXP friend SMARTP< TGlissando > **newGlissando** (StartStop::type type=StartStop::undefined)

9.68.1 Detailed Description

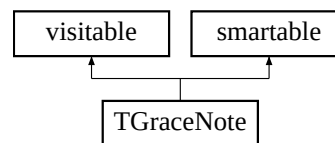
Glissando and slide elements both indicate rapidly moving from one pitch to the other so that individual notes are not discerned. The distinction is similar to that between NIFF's glissando and portamento elements. A glissando sounds the half notes in between the slide and defaults to a wavy line. A slide is continuous between two notes and defaults to a solid line.

9.69 TGraceNote Class Reference

grace note representation.

```
#include <TNote.h>
```

Inheritance diagram for TGraceNote::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- int **getStealPrevious** () const
- int **getStealFollowing** () const
- int **getMakeTime** () const
- YesNo::type **getSlash** () const
- void **setStealPrevious** (int v)
- void **setStealFollowing** (int v)
- void **setMakeTime** (int v)
- void **setSlash** (YesNo::type v)

Friends

- EXP friend SMARTP< TGraceNote > newGraceNote ()

9.69.1 Detailed Description

The slash attribute for a grace note is yes for slashed eighth notes. The other grace note attributes come from MuseData sound suggestions. Steal-time-previous indicates the percentage of time to steal from the previous note for the grace note. Steal-time-following indicates the percentage of time to steal from the following note for the grace note. Make-time indicates to make time, not steal time; the units are in real-time divisions for the grace note.

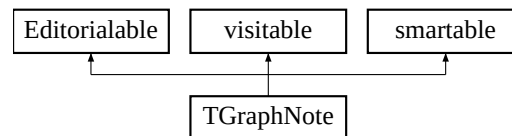
The MusicXML *grace* element is defined in note.dtd.

9.70 TGraphNote Class Reference

Represents the graphic elements of a note.

```
#include <TNote.h>
```

Inheritance diagram for TGraphNote::



Public Types

- enum {
undefined = -1, **stemdown** = 1, **stemup**, **stemnone**,
stemdouble, **last** = stemdouble }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setVoice** (long voice)
the corresponding note voice
- void **setType** (NoteType::type type)
the graphical note sign (whole, half, quarter, ...) corresponds to the type element
- void **setSize** (FullCue::type size)
the note size (full or cue)
- void **setDots** (unsigned long dots)
the number of dots
- void **setStem** (long stem)
stems can be down, up, none, or double.
- void **setStaff** (long staff)
staff assignment (only needed for music notated on the grand staff)
- long **getVoice** () const
- NoteType::type **getType** () const
- FullCue::type **getSize** () const
- unsigned int **getDots** () const
- int **getStem** () const
- int **getStaff** () const
- long **add** (const SBeam &beam)
- long **add** (const SNotation ¬ation)

- **SAccidental & accidental ()**
graphical accidentals
- **STimeModification & timemodification ()**
graphical element for tuplets
- **SNoteHead & notehead ()**
the shape of the note head
- **vvector< SBeam > & beamList ()**
beaming definition
- **vvector< SNotation > & notationList ()**
notations elements (accents, slurs, etc...)
- **TPrintout & printout ()**
the printout entity

Static Public Member Functions

- static const string **xmlstem** (int d)
convert an integer stem attribute to a MusicXML string
- static int **xmlstem** (const string str)
convert an MusicXML string to an integer stem attribute

Friends

- EXP friend **SMARTP< TGraphNode > newGraphNode ()**

9.70.1 Detailed Description

Graphic elements of a note represents the note type (whole, quarter, etc...), possible dots, accidentals, tuplets notation, stems, notehead, staff, beams and other notation elements.

Todo

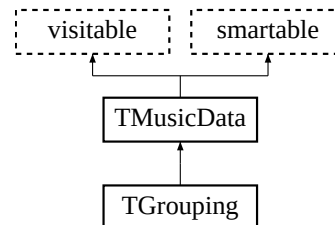
the MusicXML stem definition includes now a position attribute and should be implemented as a separate element

9.71 TGrouping Class Reference

the grouping element.

```
#include <TScore.h>
```

Inheritance diagram for TGrouping::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
- void **setNumber** (long number)
- void **setMemberOf** (const string &data)
- void **add** (const SFeature &feature)
- const StartStop::type **getType** () const
- const string & **getMemberOf** () const
- long **getNumber** () const
- vvector< SFeature > & **getFeatures** ()

Protected Member Functions

- **TGrouping** (StartStop::type type)

Friends

- EXP friend SMARTP< TGrouping > **newGrouping** (StartStop::type type)

9.71.1 Detailed Description

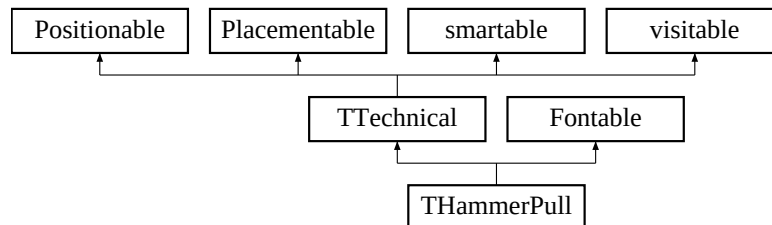
The grouping element is used for musical analysis. When the element type is "start", it usually contains one or more feature elements. The number attribute is used for distinguishing between overlapping and hierarchical groupings. The member-of attribute allows for easy distinguishing of what grouping elements are in what hierarchy. Feature elements contained within a "stop" type of grouping may be ignored.

9.72 THammerPull Class Reference

MusicXML *hammer-on* and *pull-off* elements.

```
#include <TTechnical.h>
```

Inheritance diagram for THammerPull::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const std::string &data)
differentiate between hammer-on (true) and pull-off (false)
- void **setStartStop** (StartStop::type t)
- void **setNumber** (int n)
- const std::string & **getData** () const
- StartStop::type **getStartStop** () const
- int **getNumber** () const

Protected Member Functions

- THammerPull** (StartStop::type t, bool h=true)

Friends

- EXP friend SMARTP< **THammerPull** > **newHammerPull** (bool hammer=true)

9.72.1 Detailed Description

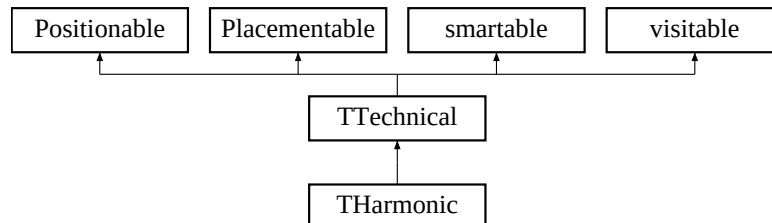
The hammer-on and pull-off elements are used in guitar and fretted instrument notation. Since a single slur can be marked over many notes, the hammer-on and pull-off elements are separate so the individual pair of notes can be specified. The element content can be used to specify how the hammer-on or pull-off should be notated. An empty element leaves this choice to the program defaults.

9.73 THarmonic Class Reference

MusicXML harmonic element.

```
#include <TTechnical.h>
```

Inheritance diagram for THarmonic::



Public Types

- enum **harmonic** { **undefined**, **natural**, **artificial** }
- enum **pitch** { **undefined_pitch**, **base_pitch**, **touching_pitch**, **sounding_pitch** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setHarmonic** (harmonic t)
- void **setPitch** (pitch p)
- harmonic **getHarmonic** () const
- pitch **getPitch** () const

Static Public Member Functions

- static const std::string **xmlHarmonic** (harmonic d)
convert a numeric harmonic to a MusicXML string
- static harmonic **xmlHarmonic** (const std::string str)
convert a MusicXML std::string to a numeric harmonic
- static const std::string **xmlPitch** (pitch d)
convert a numeric pitch to a MusicXML string
- static pitch **xmlPitch** (const std::string str)
convert a MusicXML string to a numeric pitch

Friends

- EXP friend SMARTP< THarmonic > **newHarmonic** ()

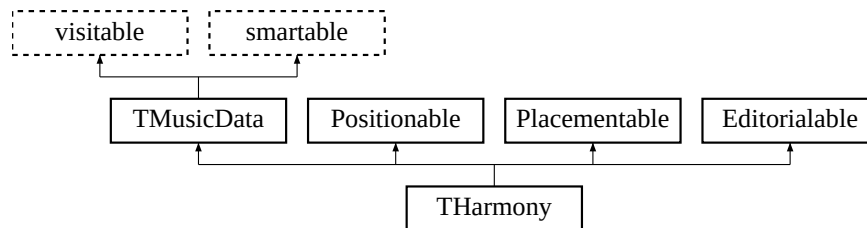
9.73.1 Detailed Description

The harmonic element indicates natural and artificial harmonics. Natural harmonics usually notate the base pitch rather than the sounding pitch. Allowing the type of pitch to be specified, combined with MusicXML's controls for appearance/playback differences, allows both the notation and the sound to be represented. Artificial harmonics can add a notated touching-pitch; the pitch or fret at which the string is touched lightly to produce the harmonic. Artificial pinch harmonics will usually not notate a touching pitch.

9.74 THarmony Class Reference

```
#include <THarmony.h>
```

Inheritance diagram for THarmony::



Public Types

- enum type {
 undefined, **kexplicit**, **implied**, **alternate**,
 last = alternate }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (type t)
- void **setPrintObject** (YesNo::type yn)
- void **setPrintFrame** (YesNo::type yn)
- void **add** (SHarmonyChord hc)
- type **getType** () const
- YesNo::type **getPrintObject** () const
- YesNo::type **getPrintFrame** () const
- SFrame** & **frame** ()
- vvector**< SHarmonyChord > & **getHarmonyChords** ()

Static Public Member Functions

- static const string **xml** (type t)
- static type **xml** (const string str)

Friends

- EXP friend SMARTP< THarmony > **newHarmony** ()

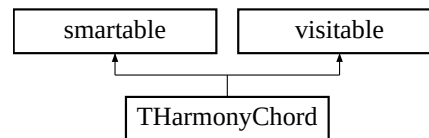
9.74.1 Detailed Description

the MusicXML *harmony* element.

9.75 THarmonyChord Class Reference

```
#include <THarmony.h>
```

Inheritance diagram for THarmonyChord::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setKind** (const std::string kind)
- void **setInversion** (int n)
- void **setRootFunction** (SRootFunction function)
- void **setBass** (SBass bass)
- void **add** (SDegree degree)
- const std::string & **getKind** () const
- int **getInversion** () const
- SRootFunction **getRootFunction** () const
- SBass **getBass** () const
- vvector< SDegree > & **getDegrees** ()

Friends

- EXP friend SMARTP< THarmonyChord > **newHarmonyChord** ()

9.75.1 Detailed Description

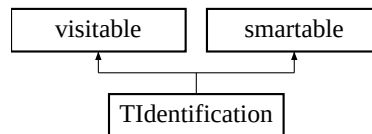
the *harmony-chord* entity part of the *harmony* element.

9.76 TIdentification Class Reference

Contains metadata about the score.

```
#include <TIdentification.h>
```

Inheritance diagram for TIdentification::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SCreator &creator)
- long **add** (const SRights &rights)
- void **setSource** (const std::string &src)
- **vvector**< SCreator > & **creators** ()
- **vvector**< SRights > & **rights** ()
- const std::string & **getSource** ()
- **SEncoding** & **encoding** ()
- **SMiscId** & **misc** ()

Friends

- EXP friend SMARTP< TIdentification > **newIdentification** ()

9.76.1 Detailed Description

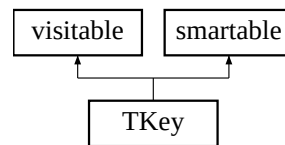
It includes information from the initial 13+ records in a MuseData file that may apply at a score-wide, movement- wide, or part-wide level. It corresponds to the MusicXML *identification* element as defined in identity.dtd.

9.77 TKey Class Reference

Represents a key signature.

```
#include <TKey.h>
```

Inheritance diagram for TKey::



Public Types

- enum { **undefined** = -1 }
- enum {
 major = 1, **minor**, **dorian**, **phrygian**,
 lydian, **mixolydian**, **aeolian**, **ionian**,
 locrian, **last** = locrian }

Public Member Functions

- void **setCancel** (long cancel)
- void **setFifths** (long fifths)
- void **setMode** (long mode)
- long **getCancel** () const
- long **getFifths** () const
- long **getMode** () const
- virtual void **accept** (TScoreVisitor &visitor)

Static Public Member Functions

- static const string **xmlmode** (int d)
 convert an integer mode to a MusicXML string
- static int **xmlmode** (const string str)
 convert an MusicXML string to an integer mode

Friends

- EXP friend SMARTP< TKey > **newKey** ()

9.77.1 Detailed Description

Traditional key signatures are represented by the number of flats and sharps, plus an optional mode for major/ minor/mode distinctions. Negative numbers are used for flats and positive numbers for sharps, reflecting the key's placement within the circle of fifths (hence the element name). A cancel element indicates that the old key signature should be cancelled before the new one appears. This will always happen when changing to C major or A minor and need not be specified then. The cancel value matches the fifths value of the cancelled key signature (e.g., a cancel of -2 will provide an explicit cancellation for changing from B flat major to F major).

Non-traditional key signatures can be represented using the Humdrum/Scot concept of a list of altered tones. The key-step and key-alter elements are represented the same way as the step and alter elements are in the pitch element in note.dtd. The different element names indicate the different meaning of altering notes in a scale versus altering a sounding pitch.

Valid mode values include major, minor, dorian, phrygian, lydian, mixolydian, aeolian, ionian, and locrian.

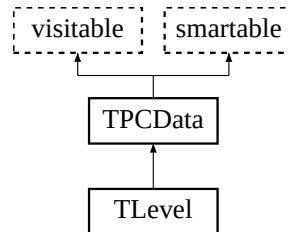
TKey (p. 126) corresponds to the MusicXML *key* element as defined in attributes.dtd.

9.78 TLevel Class Reference

the level elements.

```
#include <common.h>
```

Inheritance diagram for TLevel::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setParentheses** (YesNo::type t)
- void **setBrackets** (YesNo::type t)
- void **setSize** (FullCue::type t)
- YesNo::type **getParentheses** () const
- YesNo::type **getBrackets** () const
- FullCue::type **getSize** () const

Friends

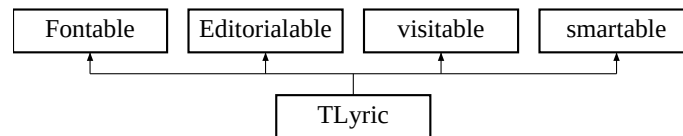
- EXP friend SMARTP< TLevel > newLevel ()

9.79 TLyric Class Reference

Representation of lyrics.

```
#include <TNote.h>
```

Inheritance diagram for TLyric::



Public Types

- enum **syllabic** {
 undefined = -1, **single** = 1, **begin**, **end**,
 middle, **last** = middle }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- string **getName** () const
- string **getText** () const
- int **getNumber** () const
- syllabic **getSyllabic** () const
- bool **getExtend** () const
- bool **getLaughing** () const
- bool **getHumming** () const
- bool **getEndLine** () const
- bool **getEndParagraph** () const
- **vvector**< SMultipleLyricPart > & **getMultipleParts** ()
- void **setName** (const string name)
- void **setText** (const string text)
- void **setNumber** (int number)
- void **setSyllabic** (syllabic syll)
- void **setExtend** (bool extend)
- void **setLaughing** (bool laugh)
- void **setHumming** (bool hum)
- void **setEndLine** (bool el)
- void **setEndParagraph** (bool ep)
- void **add** (SMultipleLyricPart part)

Static Public Member Functions

- static const string **xmlsyllabic** (syllabic d)
convert a numeric syllabic type to a MusicXML string
- static syllabic **xmlsyllabic** (const string str)
convert an MusicXML string to a numeric syllabic type

Friends

- EXP friend SMARTP< TLyric > newLyric ()

9.79.1 Detailed Description

Text underlays for lyrics, based on Humdrum with support for other formats. The lyric number indicates multiple lines, though a name can be used as well (as in Finale's verse/chorus/section specification). Word extensions are represented using the extend element. Hyphenation is indicated by the syllabic element, which can be single, begin, end, or middle. These represent single-syllable words, word-beginning syllables, word-ending syllables, and mid-word syllables. Multiple syllables on a single note are separated by elision elements. A hyphen in the text element should only be used for an actual hyphenated word. Humming and laughing representations are taken from Humdrum. The end-line and end-paragraph elements come from RP-017 for Standard MIDI File Lyric meta-events; they help facilitate lyric display for Karaoke and similar applications. Language names for text elements come from ISO 639, with optional country subcodes from ISO 3166.

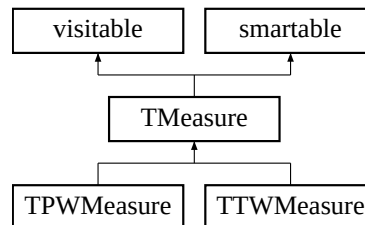
The MusicXML *lyric* element is defined in note.dtd.

9.80 TMeasure Class Reference

The MusicXML base class for partwise and timewise *measure* elements.

```
#include <TMeasure.h>
```

Inheritance diagram for TMeasure::



Public Member Functions

- void **setStringNumber** (const string &num)
- void **setImplicit** (YesNo::type type)
- void **setNonControlling** (YesNo::type type)
- const string & **getStringNumber** () const
- long **getNumber** () const
- YesNo::type **getImplicit** () const
- YesNo::type **getNonControlling** () const

Protected Member Functions

- **TMeasure** (long num)
- **TMeasure** (string num)

9.80.1 Detailed Description

Measures have a required measure number attribute.

The *implicit* attribute is set to "yes" for measures where the measure number should never appear, such as pickup measures and the last half of mid-measure repeats. The implicit attribute is "no" if not specified.

The *non-controlling* attribute indicates that this measure in this part does not necessarily synchronize with other measures in other parts.

The MusicXML *measure* element is defined in score.dtd.

Todo

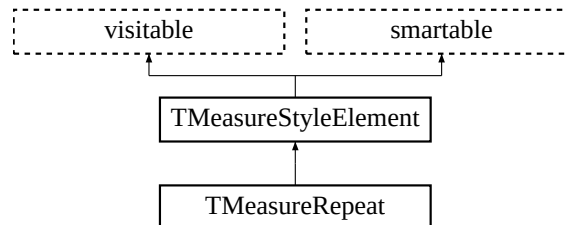
support of the *implicit* and *non-controlling* attributes.

9.81 TMeasureRepeat Class Reference

Represents measure repeat (for measure style).

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TMeasureRepeat::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- void **setSlashes** (int n)
- void **setStartStop** (StartStop::type t)
- int **getSlashes** () const
- const string & **getData** () const
- StartStop::type **getStartStop** () const

Protected Member Functions

- **TMeasureRepeat** (StartStop::type t)

Friends

- EXP friend SMARTP< TMeasureRepeat > **newMeasureRepeat** (StartStop::type type=StartStop::undefined)

9.81.1 Detailed Description

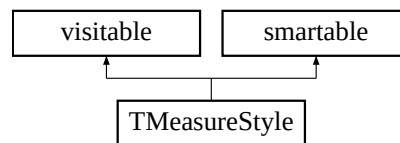
The measure-repeat element is used for both single and multiple measure repeats. The text of the element indicates the number of measures to be repeated in a single pattern. The slashes attribute specifies the number of slashes to use in the repeat sign. It is 1 if not specified. Both the start and the stop of the measure-repeat must be specified.

9.82 TMeasureStyle Class Reference

Represents measure style.

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TMeasureStyle::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setNumber** (int num)
- void **setStyle** (SMeasureStyleElement &elt)
- SMeasureStyleElement & **getStyle** ()
- int **getNumber** ()

Friends

- EXP friend SMARTP< TMeasureStyle > **newMeasureStyle** ()

9.82.1 Detailed Description

A measure-style indicates a special way to print partial to multiple measures within a part. This includes multiple rests over several measures, repeats of betas, single, or multiple measures, and use of slash notation.

The multiple-rest and measure-repeat symbols indicate the number of measures covered in the element content. The beat-repeat and slash elements can cover partial measures. All but the multiple-rest element use a type attribute to indicate starting and stopping the use of the style. The optional number attribute specifies the staff number from top to bottom on the system, as with clef.

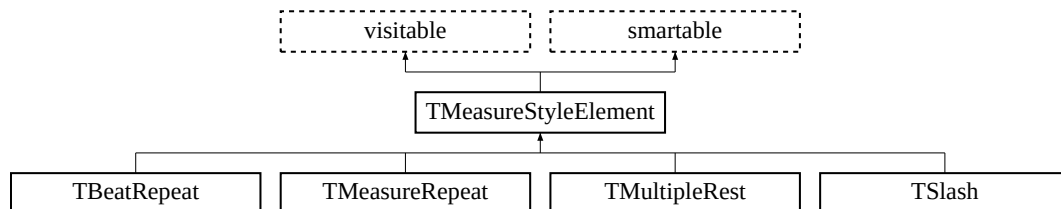
The measure-repeat and beat-repeat element specify a notation style for repetitions. The actual music being repeated needs to be repeated within the MusicXML file. These elements indicate to use the appropriate notation to indicate the repeat.

9.83 TMeasureStyleElement Class Reference

base class for all the elements of the *measure-style*.

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TMeasureStyleElement::



Public Member Functions

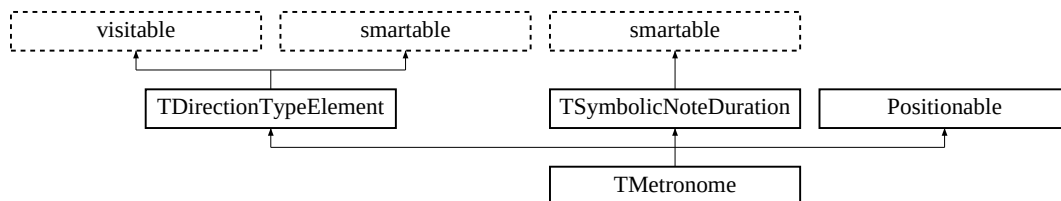
- virtual void **accept** (TScoreVisitor &visitor)

9.84 TMetronome Class Reference

Standard metronome marks.

```
#include <TDirection.h>
```

Inheritance diagram for TMetronome::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setParentheses** (YesNo::type val)
- void **setPerMinute** (string pm)
- void **setPerMinute** (long pm)
- YesNo::type **getParentheses** () const
- string **getPerMinute** () const
- SSymbolicNoteDuration & **equal** ()

Friends

- EXP friend SMARTP< TMetronome > **newMetronome** ()

9.84.1 Detailed Description

The beat-unit values are the same as for a type element, and the beat-unit-dot works like the dot element. The per-minute element can be a number, or a text description including numbers. The parentheses attribute indicates whether or not to put the metronome mark in parentheses; its value is no if not specified.

The MusicXML *metronome* element is defined in direction.dtd.

9.84.2 Member Function Documentation

9.84.2.1 SSymbolicNoteDuration& equal () [inline]

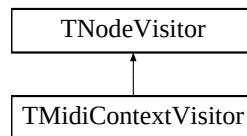
used to specify metronome marking such as <symbolic duration = symbolic duration> ("eight note = eight note" for example). When the metronome uses this symbolic form, the PerMinute value is ignored. To cancel the symbolic form, set **equal**() (p. 135)=0

9.85 TMidiContextVisitor Class Reference

A Visitor that maintains a context for MIDI generation.

```
#include <TMidiContextVisitor.h>
```

Inheritance diagram for TMidiContextVisitor::



Public Member Functions

- **TMidiContextVisitor** (long tpq)
- long **getChan** (SNote &elt)
Get the MIDI channel.
- long **getPart** (SNote &elt)
Get the note part number, to be used possibly as the MIDIFile track number.
- bool **getInstrument** (string ident, TInstrument &instrument)
- bool **getInstrument** (TInstrument &instrument)
- long **getPitch** (SNote &elt)
Get the note MIDI pitch.
- unsigned long **getFullDuration** (SNote &elt)
Get the note full duration (including the duration of tied notes).
- long **getDynamics** (SNote &elt)
Get the note dynamics, to be used as the MIDI velocity.
- long **getDate** ()
Get the current date in ticks. (MusicXML note durations and dates are expressed using a global Ticks Per Quarter value).
- long **getDuration** ()
Get the score duration in ticks. (MusicXML note durations and dates are expressed using a global Ticks Per Quarter value).
- long **getPart** ()
Get the current part.
- long **getTempo** ()
Get the current tempo.
- long **getTPQ** ()
Get the current TicksPerQuarter value.

- virtual void **visiteEnter** (SAttributes &elt)
- virtual void **visiteEnter** (SBackup &elt)
- virtual void **visiteEnter** (SForward &elt)
- virtual void **visiteEnter** (SChord &elt)
- virtual void **visiteLeave** (SChord &elt)
- virtual void **visiteEnter** (SMidiInstrument &elt)
- virtual void **visiteLeave** (SNote &elt)
- virtual void **visiteEnter** (SPWMeasure &elt)
- virtual void **visiteEnter** (SPWPart &elt)
- virtual void **visiteLeave** (SPWPart &elt)
- virtual void **visiteEnter** (SScoreHeader &elt)
- virtual void **visiteEnter** (SScorePart &elt)
- virtual void **visiteEnter** (SSound &elt)
- virtual void **visiteEnter** (STranspose &elt)

9.85.1 Detailed Description

Todo

Handling of grace notes. Handling of divisions attribute in coda or segno sound attribute. Handling of actual duration in fine sound element.

9.85.2 Member Function Documentation

9.85.2.1 long getChan (SNote & elt)

Parameters:

elt The note element.

Returns:

The note MIDI channel from 1 to 16.

9.85.2.2 long getPart (SNote & elt)

Parameters:

elt The note element.

Returns:

The note part number.

9.85.2.3 long getPitch (SNote & elt)

Parameters:

elt The note element.

Returns:

The note MIDI pitch.

9.85.2.4 unsigned long getFullDuration (SNote & *elt*)**Parameters:**

elt The note element.

Returns:

The note full duration.

9.85.2.5 long getDynamics (SNote & *elt*) [inline]**Parameters:**

elt The note element.

Returns:

The note dynamics.

9.85.2.6 long getDate () [inline]**Returns:**

The current date.

9.85.2.7 long getDuration () [inline]**Returns:**

The score duration.

9.85.2.8 long getPart () [inline]**Returns:**

The current part.

9.85.2.9 long getTempo () [inline]**Returns:**

The current tempo in Beat Per Minutes or TSound::undefined is no tempo has been encountered.

9.85.2.10 long getTPQ () [inline]**Returns:**

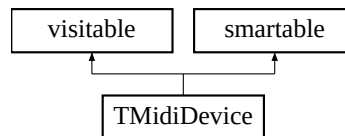
The current TicksPerQuarter value.

9.86 TMidiDevice Class Reference

Corresponds to the DeviceName meta event in Standard MIDI Files.

```
#include <TMidiDevice.h>
```

Inheritance diagram for TMidiDevice::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setPort** (long port)
- void **setName** (const std::string &name)
- long **getPort** () const
- const std::string & **getName** () const

Friends

- EXP friend SMARTP< TMidiDevice > newMidiDevice ()

9.86.1 Detailed Description

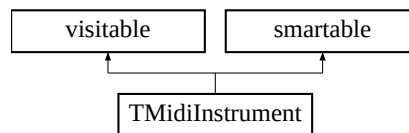
The optional port attribute is a number from 1 to 16 that can be used with the unofficial MIDI port (or cable) meta event. It correspond to the MusicXML *midi-device* element as defined in score.dtd.

9.87 TMidiInstrument Class Reference

Represents a MIDI instrument.

```
#include <TMidiInstrument.h>
```

Inheritance diagram for TMidiInstrument::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- void **setID** (const std::string &id)
ID is a MusicXML required attribute that refers to the score-instrument affected by the change.
- void **setChannel** (long chan)
optionnal MIDI channel numbers range from 1 to 16.
- void **setName** (const std::string &name)
optionnal MIDI name, corresponds to ProgramName meta-events within a Standard MIDI File.
- void **setBank** (long bank)
optionnal MIDI bank number, range from 1 to 16,384.
- void **setProgram** (long prog)
optionnal MIDI program number, range from 1 to 128.
- void **setUnpitched** (long pitch)
- const std::string & **getID** () const
- long **getChannel** () const
- const std::string & **getName** () const
- long **getBank** () const
- long **getProgram** () const
- long **getUnpitched** () const
- virtual void **accept** (TScoreVisitor &visitor)

Protected Member Functions

- TMidiInstrument** (std::string ident)

Friends

- EXP friend SMARTP< TMidiInstrument > newMidiInstrument (std::string id)

9.87.1 Detailed Description

TMidiInstrument (p. 140) includes the necessary to address a specific MIDI instrument (channel, bank, program) + additionnal information required by the MusicXML *midi-instrument* element as defined in common.dtd.

9.87.2 Member Function Documentation

9.87.2.1 void setUnpitched (long *pitch*) [inline]

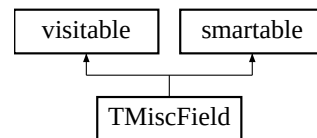
optionnal : for unpitched instruments, specify a MIDI note number ranging from 1 to 128. Usually used with MIDI banks for percussion.

9.88 TMiscField Class Reference

fields of miscellaneous identification

```
#include <TIdentification.h>
```

Inheritance diagram for TMiscField::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const std::string &type)
- void **setName** (const std::string &name)
- const std::string & **getData** () const
- const std::string & **getName** () const

Protected Member Functions

- **TMiscField** (std::string name)

Friends

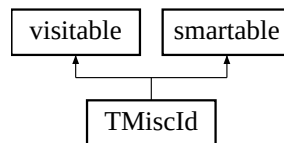
- EXP friend SMARTP< TMiscField > **newMiscField** (std::string name)

9.89 TMiscId Class Reference

miscellaneous identification.

```
#include <TIdentification.h>
```

Inheritance diagram for TMiscId::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **add** (SMiscField field)
- **vvector**< SMiscField > & **getFields** ()

Friends

- EXP friend SMARTP< TMiscId > **newMiscId** ()

9.89.1 Detailed Description

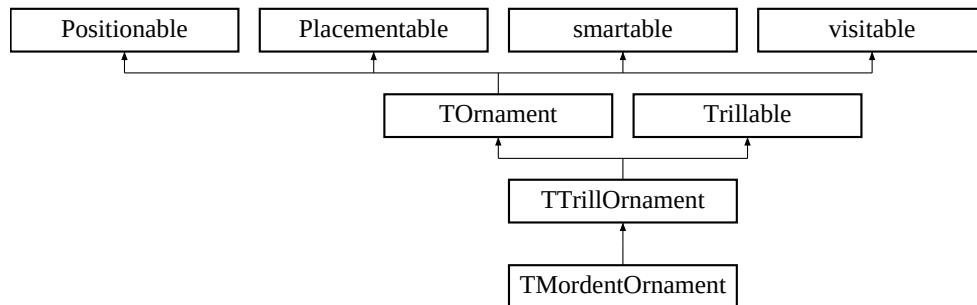
If a program has other metadata not yet supported in MusicXML, it can go in the miscellaneous area.

9.90 TMordentOrnament Class Reference

mordent and inverted-mordent ornaments

```
#include <TOrnaments.h>
```

Inheritance diagram for TMordentOrnament::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setLong** (YesNo::type yn)
- YesNo::type **getLong** () const

Protected Member Functions

- **TMordentOrnament** (type t)

Friends

- EXP friend SMARTP< **TMordentOrnament** > **newMordentOrnament** (type t)

9.90.1 Detailed Description

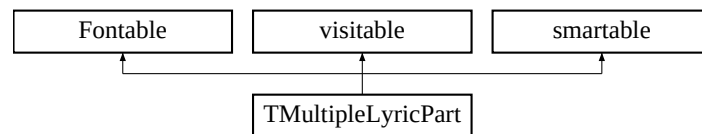
The long attribute for the mordent and inverted-mordent elements is "no" by default. The mordent element represents the sign with the vertical line; the inverted-mordent represent the sign without the vertical line.

9.91 TMultipleLyricPart Class Reference

Representation of multiple lyrics on a single note.

```
#include <TNote.h>
```

Inheritance diagram for TMultipleLyricPart::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- string **getText** () const
- TLyric::syllabic **getSyllabic** () const
- void **setText** (const string text)
- void **setSyllabic** (TLyric::syllabic syll)

Friends

- EXP friend SMARTP< TMultipleLyricPart > newMultipleLyricPart ()

9.91.1 Detailed Description

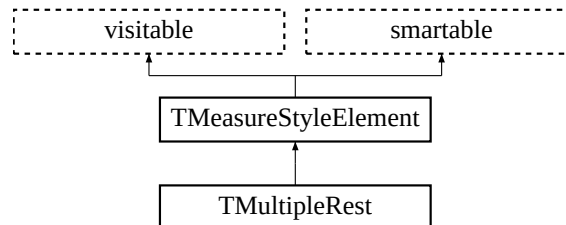
Multiple syllables on a single note are separated by elision elements. Elision are implicit in **TMultipleLyricPart** (p. 145).

9.92 TMultipleRest Class Reference

Represents multiple rests (for measure style).

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TMultipleRest::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- void **setUseSymbol** (YesNo::type t)
- YesNo::type **getUseSymbol** () const
- const string & **getData** () const

Friends

- EXP friend SMARTP< TMultipleRest > **newMultipleRest** ()

9.92.1 Detailed Description

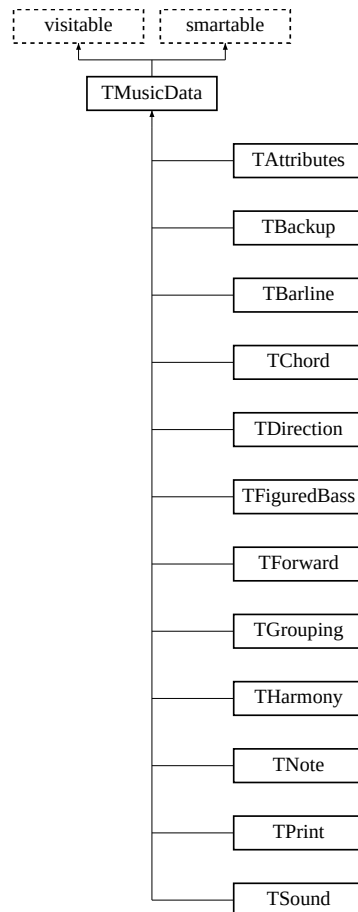
The text of the multiple-rest element indicates the number of measures in the multiple rest. Multiple rests may use the 1-bar / 2-bar / 4-bar rest symbols, or a single shape. The use-symbols attribute indicates which to use; it is no if not specified.

9.93 TMusicData Class Reference

base class for all the elements of the *music-data* entity.

```
#include <common.h>
```

Inheritance diagram for TMusicData::



9.93.1 Detailed Description

TMusicData (p. 147) is an abstract class which purpose is to define a common type for all the elements covered by the *music-data* entity.

9.94 TMusicXMLFile Class Reference

provides MusicXML files reading and writing

```
#include <TMusicXMLFile.h>
```

Public Member Functions

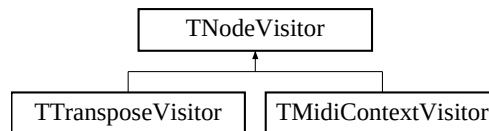
- **SScore read** (const string &file)
- bool **write** (**SScore** score, const string &file)
- bool **write** (**SScore** score, std::ostream &os)

9.95 TNodeVisitor Class Reference

A node Visitor of MusicXML scores.

```
#include <TNodeVisitor.h>
```

Inheritance diagram for TNodeVisitor::



Public Member Functions

- virtual void **visiteEnter** (SAccidental &elt)
- virtual void **visiteEnter** (SAccidentalMark &elt)
- virtual void **visiteEnter** (SArpeggiate &elt)
- virtual void **visiteEnter** (SNonArpeggiate &elt)
- virtual void **visiteEnter** (SArticulationElement &elt)
- virtual void **visiteEnter** (SArticulations &elt)
- virtual void **visiteEnter** (SAttributes &elt)
- virtual void **visiteEnter** (SBackup &elt)
- virtual void **visiteEnter** (SBarline &elt)
- virtual void **visiteEnter** (SBass &elt)
- virtual void **visiteEnter** (SBeam &elt)
- virtual void **visiteEnter** (SBeatRepeat &elt)
- virtual void **visiteEnter** (SBendWithBar &elt)
- virtual void **visiteEnter** (SBend &elt)
- virtual void **visiteEnter** (SBracket &elt)
- virtual void **visiteEnter** (SChord &elt)
- virtual void **visiteEnter** (SClef &elt)
- virtual void **visiteEnter** (SCoda &elt)
- virtual void **visiteEnter** (SCreator &elt)
- virtual void **visiteEnter** (SDashes &elt)
- virtual void **visiteEnter** (SDegree &elt)
- virtual void **visiteEnter** (SDirection &elt)
- virtual void **visiteEnter** (SDirectionType &elt)
- virtual void **visiteEnter** (SDirective &elt)
- virtual void **visiteEnter** (SDynamic &elt)
- virtual void **visiteEnter** (SEncoding &elt)
- virtual void **visiteEnter** (SEnding &elt)
- virtual void **visiteEnter** (SExpression &elt)
- virtual void **visiteEnter** (SFermata &elt)
- virtual void **visiteEnter** (SFeature &elt)
- virtual void **visiteEnter** (SFigure &elt)
- virtual void **visiteEnter** (SFiguredBass &elt)
- virtual void **visiteEnter** (SFingering &elt)
- virtual void **visiteEnter** (SForward &elt)

- virtual void **visiteEnter** (**SFrame** &elt)
- virtual void **visiteEnter** (**SFrameNote** &elt)
- virtual void **visiteEnter** (**SFunction** &elt)
- virtual void **visiteEnter** (**SGlissando** &elt)
- virtual void **visiteEnter** (**SGraceNote** &elt)
- virtual void **visiteEnter** (**SGraphNote** &elt)
- virtual void **visiteEnter** (**SGrouping** &elt)
- virtual void **visiteEnter** (**SHammerPull** &elt)
- virtual void **visiteEnter** (**SHarmonic** &elt)
- virtual void **visiteEnter** (**SHarmony** &elt)
- virtual void **visiteEnter** (**SHarmonyChord** &elt)
- virtual void **visiteEnter** (**SIdentification** &elt)
- virtual void **visiteEnter** (**SKey** &elt)
- virtual void **visiteEnter** (**SLyric** &elt)
- virtual void **visiteEnter** (**SMeasure** &elt)
- virtual void **visiteEnter** (**SMeasureRepeat** &elt)
- virtual void **visiteEnter** (**SMeasureStyle** &elt)
- virtual void **visiteEnter** (**SMeasureStyleElement** &elt)
- virtual void **visiteEnter** (**SMetronome** &elt)
- virtual void **visiteEnter** (**SMidiDevice** &elt)
- virtual void **visiteEnter** (**SMidiInstrument** &elt)
- virtual void **visiteEnter** (**SMiscField** &elt)
- virtual void **visiteEnter** (**SMiscId** &elt)
- virtual void **visiteEnter** (**SMordentOrnament** &elt)
- virtual void **visiteEnter** (**SMultipleLyricPart** &elt)
- virtual void **visiteEnter** (**SMultipleRest** &elt)
- virtual void **visiteEnter** (**SNotation** &elt)
- virtual void **visiteEnter** (**SNote** &elt)
- virtual void **visiteEnter** (**SNoteHead** &elt)
- virtual void **visiteEnter** (**SOctaveShift** &elt)
- virtual void **visiteEnter** (**SOrnament** &elt)
- virtual void **visiteEnter** (**SOrnaments** &elt)
- virtual void **visiteEnter** (**SOtherArticulation** &elt)
- virtual void **visiteEnter** (**SOtherDirection** &elt)
- virtual void **visiteEnter** (**SOtherDirections** &elt)
- virtual void **visiteEnter** (**SOtherNotation** &elt)
- virtual void **visiteEnter** (**SOtherOrnament** &elt)
- virtual void **visiteEnter** (**SPWMeasure** &elt)
- virtual void **visiteEnter** (**SPWPart** &elt)
- virtual void **visiteEnter** (**SPart** &elt)
- virtual void **visiteEnter** (**SPartGroup** &elt)
- virtual void **visiteEnter** (**SPartList** &elt)
- virtual void **visiteEnter** (**SPedal** &elt)
- virtual void **visiteEnter** (**SPitch** &elt)
- virtual void **visiteEnter** (**SPrint** &elt)
- virtual void **visiteEnter** (**SRehearsal** &elt)
- virtual void **visiteEnter** (**SRepeat** &elt)
- virtual void **visiteEnter** (**SRest** &elt)
- virtual void **visiteEnter** (**SRights** &elt)
- virtual void **visiteEnter** (**SRoot** &elt)

- virtual void **visiteEnter** (SScoreHeader &elt)
- virtual void **visiteEnter** (SScoreInstrument &elt)
- virtual void **visiteEnter** (SScorePart &elt)
- virtual void **visiteEnter** (SScorePartwise &elt)
- virtual void **visiteEnter** (SScoreTimewise &elt)
- virtual void **visiteEnter** (SSegno &elt)
- virtual void **visiteEnter** (SSlide &elt)
- virtual void **visiteEnter** (SSlash &elt)
- virtual void **visiteEnter** (SSLur &elt)
- virtual void **visiteEnter** (SStaffDetails &elt)
- virtual void **visiteEnter** (SStaffTuning &elt)
- virtual void **visiteEnter** (SSupport &elt)
- virtual void **visiteEnter** (SSound &elt)
- virtual void **visiteEnter** (SStrongAccent &elt)
- virtual void **visiteEnter** (STechnical &elt)
- virtual void **visiteEnter** (STechnicals &elt)
- virtual void **visiteEnter** (STWMeasure &elt)
- virtual void **visiteEnter** (STWPart &elt)
- virtual void **visiteEnter** (STie &elt)
- virtual void **visiteEnter** (STimeModification &elt)
- virtual void **visiteEnter** (STimeSign &elt)
- virtual void **visiteEnter** (STranspose &elt)
- virtual void **visiteEnter** (STrillOrnament &elt)
- virtual void **visiteEnter** (STuplet &elt)
- virtual void **visiteEnter** (STupletDesc &elt)
- virtual void **visiteEnter** (SUnpitched &elt)
- virtual void **visiteEnter** (SWavyLine &elt)
- virtual void **visiteEnter** (SWedge &elt)
- virtual void **visiteEnter** (SWords &elt)
- virtual void **visiteEnter** (SWork &elt)
- virtual void **visiteLeave** (SArticulationElement &elt)
- virtual void **visiteLeave** (SAccidentalMark &elt)
- virtual void **visiteLeave** (SArpeggiate &elt)
- virtual void **visiteLeave** (SNonArpeggiate &elt)
- virtual void **visiteLeave** (SArticulations &elt)
- virtual void **visiteLeave** (SAttributes &elt)
- virtual void **visiteLeave** (SBarline &elt)
- virtual void **visiteLeave** (SBeam &elt)
- virtual void **visiteLeave** (SBeatRepeat &elt)
- virtual void **visiteLeave** (SBendWithBar &elt)
- virtual void **visiteLeave** (SBend &elt)
- virtual void **visiteLeave** (SBracket &elt)
- virtual void **visiteLeave** (SChord &elt)
- virtual void **visiteLeave** (SCoda &elt)
- virtual void **visiteLeave** (SDashes &elt)
- virtual void **visiteLeave** (SDirection &elt)
- virtual void **visiteLeave** (SDirectionType &elt)
- virtual void **visiteLeave** (SDynamic &elt)
- virtual void **visiteLeave** (SFermata &elt)
- virtual void **visiteLeave** (SFigure &elt)

- virtual void **visiteLeave** (**SFiguredBass** &elt)
- virtual void **visiteLeave** (**SForward** &elt)
- virtual void **visiteLeave** (**SFrame** &elt)
- virtual void **visiteLeave** (**SFrameNote** &elt)
- virtual void **visiteLeave** (**SGlissando** &elt)
- virtual void **visiteLeave** (**SGraceNote** &elt)
- virtual void **visiteLeave** (**SGraphNote** &elt)
- virtual void **visiteLeave** (**SGrouping** &elt)
- virtual void **visiteLeave** (**SHammerPull** &elt)
- virtual void **visiteLeave** (**SHarmonic** &elt)
- virtual void **visiteLeave** (**SHarmony** &elt)
- virtual void **visiteLeave** (**SHarmonyChord** &elt)
- virtual void **visiteLeave** (**SIdentification** &elt)
- virtual void **visiteLeave** (**SKey** &elt)
- virtual void **visiteLeave** (**SLyric** &elt)
- virtual void **visiteLeave** (**SMeasure** &elt)
- virtual void **visiteLeave** (**SMeasureRepeat** &elt)
- virtual void **visiteLeave** (**SMeasureStyle** &elt)
- virtual void **visiteLeave** (**SMeasureStyleElement** &elt)
- virtual void **visiteLeave** (**SMetronome** &elt)
- virtual void **visiteLeave** (**SMidiDevice** &elt)
- virtual void **visiteLeave** (**SMidiInstrument** &elt)
- virtual void **visiteLeave** (**SMiscField** &elt)
- virtual void **visiteLeave** (**SMiscId** &elt)
- virtual void **visiteLeave** (**SMordentOrnament** &elt)
- virtual void **visiteLeave** (**SMultipleLyricPart** &elt)
- virtual void **visiteLeave** (**SMultipleRest** &elt)
- virtual void **visiteLeave** (**SNotation** &elt)
- virtual void **visiteLeave** (**SNote** &elt)
- virtual void **visiteLeave** (**SOctaveShift** &elt)
- virtual void **visiteLeave** (**SOrnament** &elt)
- virtual void **visiteLeave** (**SOrnaments** &elt)
- virtual void **visiteLeave** (**SOtherArticulation** &elt)
- virtual void **visiteLeave** (**SOtherOrnament** &elt)
- virtual void **visiteLeave** (**SPWMeasure** &elt)
- virtual void **visiteLeave** (**SPWPart** &elt)
- virtual void **visiteLeave** (**SPartList** &elt)
- virtual void **visiteLeave** (**SScoreHeader** &elt)
- virtual void **visiteLeave** (**SScorePart** &elt)
- virtual void **visiteLeave** (**SScorePartwise** &elt)
- virtual void **visiteLeave** (**SScoreTimewise** &elt)
- virtual void **visiteLeave** (**SSegno** &elt)
- virtual void **visiteLeave** (**SSlide** &elt)
- virtual void **visiteLeave** (**SSlash** &elt)
- virtual void **visiteLeave** (**SSLur** &elt)
- virtual void **visiteLeave** (**SStaffDetails** &elt)
- virtual void **visiteLeave** (**SStaffTuning** &elt)
- virtual void **visiteLeave** (**SSound** &elt)
- virtual void **visiteLeave** (**SStrongAccent** &elt)
- virtual void **visiteLeave** (**STechnical** &elt)

- virtual void **visiteLeave** (**STechnicals** &elt)
- virtual void **visiteLeave** (**STrillOrnament** &elt)
- virtual void **visiteLeave** (**STWMeasure** &elt)
- virtual void **visiteLeave** (**STWPart** &elt)
- virtual void **visiteLeave** (**STie** &elt)
- virtual void **visiteLeave** (**STuplet** &elt)
- virtual void **visiteLeave** (**SWavyLine** &elt)
- virtual void **visiteLeave** (**SWedge** &elt)
- virtual void **visiteLeave** (**SWords** &elt)
- virtual void **visiteEnter** (**TOrientation** *elt)
- virtual void **visiteEnter** (**TPlacement** *elt)
- virtual void **visiteEnter** (**TPosition** *elt)
- virtual void **visiteEnter** (**TTrillSound** *elt)
- virtual void **visiteEnter** (**TBendSound** *elt)
- virtual void **visiteEnter** (**TFont** *elt)
- virtual void **visiteEnter** (**TPrintout** *elt)
- virtual void **visiteEnter** (**TEditorial** *elt)
- virtual void **visiteEnter** (**TBezier** *elt)

9.95.1 Detailed Description

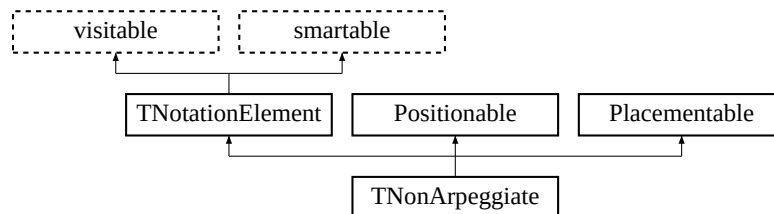
A node Visitor of MusicXML scores. Nodes visitor are to be used with a **TRolledVisitor** (p. 198) or **TUnrolledVisitor** (p. 251) that does the actual score traversing. A node Visitor can implement **visiteEnter** and/or **visiteLeave** methods for each type of node. For terminal nodes, only the **visiteEnter** method can be implemented.

9.96 TNonArpeggiate Class Reference

MusicXML representation for *non-arpeggiate*.

```
#include <TArpeggiate.h>
```

Inheritance diagram for TNonArpeggiate::



Public Types

- enum type { **undefined**, **top**, **bottom** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setNumber** (int n)
- void **setType** (type t)
- int **getNumber** () const
- type **getType** () const

Friends

- EXP friend SMARTP< TNonArpeggiate > **newNonArpeggiate** ()

9.96.1 Detailed Description

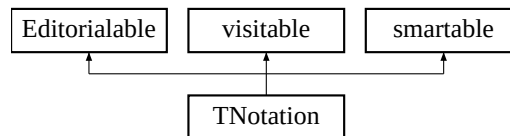
Indicates that this note is at the top or bottom of a bracket indicating to not arpeggiate these notes. Since this does not involve playback, it is only used on the top or bottom notes, not on each note as for the arpeggiate element.

9.97 TNotation Class Reference

Represents musical notations.

```
#include <TNotation.h>
```

Inheritance diagram for TNotation::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SNotationElement &elt)
- **vvector**< SNotationElement > & **list** ()

Friends

- EXP friend SMARTP< TNotation > **newNotation** ()

9.97.1 Detailed Description

Multiple notations are allowed in order to represent multiple editorial levels. The set of notations will be refined and expanded over time, especially to handle more instrument-specific technical notations.

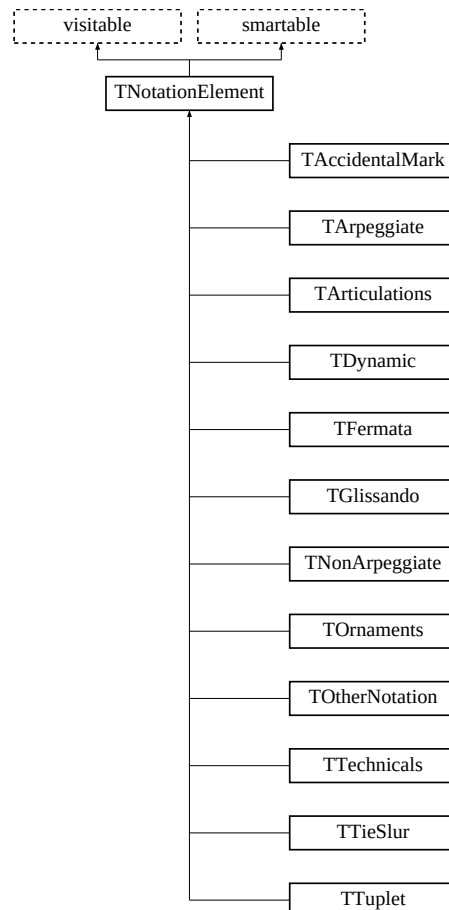
TNotation (p. 155) corresponds to the MusicXML *notation* element as defined in note.dtd.

9.98 TNotationElement Class Reference

base class for all the elements of the *notation* element.

```
#include <common.h>
```

Inheritance diagram for TNotationElement::



9.98.1 Detailed Description

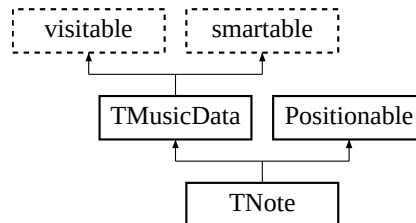
TNotationElement (p. 156) is an abstract class which purpose is to define a common type for all the elements of a *notation* element.

9.99 TNote Class Reference

TNote (p. 157) is the main common type of the music representation.

```
#include <TNote.h>
```

Inheritance diagram for TNote::



Public Types

- enum **style** { **normal**, **cue**, **grace** }
- enum **type** { **pitched**, **rest**, **unpitched** }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDuration** (unsigned long dur)
set the note duration, specified in division units
- unsigned long **getDuration** () const
gives the note duration
- unsigned long **getFullDuration** ()
gives the sounding duration ie takes account of possible ties
- void **setInstrumentID** (string id)
the score-instrument (if any)
- const string **getInstrumentID** () const
- void **setStyle** (style t)
to set the style of the note (normal, cue or grace - defaults to normal)
- style **getStyle** () const
- void **setType** (type t)
to set the type of the note (regular, rest, unpitched - defaults to regular)
- type **getType** () const
- **vvector**< SLyric > **getLyrics** ()
the lyrics
- void **add** (SLyric lyric)
add a lyric to the note

- **SPitch & pitch ()**
the note pitch. May also be used for unpitched and rest placement.
- **SExpression & expression ()**
the expression context of the note, expressed in terms of current division
- **SNote & tiePrevious ()**
the optional previous tied note (corresponds to the tie element)
- **SNote & tieNext ()**
the optional next tied note (corresponds to the tie element)
- **SGraphNode & graphics ()**
the graphical attributes of the note
- **SGraceNote & graceNote ()**
the grace note optionnal element
- **bool isTie ()**
the note is the first of a group of tied notes
- **bool isSingle ()**
the note is "single", not part of a tied group
- **bool isNormal ()**
- **bool isGrace ()**
- **bool isCue ()**
- **bool isPitched ()**
- **bool isRest ()**
- **bool isUnpitched ()**
- **long getVoice () const**

Friends

- EXP friend **SMARTP< TNote > newNote ()**

9.99.1 Detailed Description

It is derived from the MusicXML note element as described in note.dtd but only the sounding attributes has been retained at core level. All the other elements are stored in contextual information which is divided in 2 parts:

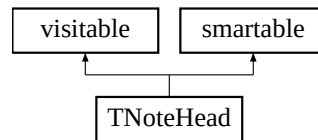
- the graphical notation context (like voice, notehead, accidentals etc...), available using the **graphics()** (p. 158) method
- the expression context (such as dynamics, attack, release etc...), available using the **expression()** (p. 158) method

9.100 TNoteHead Class Reference

the shape of the note head

```
#include <TNote.h>
```

Inheritance diagram for TNoteHead::



Public Types

- enum **notehead** {
 undefined = -1, **slash** = 1, **triangle**, **diamond**,
 square, **cross**, **x**, **circle_x**,
 normal, **none**, **last** = none }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setShape** (notehead shape)
- void **setFilled** (YesNo::type filled)
- void **setParentheses** (YesNo::type par)
- notehead **getShape** () const
- YesNo::type **getFilled** () const
- YesNo::type **getParentheses** () const

Static Public Member Functions

- static const string **xmlnotehead** (notehead d)
 convert a numeric beam type to a MusicXML string
- static notehead **xmlnotehead** (const string str)
 convert an MusicXML string to a numeric beam type

Friends

- EXP friend SMARTP< TNoteHead > **newNoteHead** ()

9.100.1 Detailed Description

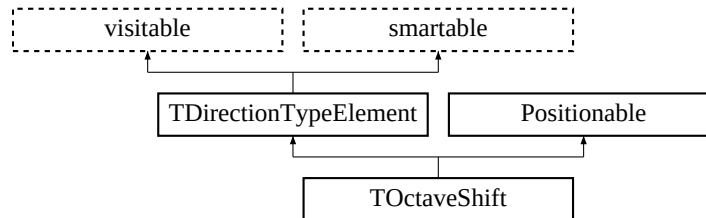
The `notehead` element indicates shapes other than the open and closed ovals associated with note durations. The element value can be slash, triangle, diamond, square, cross, x, circle-x, normal, or none. For the enclosed shapes, the default is to be hollow for half notes and longer, and filled otherwise. The `filled` attribute can be set to change this if needed. If the `parentheses` attribute is set to yes, the notehead is parenthesized. It is no by default.

The MusicXML *notehead* element as defined in `note.dtd`.

9.101 TOctaveShift Class Reference

```
#include <TDirection.h>
```

Inheritance diagram for TOctaveShift::



Public Types

- enum {
 undefined = -1, **up** = 1, **down**, **stop**,
 last = stop }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (int type)
- void **setNumber** (int num)
- void **setSize** (int size)
- int **getType** () const
- int **getNumber** () const
- int **getSize** () const

Static Public Member Functions

- static const string **xmltype** (int d)
 convert an numeric type value to a MusicXML string
- static int **xmltype** (const string str)
 convert an MusicXML string to a numeric type value

Protected Member Functions

- TOctaveShift** (int type)

Friends

- EXP friend SMARTP< TOctaveShift > **newOctaveShift** (int type=TOctaveShift::undefined)

9.101.1 Detailed Description

The MusicXML *octave-shift* element.

Octave shifts indicate where notes are shifted up or down from their true pitched values because of printing difficulty. Thus a treble clef line noted with 8va will be indicated with an octave-shift down from the pitch data indicated in the notes. A size of 8 indicates one octave; a size of 15 indicates two octaves.

Defined in direction.dtd.

9.102 TOrientation Class Reference

Represents the MusicXml *orientation* entity.

```
#include <common.h>
```

Public Types

- enum **type** { **undefined**, **over**, **under**, **last** = under }

Public Member Functions

- **TOrientation** (int orient=undefined)
- virtual void **accept** (TScoreVisitor &visitor)
- void **set** (int orient)
- int **get** () const

Static Public Member Functions

- static const string **xmlorientation** (int d)
convert an integer orientation to a MusicXML string
- static int **xmlorientation** (const string str)
convert an MusicXML string to an integer orientation

9.102.1 Detailed Description

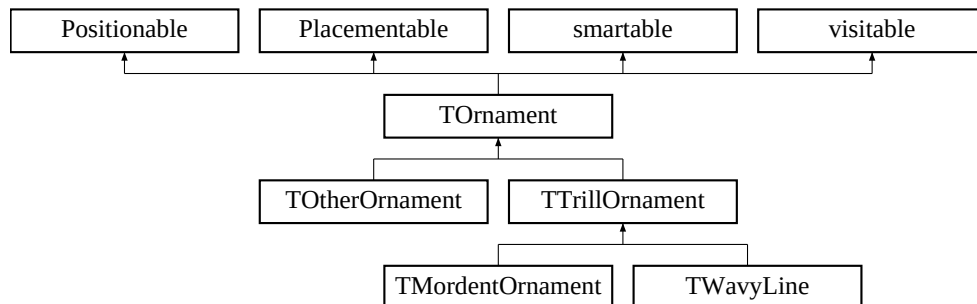
The orientation attribute indicates whether slurs and ties are overhand (tips down) or underhand (tips up). It is defined in common.dtd.

9.103 TOrnament Class Reference

basic ornament representation

```
#include <TOrnaments.h>
```

Inheritance diagram for TOrnament::



Public Types

- enum type {
 trill_mark = 1, turn, delayed_turn, shake,
 wavy_line, mordent, inverted_mordent, schleifer,
 other_ornament, last = other_ornament }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (type t)
- type **getType** () const
- void **add** (SAccidentalMark acc)
- vvector**< SAccidentalMark > & **accidents** ()

Static Public Member Functions

- static const string **xml** (type d)
 convert a numeric ornament type to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric ornament type

Protected Member Functions

- TOrnament** (type t)

Friends

- EXP friend SMARTP< TOrnament > newOrnament (type t)

9.103.1 Detailed Description

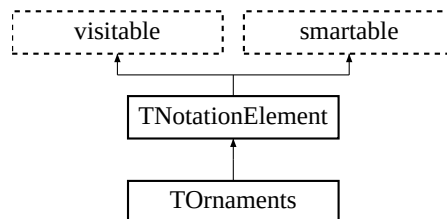
Ornaments can be any of several types, followed optionally by accidentals. The accidental-mark element's content is represented the same as an accidental element, but with a different name to reflect the different musical meaning. This base **TOrnament** (p. 164) class is intended to be derived but may be used as is if additional information such as trill-sound is not required.

9.104 TOrnaments Class Reference

basic ornament representation

```
#include <TOrnaments.h>
```

Inheritance diagram for TOrnaments::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **add** (SOrnament ornament)
- **vvector**< SOrnament > **ornaments** ()
- SOrnament **getLast** ()

Friends

- EXP friend SMARTP< TOrnaments > **newOrnaments** ()

9.104.1 Detailed Description

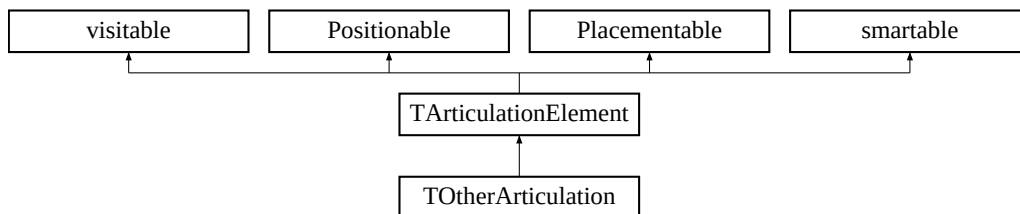
Ornaments can be any of several types, followed optionally by accidentals. The accidental-mark element's content is represented the same as an accidental element, but with a different name to reflect the different musical meaning.

9.105 TOtherArticulation Class Reference

other articulations

```
#include <TNotation.h>
```

Inheritance diagram for TOtherArticulation::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- const string & **getData** () const

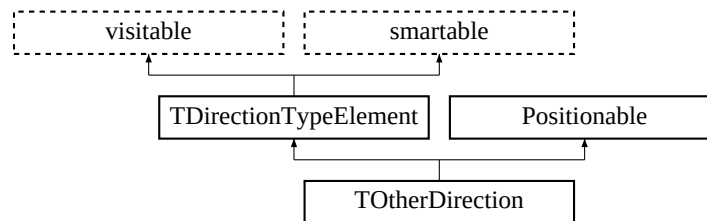
Friends

- EXP friend SMARTP< TOtherArticulation > newOtherArticulation ()

9.106 TOtherDirection Class Reference

```
#include <TOtherDirections.h>
```

Inheritance diagram for TOtherDirection::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- const string & **getData** () const

Friends

- EXP friend SMARTP< TOtherDirection > newOtherDirection ()

9.106.1 Detailed Description

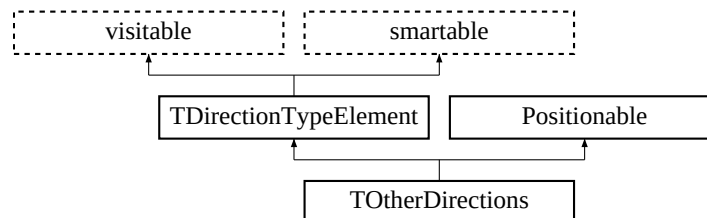
MusicXML *other-direction* element.

The other-direction element is used to define any direction symbols not yet in the current version of MusicXML. This allows extended representation, though without application interoperability.

9.107 TOtherDirections Class Reference

```
#include <TOtherDirections.h>
```

Inheritance diagram for TOtherDirections::



Public Types

- enum type { damp = 1, damp_all, eyeglasses, last = eyeglasses }

Public Member Functions

- virtual void accept (TScoreVisitor &visitor)
- void setType (type t)
- type getType () const

Static Public Member Functions

- static const string xml (type d)
convert an numeric type value to a MusicXML string
- static type xml (const string str)
convert an MusicXML string to a numeric type value

Protected Member Functions

- TOtherDirections (type t)

Friends

- EXP friend SMARTP< TOtherDirections > newOtherDirections (type t)

9.107.1 Detailed Description

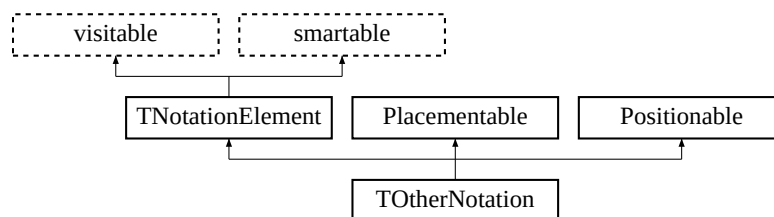
additional MusicXML direction elements.

9.108 TOtherNotation Class Reference

other notations representation

```
#include <TNotation.h>
```

Inheritance diagram for TOtherNotation::



Public Types

- enum {
undefined = -1, **actual** = 1, **both**, **none**,
last = none }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
the required type attribute
- void **setNumber** (int number)
- StartStop::type **getType** () const
- int **getNumber** () const

Protected Member Functions

- TOtherNotation** (StartStop::type type)

Friends

- EXP friend SMARTP< TOtherNotation > **newOtherNotation** (StartStop::type type=Start-Stop::undefined)

9.108.1 Detailed Description

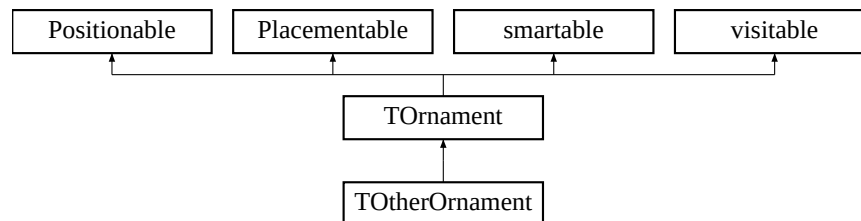
The other-notation element is used to define any notations not yet in MusicXML. This allows extended representation, though without application interoperability. It handles notations where more specific extension elements such as other-dynamics and other-technical are not appropriate.

9.109 TOtherOrnament Class Reference

other ornaments

```
#include <TOrnaments.h>
```

Inheritance diagram for TOtherOrnament::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- const string & **getData** () const

Friends

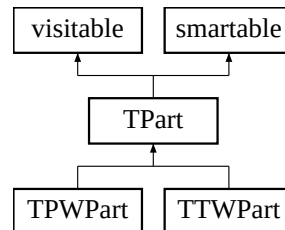
- EXP friend SMARTP< TOtherOrnament > newOtherOrnament ()

9.110 TPart Class Reference

The base class for the MusicXML timewise and partwise *part* elements.

```
#include <TPart.h>
```

Inheritance diagram for TPart::



Public Member Functions

- void **setID** (const std::string &id)
- const std::string & **getID** () const

Protected Member Functions

- **TPart** (const std::string &id)

9.110.1 Detailed Description

In either format (partwise or timewise), the *part* element has a required id attribute that is an IDREF back to a score-part in the part-list.

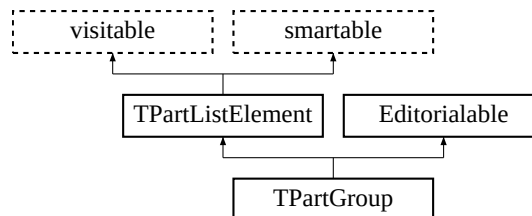
The MusicXML *part* element is defined in score.dtd.

9.111 TPartGroup Class Reference

Indicates groupings of parts in a score.

```
#include <TPartGroup.h>
```

Inheritance diagram for TPartGroup::



Public Types

- enum **symbol** {
undefined = -1, **none** = 1, **brace**, **line**,
bracket, **last** = bracket }
the possible values for the group symbol

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
the required type attribute
- void **setNumber** (int num)
the optional group number
- void **setGroupName** (string name)
optional group name
- void **setGroupAbbrev** (string abbrev)
optional name abbreviation
- void **setGroupBarline** (string barline)
optional group barline
- void **setSymbol** (symbol sym)
optional group symbol
- StartStop::type **getType** () const
- int **getNumber** () const
- string **getGroupName** () const
- string **getGroupAbbrev** () const
- string **getGroupBarline** () const
- symbol **getSymbol** () const

Static Public Member Functions

- static const string **xmlsymbol** (symbol d)
convert an integer dynamic to a MusicXML string
- static symbol **xmlsymbol** (const string str)
convert an MusicXML string to an integer dynamic

Protected Member Functions

- **TPartGroup** (StartStop::type type)

Friends

- EXP friend SMARTP< **TPartGroup** > **newPartGroup** (StartStop::type type)

9.111.1 Detailed Description

Groupings of parts in the score are usually indicated by braces and brackets. The number attribute is used to distinguish overlapping and nested part-groups, not the sequence of groups. As with parts, groups can have a name and abbreviation. The group-symbol element indicates how the group is indicated on the score. Values include none (default), brace, line, and bracket. Values for the child elements are ignored at the stop of a group.

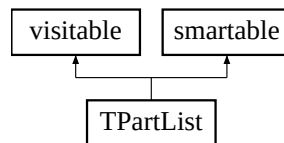
TPartGroup (p. 173) corresponds to the MusicXML *part-group* element as defined in score.dtd.

9.112 TPartList Class Reference

Identifies the different musical parts in a movement.

```
#include <TPartList.h>
```

Inheritance diagram for TPartList::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SScorePart &part)
- long **add** (const SPartGroup &part)
- **vvector**< SPartListElement > & **parts** ()

Friends

- EXP friend SMARTP< TPartList > **newPartList** ()

9.112.1 Detailed Description

There must be at least one score-part, combined as desired with part-group elements that indicate braces and brackets. Parts are ordered from top to bottom in a score based on the order in which they appear in the part-list.

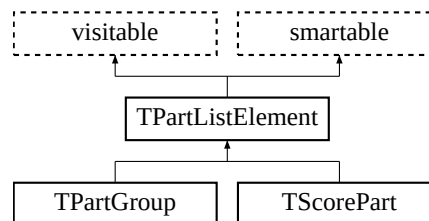
TPartList (p. 175) corresponds to the MusicXML *part-list* element as defined in score.dtd.

9.113 TPartListElement Class Reference

base class for the elements of a *part-list*.

```
#include <common.h>
```

Inheritance diagram for TPartListElement::



9.113.1 Detailed Description

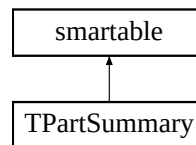
TPartListElement (p. 176) is an abstract class which purpose is to define a common type for all the elements of a *part-list* element.

9.114 TPartSummary Class Reference

Produces a summary of a MusicXML part.

```
#include <TScanVisitor.h>
```

Inheritance diagram for TPartSummary::



Public Member Functions

- const string & **getID** () const
returns the part ID
- void **setStaff** (long staff, long index)
*associates the staff to a value (a global unique index when used by **TScanVisitor** (p. 205))*
- long **getStaffIndex** (long staff)
*returns the staff associated value (a global unique index when used by **TScanVisitor** (p. 205))*
- long **getStaff** (long index) const
returns the staff number from its index in the staff list
- void **incCue** (long staff)
increments the number of cue notes for the corresponding staff
- void **incVoice** (long voice, long staff)
increments the number of notes for the corresponding voice
- long **countStaves** () const
returns the number of staves for the part
- long **countCueNotes** (long staff)
returns the number of cue notes for the staff
- long **countVoices** () const
returns the number of voices for the part
- long **countVoices** (long staff)
returns the number of voices for the staff
- long **countVoiceNotes** (long voice)
returns the number of notes on a voice
- long **countStaffNotes** (long staff)

returns the number of notes on a staff

- long **countNotes** (long voice, long staff)
returns the number of notes for both a voices and a staff
- long **getVoice** (long index) const
returns the voice number for the corresponding index
- long **getMainStaff** (long voice)
returns the main staff for a voice (the staff where most of the notes are located)

Protected Member Functions

- **TPartSummary** (const string &id)

Friends

- VEXP friend **SMARTP**< **TPartSummary** > **newPartSummary** (const string &id)

9.114.1 Detailed Description

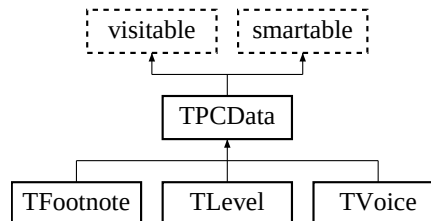
A part summary consists in a list of voices included in the part, a count of cue notes associated to each staff, a count of staves associated to a unique index in the global score context.

9.115 TPCData Class Reference

base class for all the PCDATA simple elements.

```
#include <common.h>
```

Inheritance diagram for TPCData::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const std::string &s)
- const std::string & **getData** () const
- const std::string & **getName** () const

Protected Member Functions

- **TPCData** (const std::string eltname)

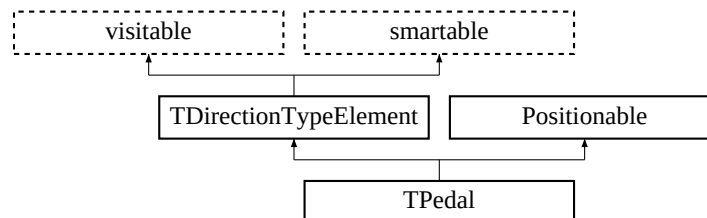
Friends

- EXP friend SMARTP< TPCData > newPCData (const std::string eltname)

9.116 TPedal Class Reference

```
#include <TOtherDirections.h>
```

Inheritance diagram for TPedal::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type st)
- void **setLine** (YesNo::type line)
- StartStop::type **getType** () const
- YesNo::type **getLine** () const

Protected Member Functions

- **TPedal** (StartStop::type t)

Friends

- EXP friend SMARTP< TPedal > **newPedal** (StartStop::type t=StartStop::undefined)

9.116.1 Detailed Description

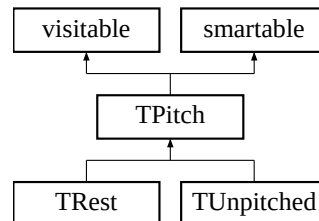
MusicXML *pedal* element.

9.117 TPitch Class Reference

The MusicXML representation of the pitch.

```
#include <TPitch.h>
```

Inheritance diagram for TPitch::



Public Types

- enum {
 undefined = 0, **C** = 1, **D**, **E**,
 F, **G**, **A**, **B**,
 last = **B**, **diatonicSteps** = last }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- int **getStep** () const
- float **getAlter** () const
- int **getOctave** () const
- void **setStep** (int step)
- void **setAlter** (float alter)
- void **setOctave** (int oct)
- TPitch & operator++** (int)
 the ++ operator adds a sharp
- TPitch & operator--** (int)
 the -- operator adds a flat
- TPitch & operator+=** (int n)
- TPitch & operator+=** (unsigned int n)
 adds n diatonic steps to the pitch field and update the octave when necessary
- TPitch & operator-=** (int n)
- TPitch & operator-=** (unsigned int n)
 subtracts n diatonic steps to the pitch field and update the octave when necessary
- unsigned short **MIDIPitch** () const
 returns the corresponding MIDI pitch

Static Public Member Functions

- static const string **xmlpitch** (int d)
convert a numeric pitch to a MusicXML string
- static int **xmlpitch** (const string str)
convert a MusicXML string to a numeric pitch

Protected Attributes

- int **fStep**
- float **fAlter**
- int **fOctave**

Friends

- EXP friend **SMARTP< TPitch > newPitch ()**

9.117.1 Detailed Description

Pitch is represented as a combination of the step of the diatonic scale, the chromatic alteration, and the octave. The step element uses the English letters A through G: in a future revision, this could expand to international namings. The alter element represents chromatic alteration in number of semitones (e.g., -1 for flat, 1 for sharp). Decimal values like 0.5 (quarter tone sharp) may be used for microtones. The octave element is represented by the numbers 0 to 9, where 4 indicates the octave started by middle C.

The MusicXML *pitch* element is defined in note.dtd.

9.118 TPlacement Class Reference

Represents the MusicXml *placement* entity.

```
#include <common.h>
```

Public Types

- enum **type** { **undefined**, **above**, **below**, **last** = below }

Public Member Functions

- **TPlacement** (int place=undefined)
- void **accept** (TScoreVisitor &visitor)
- void **set** (int place)
- int **get** () const

Static Public Member Functions

- static const string **xmlplace** (int d)
convert an integer placement to a MusicXML string
- static int **xmlplace** (const string str)
convert an MusicXML string to an integer placement

9.118.1 Detailed Description

It is defined in common.dtd.

9.119 TPosition Class Reference

Represents a MusicXML *position* entity.

```
#include <common.h>
```

Public Types

- enum {
 undefined = 0, **defaultx** = 1, **defaulty**, **relativex**,
 relativey, **last** = relativey }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **set** (int pos, long val)
- long **get** (int pos) const

Static Public Member Functions

- static const string **xmlpos** (int d)
 convert an integer position to a MusicXML string
- static int **xmlpos** (const string str)
 convert an MusicXML string to an integer position

9.119.1 Detailed Description

The position attributes are based on MuseData print suggestions. For most elements, any program will compute a default x and y position. The position attribute lets this be changed two ways. The default-x and default-y attributes change the computation of the default position. The origin becomes the left-hand side of the note or the musical position within the bar (x) and the top line of the staff (y). The relative-x and relative-y attributes change the position relative to the default position, either as computed by the individual program, or as overridden by the default-x and default-y attributes.

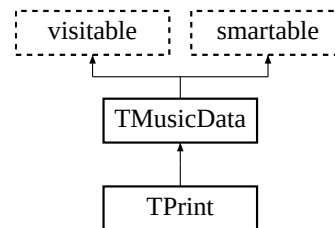
Positive x is right, negative x is left; positive y is up, negative y is down. All units are in tenths of interline space. Positions can be applied to notes, notations, directions, and stems. For stems, positive y lengthens a stem while negative y shortens it. Negative values for default-y are not allowed.

position is defined in common.dtd

9.120 TPrint Class Reference

```
#include <TOtherDirections.h>
```

Inheritance diagram for TPrint::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setStaffSpacing** (int tenth)
- void **setNewSystem** (YesNo::type ns)
- void **setNewPage** (YesNo::type ns)
- int **getStaffSpacing** () const
- YesNo::type **getNewSystem** () const
- YesNo::type **getNewPage** () const

Friends

- EXP friend SMARTP< TPrint > **newPrint** ()

9.120.1 Detailed Description

MusicXML *print* element.

The print element contains general printing parameters.

Staff spacing between multiple staves is measured in tenths of staff lines (e.g. 100 = 10 staff lines).

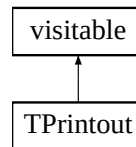
The new-system and new-page attributes indicate whether to force a system or page break, or to force the current music onto the same system or page as the preceding music. Normally this is the first music data within a measure. If used in multi-part music, they should be placed in the same positions within each part, or the results are undefined.

9.121 TPrintout Class Reference

Represents the MusicXML *printout* entity.

```
#include <common.h>
```

Inheritance diagram for TPrintout::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setPrintObject** (YesNo::type t)
- void **setPrintDot** (YesNo::type t)
- void **setPrintSpacing** (YesNo::type t)
- YesNo::type **getPrintObject** () const
- YesNo::type **getPrintDot** () const
- YesNo::type **getPrintSpacing** () const

9.121.1 Detailed Description

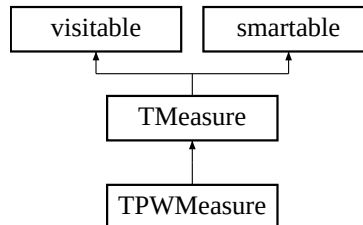
The printout entity is based on MuseData print suggestions. They allow a way to specify not to print print an object (e.g. note or rest) or its augmentation dots. This is especially useful for notes that overlap in different voices. By default, all these attributes are set to yes. For wholly invisible notes, such as those providing sound-only data, the attribute for print-spacing may be set to no so that no space is left for this note. The print-spacing value is only used if both print-object and print-dot are no. It is defined in common.dtd.

9.122 TPWMeasure Class Reference

The MusicXML partwise *measure* element.

```
#include <TScorePartwise.h>
```

Inheritance diagram for TPWMeasure::



Public Types

- enum { **allVoices** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SMusicData &data)
adds music data and returns its index;
- **vvector**< SMusicData > & **data** ()
- unsigned long **getDuration** (long voice=allVoices)

Protected Member Functions

- **TPWMeasure** (string num)
- **TPWMeasure** (long num)

Friends

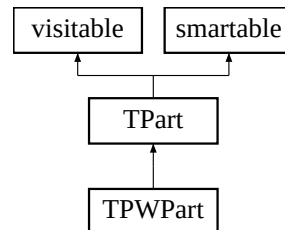
- EXP friend **SMARTP**< **TPWMeasure** > **newPWMeasure** (string num)
- EXP friend **SMARTP**< **TPWMeasure** > **newPWMeasure** (long num)

9.123 TPWPart Class Reference

The MusicXML partwise *part* element.

```
#include <TScorePartwise.h>
```

Inheritance diagram for TPWPart::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SPWMeasure &measure)
 ! adds a measure and returns its index;
- vvector< SPWMeasure > & **measures** ()

Protected Member Functions

- **TPWPart** (const string &id)

Friends

- EXP friend SMARTP< TPWPart > **newPWPart** (string id)

9.124 TRational Class Reference

Rational number representation.

```
#include <TRational.h>
```

Public Member Functions

- **TRational** (long int num=0, long int denom=1)
- **TRational** (const **TRational** &d)
- **TRational** (const string &str)
- long int **getNumerator** () const
- long int **getDenominator** () const
- void **setNumerator** (long int d)
- void **setDenominator** (long int d)
- void **set** (long int n, long int d)
- **TRational operator+** (const **TRational** &dur) const
- **TRational operator-** (const **TRational** &dur) const
- **TRational operator *** (const **TRational** &dur) const

Useful for notes with dots.

- **TRational operator/** (const **TRational** &dur) const
- **TRational operator *** (int num) const
- **TRational operator/** (int num) const
- **TRational & operator+=** (const **TRational** &dur)
- **TRational & operator-=** (const **TRational** &dur)
- **TRational & operator *=** (const **TRational** &dur)

Useful for notes with dots.

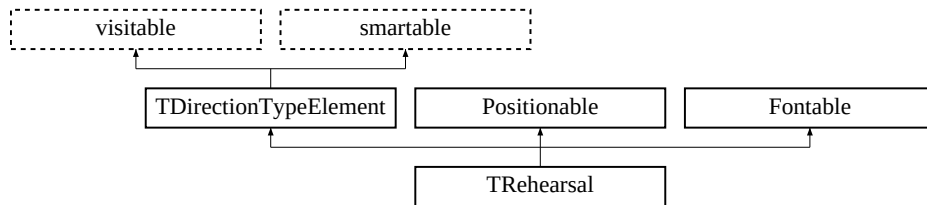
- **TRational & operator/=** (const **TRational** &dur)
- **TRational & operator *=** (long int num)
- **TRational & operator/=** (long int num)
- **TRational & operator=** (const **TRational** &dur)
- bool **operator>** (const **TRational** &dur) const
- bool **operator>=** (const **TRational** &dur) const
- bool **operator<** (const **TRational** &dur) const
- bool **operator<=** (const **TRational** &dur) const
- bool **operator==** (const **TRational** &dur) const
- bool **operator!=** (const **TRational** &dur) const
- bool **operator>** (double num) const
- bool **operator>=** (double num) const
- bool **operator<** (double num) const
- bool **operator<=** (double num) const
- bool **operator==** (double) const
- void **rationalise** ()
- **operator string** () const
- **operator double** () const
- **operator float** () const
- **operator int** () const
- string **toString** () const

- double **toDouble** () const
- float **toFloat** () const
- int **toInt** () const

9.125 TRehearsal Class Reference

```
#include <TOtherDirections.h>
```

Inheritance diagram for TRehearsal::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setData** (const string &data)
- const string & **getData** () const

Friends

- EXP friend SMARTP< TRehearsal > newRehearsal ()

9.125.1 Detailed Description

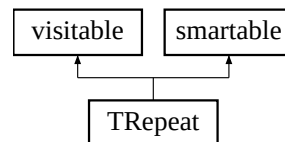
MusicXML *rehearsal* element.

9.126 TRepeat Class Reference

Represents repeat marks.

```
#include <TBarline.h>
```

Inheritance diagram for TRepeat::



Public Types

- enum { **undefined** = 0, **backward**, **forward**, **last** = forward }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- int **getDirection** () const
- int **getTimes** () const
- void **setDirection** (int dir)
- void **setTimes** (int times)

Static Public Member Functions

- static const string **xmldirection** (int d)
convert an integer direction to a MusicXML string
- static int **xmldirection** (const string str)
convert an MusicXML string to an integer direction

Protected Member Functions

- TRepeat** (int direction)

Friends

- EXP friend SMARTP< TRepeat > **newRepeat** (int direction)

9.126.1 Detailed Description

The start of the repeat has a forward direction while the end of the repeat has a backward direction. Backward repeats that are not part of an ending can use the times attribute to indicate the number of times the repeated section is played.

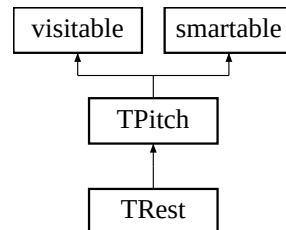
The MusicXML *repeat* element is defined in barline.dtd.

9.127 TRest Class Reference

The MusicXML representation of a rest.

```
#include <TPitch.h>
```

Inheritance diagram for TRest::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

- EXP friend SMARTP< TRest > newRest ()

9.127.1 Detailed Description

The rest element indicates notated rests or silences. Rest are usually empty, but placement on the staff can be specified using display-step and display-octave elements.

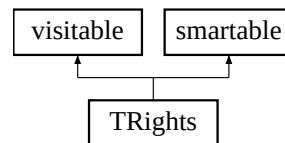
The MusicXML *rest* element is defined in note.dtd.

9.128 TRights Class Reference

Contains copyright and other intellectual property notices.

```
#include <TIdentification.h>
```

Inheritance diagram for TRights::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- const std::string & **getType** () const
- const std::string & **getRight** () const
- void **setType** (const std::string &type)
- void **setRights** (const std::string &rights)

Friends

- EXP friend SMARTP< TRights > **newRights** ()

9.128.1 Detailed Description

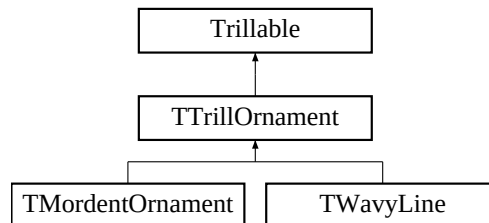
Words, music, and derivatives can have different types, so multiple rights tags with different type attributes are supported. It corresponds to the MusicXML *right* element as defined in identity.dtd.

9.129 Trillable Class Reference

base class for all elements that carry the trill-sound entity.

```
#include <common.h>
```

Inheritance diagram for Trillable::



Public Member Functions

- **TTrillSound & trill ()**

9.130 TrillStart Class Reference

provides conversions between numeric trill start notes and strings

```
#include <conversions.h>
```

Public Types

- enum **type** {
 undefined, **upper**, **main**, **below**,
 last = below }

Static Public Member Functions

- static const string **xml** (type d)
 convert a numeric start note value to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric start note value

9.131 TrillStep Class Reference

provides conversions between numeric trill step, turn and strings

```
#include <conversions.h>
```

Public Types

- enum **type** {
 undefined, **whole**, **half**, **unison**,
 none, **last** = none }

Static Public Member Functions

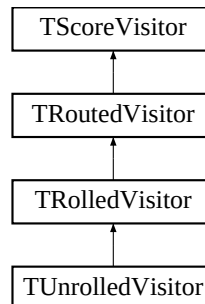
- static const string **xml** (type d)
 convert a numeric trill step or turn value to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric trill step or turn value

9.132 TRolledVisitor Class Reference

A visitor that transport a **TNodeVisitor** (p. 149) along a score.

```
#include <TRolledVisitor.h>
```

Inheritance diagram for TRolledVisitor::



Public Member Functions

- **TRolledVisitor** (**TNodeVisitor** *visitor)
- virtual void **visite** (**SAccidental** &elt)
- virtual void **visite** (**SAccidentalMark** &elt)
- virtual void **visite** (**SArticulationElement** &elt)
- virtual void **visite** (**SArticulations** &elt)
- virtual void **visite** (**SAttributes** &elt)
- virtual void **visite** (**SBackup** &elt)
- virtual void **visite** (**SBarline** &elt)
- virtual void **visite** (**SBeam** &elt)
- virtual void **visite** (**SBracket** &elt)
- virtual void **visite** (**SChord** &elt)
- virtual void **visite** (**SClef** &elt)
- virtual void **visite** (**SCoda** &elt)
- virtual void **visite** (**SCreator** &elt)
- virtual void **visite** (**SDashes** &elt)
- virtual void **visite** (**SDirection** &elt)
- virtual void **visite** (**SDirectionType** &elt)
- virtual void **visite** (**SDirective** &elt)
- virtual void **visite** (**SDynamic** &elt)
- virtual void **visite** (**SEncoding** &elt)
- virtual void **visite** (**SEnding** &elt)
- virtual void **visite** (**SExpression** &elt)
- virtual void **visite** (**SFermata** &elt)
- virtual void **visite** (**SForward** &elt)
- virtual void **visite** (**SGraceNote** &elt)
- virtual void **visite** (**SGraphNode** &elt)
- virtual void **visite** (**SIdentification** &elt)
- virtual void **visite** (**SKey** &elt)
- virtual void **visite** (**SMeasure** &elt)
- virtual void **visite** (**SMetronome** &elt)

- virtual void **visite** (**SMidiDevice** &elt)
- virtual void **visite** (**SMidiInstrument** &elt)
- virtual void **visite** (**SMordentOrnament** &elt)
- virtual void **visite** (**SNotation** &elt)
- virtual void **visite** (**SNote** &elt)
- virtual void **visite** (**SNoteHead** &elt)
- virtual void **visite** (**SOctaveShift** &elt)
- virtual void **visite** (**SOrnaments** &elt)
- virtual void **visite** (**SOrnament** &elt)
- virtual void **visite** (**SOtherOrnament** &elt)
- virtual void **visite** (**SPWMeasure** &elt)
- virtual void **visite** (**SPWPart** &elt)
- virtual void **visite** (**SPart** &elt)
- virtual void **visite** (**SPartGroup** &elt)
- virtual void **visite** (**SPartList** &elt)
- virtual void **visite** (**SPitch** &elt)
- virtual void **visite** (**SRepeat** &elt)
- virtual void **visite** (**SRest** &elt)
- virtual void **visite** (**SRights** &elt)
- virtual void **visite** (**SScoreHeader** &elt)
- virtual void **visite** (**SScoreInstrument** &elt)
- virtual void **visite** (**SScorePart** &elt)
- virtual void **visite** (**SScorePartwise** &elt)
- virtual void **visite** (**SScoreTimewise** &elt)
- virtual void **visite** (**SSegno** &elt)
- virtual void **visite** (**SSLur** &elt)
- virtual void **visite** (**SSound** &elt)
- virtual void **visite** (**SStrongAccent** &elt)
- virtual void **visite** (**STWMeasure** &elt)
- virtual void **visite** (**STWPart** &elt)
- virtual void **visite** (**STie** &elt)
- virtual void **visite** (**STimeModification** &elt)
- virtual void **visite** (**STimeSign** &elt)
- virtual void **visite** (**STranspose** &elt)
- virtual void **visite** (**STRillOrnament** &elt)
- virtual void **visite** (**STuplet** &elt)
- virtual void **visite** (**STupletDesc** &elt)
- virtual void **visite** (**SUnpitched** &elt)
- virtual void **visite** (**SWavyLine** &elt)
- virtual void **visite** (**SWedge** &elt)
- virtual void **visite** (**SWords** &elt)
- virtual void **visite** (**SWork** &elt)

Protected Attributes

- **TNodeVisitor** * **fVisitor**

9.132.1 Detailed Description

For non-terminal nodes, The **TRolledVisitor** (p. 198) calls:

- the `TNodeVisitor::visiteEnter` method
- `TRoutedVisitor::visite` on the node to visite sub-nodes
- the `TNodeVisitor::visiteLeave` method

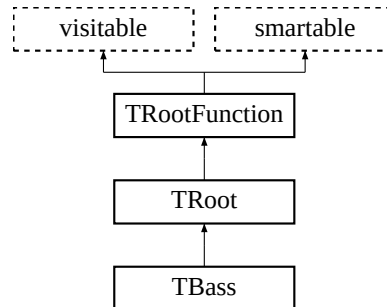
For terminal nodes, only the `TNodeVisitor::visiteEnter` method is called.

9.133 TRoot Class Reference

the root element of the *harmony-chord* entity.

```
#include <THarmony.h>
```

Inheritance diagram for TRoot::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setRoot** (const std::string &root)
- void **setAlter** (const std::string &alter)
- const std::string & **getRoot** () const
- const std::string & **getAlter** () const

Friends

- EXP friend SMARTP< TRoot > **newRoot** ()

9.133.1 Detailed Description

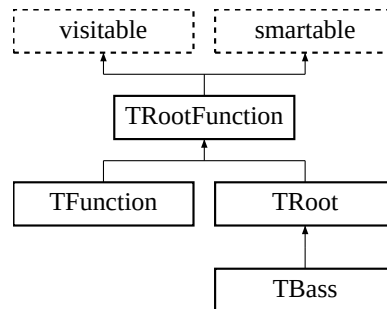
A root is a pitch name like C, D, E. Root is generally used with pop chord symbols, function with classical functional harmony. The root element has a root-step and optional root-alter similar to the step and alter elements in a pitch.

9.134 TRootFunction Class Reference

base class for the root and function elements of the *harmony-chord* entity.

```
#include <THarmony.h>
```

Inheritance diagram for TRootFunction::

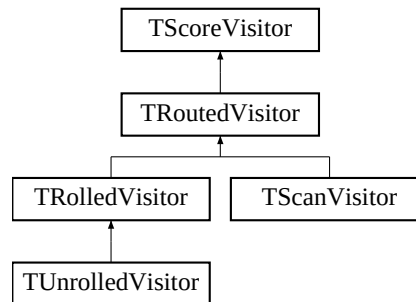


9.135 TRoutedVisitor Class Reference

A visitor that implements a traversing of a score.

```
#include <TRoutedVisitor.h>
```

Inheritance diagram for TRoutedVisitor::



Public Member Functions

- virtual void **visite** (SAccidentalMark &elt)
- virtual void **visite** (SArpeggiate &elt)
- virtual void **visite** (SNonArpeggiate &elt)
- virtual void **visite** (SArticulationElement &elt)
- virtual void **visite** (SArticulations &elt)
- virtual void **visite** (SAttributes &elt)
- virtual void **visite** (SBarline &elt)
- virtual void **visite** (SBendWithBar &elt)
- virtual void **visite** (SBend &elt)
- virtual void **visite** (SBracket &elt)
- virtual void **visite** (SChord &elt)
- virtual void **visite** (SCoda &elt)
- virtual void **visite** (SDashes &elt)
- virtual void **visite** (SDirection &elt)
- virtual void **visite** (SDirectionType &elt)
- virtual void **visite** (SDynamic &elt)
- virtual void **visite** (SFermata &elt)
- virtual void **visite** (SFiguredBass &elt)
- virtual void **visite** (SFingering &elt)
- virtual void **visite** (SFrame &elt)
- virtual void **visite** (SFrameNote &elt)
- virtual void **visite** (SGlissando &elt)
- virtual void **visite** (SGraphNode &elt)
- virtual void **visite** (SGrouping &elt)
- virtual void **visite** (SHammerPull &elt)
- virtual void **visite** (SHarmonic &elt)
- virtual void **visite** (SHarmony &elt)
- virtual void **visite** (SHarmonyChord &elt)
- virtual void **visite** (SIdentification &elt)
- virtual void **visite** (SLyric &elt)

- virtual void **visite** (**SMiscId** &elt)
- virtual void **visite** (**SMeasureStyle** &elt)
- virtual void **visite** (**SMordentOrnament** &elt)
- virtual void **visite** (**SNotation** &elt)
- virtual void **visite** (**SNote** &elt)
- virtual void **visite** (**SOctaveShift** &elt)
- virtual void **visite** (**SOrnament** &elt)
- virtual void **visite** (**SOrnaments** &elt)
- virtual void **visite** (**SOtherOrnament** &elt)
- virtual void **visite** (**SPWMeasure** &elt)
- virtual void **visite** (**SPWPart** &elt)
- virtual void **visite** (**SPartList** &elt)
- virtual void **visite** (**SRehearsal** &elt)
- virtual void **visite** (**SScoreHeader** &elt)
- virtual void **visite** (**SScorePart** &elt)
- virtual void **visite** (**SScorePartwise** &elt)
- virtual void **visite** (**SScoreTimewise** &elt)
- virtual void **visite** (**SSegno** &elt)
- virtual void **visite** (**SSlide** &elt)
- virtual void **visite** (**SSlur** &elt)
- virtual void **visite** (**SStaffDetails** &elt)
- virtual void **visite** (**SSound** &elt)
- virtual void **visite** (**SStrongAccent** &elt)
- virtual void **visite** (**STechnical** &elt)
- virtual void **visite** (**STechnicals** &elt)
- virtual void **visite** (**STWMeasure** &elt)
- virtual void **visite** (**STWPart** &elt)
- virtual void **visite** (**STie** &elt)
- virtual void **visite** (**STRillOrnament** &elt)
- virtual void **visite** (**STuplet** &elt)
- virtual void **visite** (**SWavyLine** &elt)
- virtual void **visite** (**SWedge** &elt)
- virtual void **visite** (**SWords** &elt)

9.135.1 Detailed Description

A **TRoutedVisitor** (p. 203) provides a path for traversing of a score tree. Each implemented *visite* method calls *accept()* for its subclasses. Order of these calls is the following:

- subclasses representing attributes are called first,
- subclasses representing elements are called in the MusicXML defined order.

9.135.2 Member Function Documentation

9.135.2.1 void visite (SLyric & elt) [virtual]

Todo

route to the font associated to the text elements

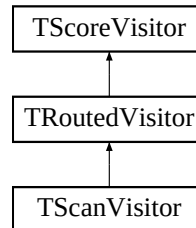
Reimplemented from **TScoreVisitor** (p. 212).

9.136 TScanVisitor Class Reference

A visitor that only scans a score to gather information.

```
#include <TScanVisitor.h>
```

Inheritance diagram for TScanVisitor::



Public Member Functions

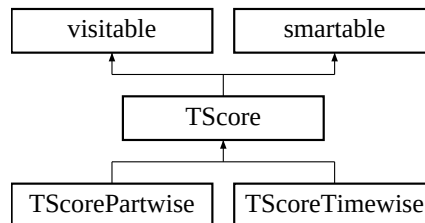
- virtual void **visite** (SNote &elt)
- virtual void **visite** (SPWPart &elt)
- long **countParts** () const
- map< string, SPartSummary > & **summary** ()
- const SPartSummary & **getPartSummary** (const string &partID)
- const SPartSummary & **current** () const

9.137 TScore Class Reference

a MusicXML score.

```
#include <TScore.h>
```

Inheritance diagram for TScore::



Public Member Functions

- Sxmlheader & xmlheader ()
- SScoreHeader & scoreHeader ()

Protected Attributes

- Sxmlheader fHeader
- SScoreHeader fScoreHeader

9.137.1 Detailed Description

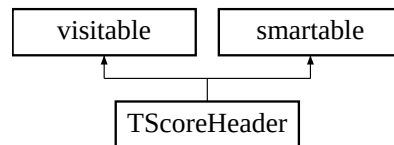
Base class for the partwise and timewise MusicXML formats.

9.138 TScoreHeader Class Reference

Contains basic score meta-data plus the part list.

```
#include <TScoreHeader.h>
```

Inheritance diagram for TScoreHeader::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setMovementNum** (const string &num)
- void **setMovementTitle** (const string &title)
- const string & **getMovementNum** ()
- const string & **getMovementTitle** ()
- SWork & **work** ()
- SIdentification & **identification** ()
- SPartList & **partList** ()

Friends

- EXP friend SMARTP< TScoreHeader > **newScoreHeader** ()

9.138.1 Detailed Description

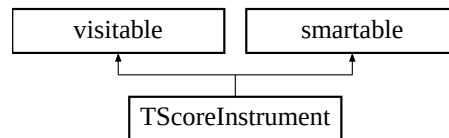
Meta-data includes work and movement name, movement-wide identification data. It corresponds to the MusicXML *score-header* entity as defined in score.dtd.

9.139 TScoreInstrument Class Reference

Represents an instrument on the score.

```
#include <TScoreInstrument.h>
```

Inheritance diagram for TScoreInstrument::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setID** (const std::string &id)
a MusicXML required attribute used for midi-instrument assignment.
- void **setName** (const std::string &name)
the instrument name.
- void **setAbbrev** (const std::string &abbrev)
the optional name abbreviation.
- const std::string & **getID** () const
- const std::string & **getName** () const
- const std::string & **getAbbrev** () const

Protected Member Functions

- TScoreInstrument (std::string id)

Friends

- EXP friend SMARTP< TScoreInstrument > newScoreInstrument (std::string id)

9.139.1 Detailed Description

The score-instrument element allows for multiple instruments per score-part. As with the score-part element, each score-instrument has a required ID attribute, a name, and an optional abbreviation. A score-instrument element is also required if the score specifies MIDI channels, banks, or programs. An initial midi-instrument assignment can also be made here. MusicXML software should be able to automatically assign reasonable channels and instruments without these elements in simple cases, such as where part names match General MIDI instrument names.

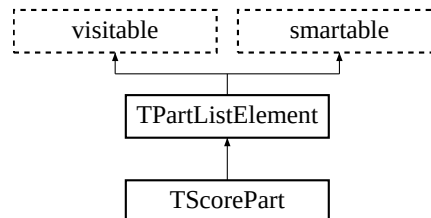
The MusicXML *score-instrument* is defined in score.dtd.

9.140 TScorePart Class Reference

Identifies a musical part in a score.

```
#include <TScorePart.h>
```

Inheritance diagram for TScorePart::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- const string & **getID** () const
- const string & **getPartName** () const
- const string & **getPartAbbrev** () const
- **vvector**< SScoreInstrument > & **scoreInstruments** ()
- **vvector**< SMidiInstrument > & **midiInstruments** ()
- void **setID** (const string &id)
- void **setPartName** (const string &name)
- void **setPartAbbrev** (const string &abbrev)
- long **add** (const SScoreInstrument &instr)
- long **add** (const SMidiInstrument &instr)
- **SIdentification** & **identification** ()
- **SMidiDevice** & **mididevice** ()

Protected Member Functions

- **TScorePart** (string id)

Friends

- EXP friend SMARTP< TScorePart > **newScorePart** (string id)

9.140.1 Detailed Description

Each part has an ID that is used later within the musical data. Since parts may be encoded separately and combined later, identification elements are present at both the score and score-part levels.

Each MusicXML part corresponds to a track in a Standard MIDI Format 1 file. The score-instrument elements are used when there are multiple instruments per track. The midi-device element is used to make a MIDI device or port assignment for the given track. Initial midi-instrument assignments may be made here as well.

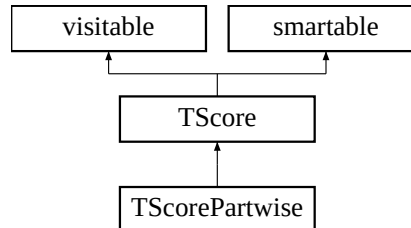
TScorePart (p. 209) corresponds to the MusicXML *score-part* element as defined in score.dtd.

9.141 TScorePartwise Class Reference

a MusicXML partwise score.

```
#include <TScorePartwise.h>
```

Inheritance diagram for TScorePartwise::



Public Member Functions

- long **add** (const **SPWPart** &part)
! adds a part and returns its index;
- **vvector**< **SPWPart** > & **partList** ()
- virtual void **accept** (**TScoreVisitor** &visitor)

Friends

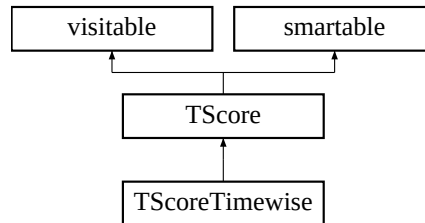
- EXP friend **SMARTP**< **TScorePartwise** > **newScorePartwise** ()

9.142 TScoreTimewise Class Reference

a MusicXML timewise score.

```
#include <TScoreTimewise.h>
```

Inheritance diagram for TScoreTimewise::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const STWMeasure &part)
- **vvector**< STWMeasure > & **measureList** ()

Friends

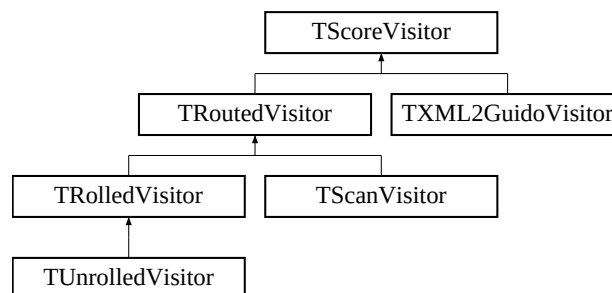
- EXP friend SMARTP< TScoreTimewise > **newScoreTimewise** ()

9.143 TScoreVisitor Class Reference

A base class to implement MusicXML score visitors.

```
#include <TScoreVisitor.h>
```

Inheritance diagram for TScoreVisitor::



Public Member Functions

- virtual void **visite** (SAccidental &elt)
- virtual void **visite** (SAccidentalMark &elt)
- virtual void **visite** (SArpeggiate &elt)
- virtual void **visite** (SNonArpeggiate &elt)
- virtual void **visite** (SArticulationElement &elt)
- virtual void **visite** (SArticulations &elt)
- virtual void **visite** (SAttributes &elt)
- virtual void **visite** (SBackup &elt)
- virtual void **visite** (SBarline &elt)
- virtual void **visite** (SBass &elt)
- virtual void **visite** (SBeam &elt)
- virtual void **visite** (SBeatRepeat &elt)
- virtual void **visite** (SBendWithBar &elt)
- virtual void **visite** (SBend &elt)
- virtual void **visite** (SBracket &elt)
- virtual void **visite** (SChord &elt)
- virtual void **visite** (SClef &elt)
- virtual void **visite** (SCoda &elt)
- virtual void **visite** (SCreator &elt)
- virtual void **visite** (SDashes &elt)
- virtual void **visite** (SDegree &elt)
- virtual void **visite** (SDirection &elt)
- virtual void **visite** (SDirectionType &elt)
- virtual void **visite** (SDirective &elt)
- virtual void **visite** (SDynamic &elt)
- virtual void **visite** (SEncoding &elt)
- virtual void **visite** (SEnding &elt)
- virtual void **visite** (SExpression &elt)
- virtual void **visite** (SFermata &elt)
- virtual void **visite** (SFigure &elt)

- virtual void **visite** (SFeature &elt)
- virtual void **visite** (SFiguredBass &elt)
- virtual void **visite** (SFingering &elt)
- virtual void **visite** (SFootnote &elt)
- virtual void **visite** (SFrame &elt)
- virtual void **visite** (SFrameNote &elt)
- virtual void **visite** (SFunction &elt)
- virtual void **visite** (SForward &elt)
- virtual void **visite** (SGlissando &elt)
- virtual void **visite** (SGraceNote &elt)
- virtual void **visite** (SGraphNode &elt)
- virtual void **visite** (SGrouping &elt)
- virtual void **visite** (SHammerPull &elt)
- virtual void **visite** (SHarmonic &elt)
- virtual void **visite** (SHarmony &elt)
- virtual void **visite** (SHarmonyChord &elt)
- virtual void **visite** (SIdentification &elt)
- virtual void **visite** (SKey &elt)
- virtual void **visite** (SLevel &elt)
- virtual void **visite** (SLyric &elt)
- virtual void **visite** (SMeasure &elt)
- virtual void **visite** (SMeasureRepeat &elt)
- virtual void **visite** (SMeasureStyle &elt)
- virtual void **visite** (SMeasureStyleElement &elt)
- virtual void **visite** (SMetronome &elt)
- virtual void **visite** (SMidiDevice &elt)
- virtual void **visite** (SMidiInstrument &elt)
- virtual void **visite** (SMiscField &elt)
- virtual void **visite** (SMiscId &elt)
- virtual void **visite** (SMordentOrnament &elt)
- virtual void **visite** (SMultipleLyricPart &elt)
- virtual void **visite** (SMultipleRest &elt)
- virtual void **visite** (SNotation &elt)
- virtual void **visite** (SNote &elt)
- virtual void **visite** (SNoteHead &elt)
- virtual void **visite** (SOctaveShift &elt)
- virtual void **visite** (SOrnament &elt)
- virtual void **visite** (SOrnaments &elt)
- virtual void **visite** (SOtherArticulation &elt)
- virtual void **visite** (SOtherDirection &elt)
- virtual void **visite** (SOtherDirections &elt)
- virtual void **visite** (SOtherNotation &elt)
- virtual void **visite** (SOtherOrnament &elt)
- virtual void **visite** (SPWMeasure &elt)
- virtual void **visite** (SPWPart &elt)
- virtual void **visite** (SPart &elt)
- virtual void **visite** (SPartGroup &elt)
- virtual void **visite** (SPartList &elt)
- virtual void **visite** (SPCData &elt)
- virtual void **visite** (SPedal &elt)

- virtual void **visite** (**SPitch** &elt)
- virtual void **visite** (**SPrint** &elt)
- virtual void **visite** (**SRehearsal** &elt)
- virtual void **visite** (**SRepeat** &elt)
- virtual void **visite** (**SRest** &elt)
- virtual void **visite** (**SRights** &elt)
- virtual void **visite** (**SRoot** &elt)
- virtual void **visite** (**SScoreHeader** &elt)
- virtual void **visite** (**SScoreInstrument** &elt)
- virtual void **visite** (**SScorePart** &elt)
- virtual void **visite** (**SScorePartwise** &elt)
- virtual void **visite** (**SScoreTimewise** &elt)
- virtual void **visite** (**SSegno** &elt)
- virtual void **visite** (**SSlash** &elt)
- virtual void **visite** (**SSlide** &elt)
- virtual void **visite** (**SSlur** &elt)
- virtual void **visite** (**SStaffDetails** &elt)
- virtual void **visite** (**SStaffTuning** &elt)
- virtual void **visite** (**SSupport** &elt)
- virtual void **visite** (**SSound** &elt)
- virtual void **visite** (**SStrongAccent** &elt)
- virtual void **visite** (**STechnical** &elt)
- virtual void **visite** (**STechnicals** &elt)
- virtual void **visite** (**STWMeasure** &elt)
- virtual void **visite** (**STWPart** &elt)
- virtual void **visite** (**STie** &elt)
- virtual void **visite** (**STimeModification** &elt)
- virtual void **visite** (**STimeSign** &elt)
- virtual void **visite** (**STranspose** &elt)
- virtual void **visite** (**STrillOrnament** &elt)
- virtual void **visite** (**STuplet** &elt)
- virtual void **visite** (**STupletDesc** &elt)
- virtual void **visite** (**SUnpitched** &elt)
- virtual void **visite** (**SVoice** &elt)
- virtual void **visite** (**SWavyLine** &elt)
- virtual void **visite** (**SWedge** &elt)
- virtual void **visite** (**SWords** &elt)
- virtual void **visite** (**SWork** &elt)
- virtual void **visite** (**TOrientation** *elt)
- virtual void **visite** (**TPlacement** *elt)
- virtual void **visite** (**TPosition** *elt)
- virtual void **visite** (**TTrillSound** *elt)
- virtual void **visite** (**TBendSound** *elt)
- virtual void **visite** (**TFont** *elt)
- virtual void **visite** (**TPrintout** *elt)
- virtual void **visite** (**TEditorial** *elt)
- virtual void **visite** (**TBezier** *elt)

9.143.1 Detailed Description

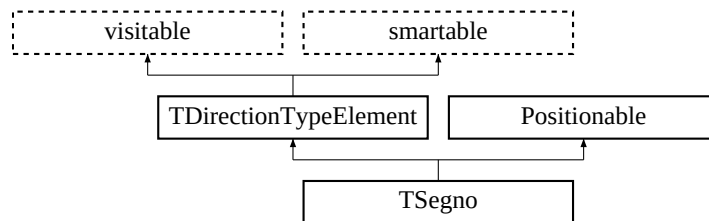
TScoreVisitor (p.212) implements the concept of visitors as defined in *Design Patterns - Elements of Reusable Object-Oriented Software* E. Gamma, R.Helm, R.Johnson, J.Vlissides - Addison-Wesley 1999

9.144 TSegno Class Reference

Represents the MusicXML *segno* element.

```
#include <common.h>
```

Inheritance diagram for TSegno::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

- EXP friend SMARTP< TSegno > newSegno ()

9.144.1 Detailed Description

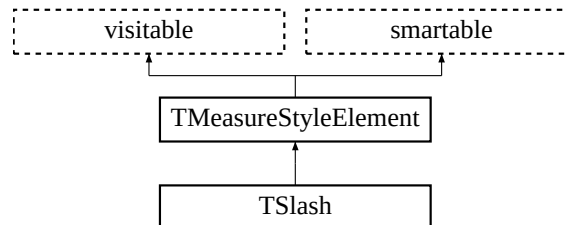
TSegno (p. 215) is a visual indicator only. It is defined in common.dtd.

9.145 TSlash Class Reference

the slash element (for measure style).

```
#include <TMeasureStyle.h>
```

Inheritance diagram for TSlash::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setUseDots** (YesNo::type t)
- void **setUseStems** (YesNo::type t)
- void **setStartStop** (StartStop::type t)
- YesNo::type **getUseDots** () const
- YesNo::type **getUseStems** () const
- StartStop::type **getStartStop** () const

Protected Member Functions

- **TSlash** (StartStop::type t)

Friends

- EXP friend SMARTP< TSlash > **newSlash** (StartStop::type type=StartStop::undefined)

9.145.1 Detailed Description

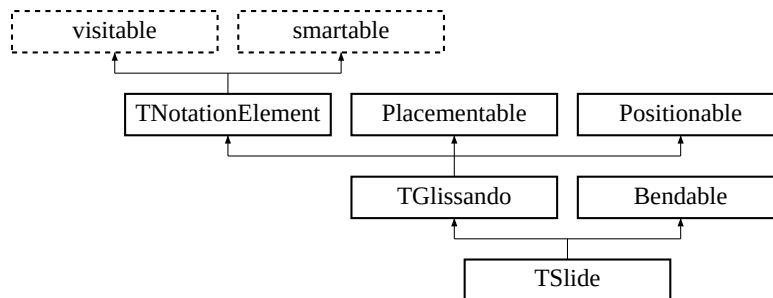
The slash element is used to indicate that slash notation is to be used. If the slash is on every beat, use-stems is no (the default). To indicate rhythms but not pitches, use-stems is set to yes. The type attribute indicates whether this is the start or stop of a slash notation style. The use-dots attribute works as for the beat-repeat element, and only has effect if use-stems is no.

9.146 TSlide Class Reference

slide representation

```
#include <TNotation.h>
```

Inheritance diagram for TSlide::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Protected Member Functions

- TSlide (StartStop::type type)

Friends

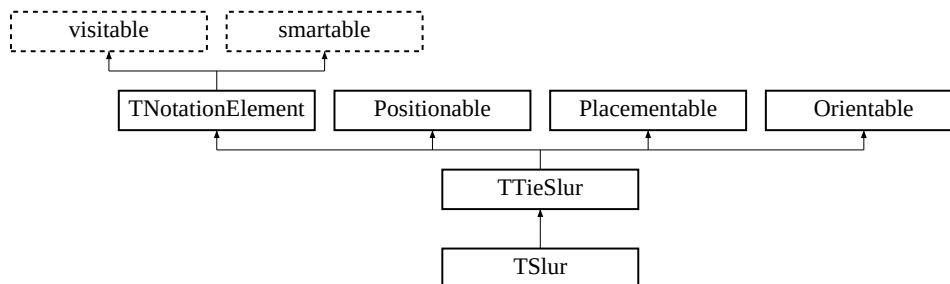
- EXP friend SMARTP< TSlide > newSlide (StartStop::type type=StartStop::undefined)

9.147 TSlur Class Reference

Slur representation.

```
#include <TNotation.h>
```

Inheritance diagram for TSlur::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- TBezier & bezier ()

Protected Member Functions

- TSlur (StartStop::type type)

Friends

- EXP friend SMARTP< TSlur > newSlur (StartStop::type type=StartStop::undefined)

9.147.1 Detailed Description

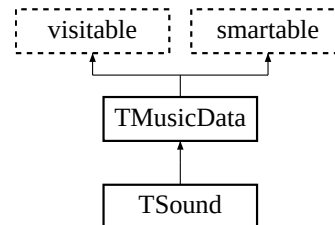
MusicXML *slur* are defined in note.dtd.

9.148 TSound Class Reference

Represents the MusicXML *sound* element.

```
#include <TSound.h>
```

Inheritance diagram for TSound::



Public Types

- enum { **yes** = -2, **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setTempo** (long tempo)
Tempo is expressed in quarter notes per minute.
- void **setDynamics** (long dyn)
Dynamics (or MIDI velocity) are expressed as a percentage of the default forte value (90 for MIDI).
- void **setDacapo** (YesNo::type yn)
Dacapo indicates to go back to the beginning of the movement.
- void **setSegno** (long segno)
a segno mark. The value can be used to distinguish multiple jumps.
- void **setDalsegno** (long ds)
dalsegno is used for backward jump to a segno sign. The value can be used to distinguish multiple jumps.
- void **setCoda** (long coda)
a coda mark. The value can be used to distinguish multiple jumps.
- void **setTocoda** (long tc)
tocoda is used for forward jump to a coda sign. The value can be used to distinguish multiple jumps.
- void **setDivisions** (long div)
to be used with segno or coda attributes to avoid computation of divisions.
- void **setForwardRepeat** (YesNo::type yn)
Forward-repeat is used when a forward repeat sign is implied, and usually follows a bar line.

- void **setFine** (long fine)
The fine attribute follows the final note or rest in a movement with a da capo direction. It may take the special yes value.
- void **setTimeOnly** (long time)
If the sound element applies only one time through a repeat, the time-only attribute indicates which time to apply the sound element.
- void **setPizzicato** (YesNo::type pizz)
Pizzicato in a sound element effects all following notes. true indicates pizzicato, false indicates arco.
- void **setPan** (long pan)
- void **setElevation** (long el)
see setPan.
- void **setDamperPedal** (YesNo::type yn)
- void **setSoftPedal** (YesNo::type yn)
see setDamperPedal.
- void **setSostenutoPedal** (YesNo::type yn)
see setDamperPedal.
- long **add** (const **SMidiInstrument** &instr)
- long **getTempo** () const
- long **getDynamics** () const
- YesNo::type **getDacapo** () const
- long **getSegno** () const
- long **getDalsegno** () const
- long **getCoda** () const
- long **getTocoda** () const
- long **getDivisions** () const
- YesNo::type **getForwardRepeat** () const
- long **getFine** () const
- long **getTimeOnly** () const
- YesNo::type **getPizzicato** () const
- long **getPan** () const
- long **getElevation** () const
- YesNo::type **getDamperPedal** () const
- YesNo::type **getSoftPedal** () const
- YesNo::type **getSostenutoPedal** () const
- **vvector< SMidiInstrument > & instrumentList** ()

Protected Member Functions

- void **init** ()

Friends

- EXP friend **SMARTP< TSound > newSound** ()

9.148.1 Detailed Description

The sound element contains general playback parameters, based on MuseData's C0 sound suggestions. They can stand alone within a part/measure, or be a component element within a direction.

Segno and dalsegno are used for backwards jumps to a segno sign; coda and tocoda are used for forward jumps to a coda sign. If there are multiple jumps, the value of these parameters can be used to name and distinguish them. If segno or coda is used, the divisions attribute can also be used to indicate the number of divisions per quarter note. Otherwise sound and MIDI generating programs may have to recompute this.

A dalsegno or dacapo attribute indicates that the jump should occur the first time through; a tocoda attribute indicates that the jump should occur the second time through.

The fine attribute follows the final note or rest in a movement with a da capo direction. If numeric, the value represents the actual duration of the final note or rest, which can be ambiguous in written notation and different among parts and voices. The value may also be "yes" to indicate no change to the final duration.

The MusicXML *sound* element is defined in direction.dtd. All its attributes are implied.

9.148.2 Member Function Documentation

9.148.2.1 void setPan (long *pan*)

Pan and elevation allow placing of sound in a 3-D space relative to the listener. Both are expressed in degrees ranging from -180 to 180. For pan, 0 is straight ahead, -90 is hard left, 90 is hard right, and -180 and 180 are directly behind the listener. For elevation, 0 is level with the listener, 90 is directly above, and -90 is directly below.

9.148.2.2 void setDamperPedal (YesNo::type *yn*)

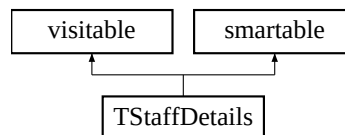
The damper-pedal, soft-pedal, and sostenuto-pedal attributes effect playback of the three common piano pedals and their MIDI controller equivalents. Yes indicates the pedal is depressed; no indicates the pedal is released.

9.149 TStaffDetails Class Reference

Represents staff details.

```
#include <TStaffDetails.h>
```

Inheritance diagram for TStaffDetails::



Public Types

- enum { **undefined** = -1, **numbers**, **letters** }
- enum **type** {
 undefType, **ossia**, **cue**, **editorial**,
 regular, **alternate**, **last** = alternate }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (type t)
- void **setNumber** (int n)
- void **setShowFret** (int t)
- void **setLines** (int n)
- void **setCapo** (const string &capo)
- void **add** (SStaffTuning st)
- int **getNumber** () const
- int **getShowFret** () const
- int **getLines** () const
- type **getType** () const
- const string & **getCapo** () const
- **vvector**< SStaffTuning > & **tuning** ()

Static Public Member Functions

- static const string **xml** (type d)
 convert a numeric staff type to a MusicXML string
- static type **xml** (const string str)
 convert a MusicXML string to a numeric staff type

Friends

- EXP friend SMARTP< TStaffDetails > **newStaffDetails** ()

9.149.1 Detailed Description

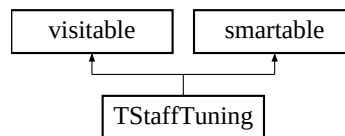
The staff-details element is used to indicate different types of staves. The staff-type element can be ossia, cue, editorial, regular, or alternate. An alternate staff indicates one that shares the same musical data as the prior staff, but displayed differently (e.g., treble and bass clef, standard notation and tab). The staff-lines element specifies the number of lines for non 5-line staves. The staff-tuning and capo elements are used to specify tuning when using tablature notation. The optional number attribute specifies the staff number from top to bottom on the system, as with clef. The optional show-frets attribute indicates whether to show tablature frets as numbers (0, 1, 2) or letters (a, b, c). The default choice is numbers.

9.150 TStaffTuning Class Reference

the staff tuning element (for staff details)

```
#include <TStaffDetails.h>
```

Inheritance diagram for TStaffTuning::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- const string & **getStep** () const
- float **getAlter** () const
- int **getOctave** () const
- int **getLine** () const
- void **setStep** (const string &step)
- void **setAlter** (float alter)
- void **setOctave** (int oct)
- void **setLine** (int line)

Protected Member Functions

- **TStaffTuning** (int line)

Friends

- EXP friend SMARTP< TStaffTuning > **newStaffTuning** (int line)

9.150.1 Detailed Description

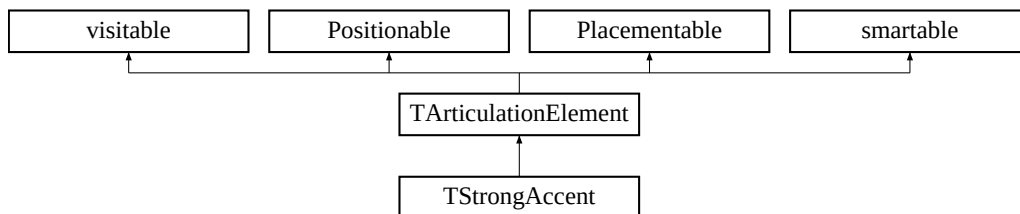
The tuning-step, tuning-alter, and tuning-octave elements are represented like the step, alter, and octave elements, with different names to reflect their different function. Staff lines are numbered from bottom to top.

9.151 TStrongAccent Class Reference

a strong accent representation.

```
#include <TNotation.h>
```

Inheritance diagram for TStrongAccent::



Public Types

- enum { **undefined**, **up** = 1, **down**, **last** = down }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- int **getType** () const
- void **setType** (int type)

Static Public Member Functions

- static const string **xmltype** (int d)
convert a numeric accent type to a MusicXML string
- static int **xmltype** (const string str)
convert an MusicXML string to a numeric accent type

Friends

- EXP friend SMARTP< TStrongAccent > newStrongAccent ()

9.151.1 Detailed Description

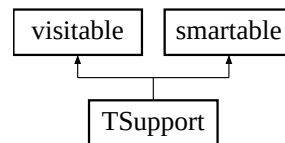
a strong accent is an articulation but have an additional type attributes.

9.152 TSupport Class Reference

the MusciXML *support* element.

```
#include <TIdentification.h>
```

Inheritance diagram for TSupport::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- YesNo::type **getType** () const
- const std::string & **getElement** () const
- void **setType** (YesNo::type type)
- void **setElement** (const std::string &element)

Protected Member Functions

- **TSupport** (YesNo::type type, const string &element)

Friends

- EXP friend SMARTP< TSupport > **newSupport** (YesNo::type type, const string &element)

9.152.1 Detailed Description

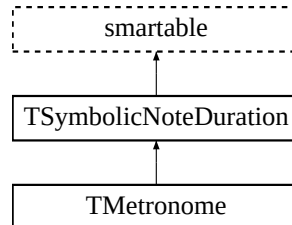
The supports element indicates if the encoding supports a particular MusicXML element. This is recommended for elements like beam, stem, and accidental, where the absence of an element is ambiguous if you do not know if the encoding supports that element.

9.153 TSymbolicNoteDuration Class Reference

the base class for symbolic note duration

```
#include <TDirection.h>
```

Inheritance diagram for TSymbolicNoteDuration::



Public Member Functions

- void **setBeat** (NoteType::type beatUnit)
- void **setDots** (unsigned dots)
- NoteType::type **getBeat** () const
- unsigned **getDots** () const

Friends

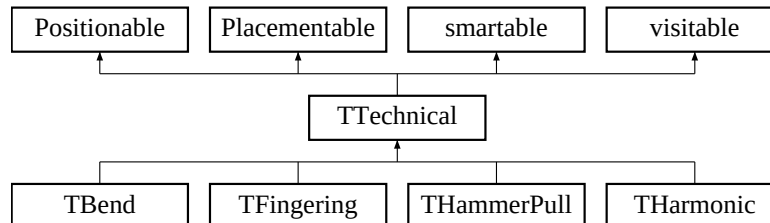
- EXP friend SMARTP< TSymbolicNoteDuration > newSymbolicNoteDuration ()

9.154 TTechnical Class Reference

basic technical representation

```
#include <TTechnical.h>
```

Inheritance diagram for TTechnical::



Public Types

- enum **type** {
 undefined, **up_bow**, **down_bow**, **harmonic**,
 open_string, **thumb_position**, **fingering**, **double_tongue**,
 triple_tongue, **stopped**, **snap_pizzicato**, **fret**,
 _string, **hammer_on**, **pull_off**, **bend**,
 tap, **heel**, **toe**, **fingernails**,
 other_technical, **last** = other_technical }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (type t)
- type **getType** () const
- void **setData** (const std::string &data)
- const std::string & **getData** () const

Static Public Member Functions

- static const std::string **xml** (type d)
 convert a numeric technical type to a MusicXML string
- static type **xml** (const std::string str)
 convert a MusicXML string to a numeric technical type
- static const string * **getTechnicalStrings** ()

Protected Member Functions

- TTechnical** (type t)

Friends

- EXP friend SMARTP< TTechnical > newTechnical (type t)

9.154.1 Detailed Description

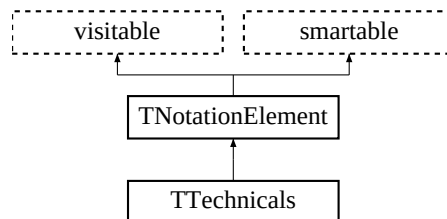
The basic **TTechnical** (p. 228) class is enough to cover almost all of the MusicXML defined technicals which are empty elements including position and placement entities. For simplification, a data field is included and the string and fret elements are embedded although they don't carry a position and a placement.

9.155 TTechnicals Class Reference

MusicXML *technical* element.

```
#include <TTechnical.h>
```

Inheritance diagram for TTechnicals::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **add** (const STechnical &elt)
- **vvector**< STechnical > **technicals** ()

Friends

- EXP friend SMARTP< TTechnicals > **newTechnicals** ()

9.155.1 Detailed Description

Technical indications give performance information for individual instruments. The current elements are weighted towards keyboard and string instruments.

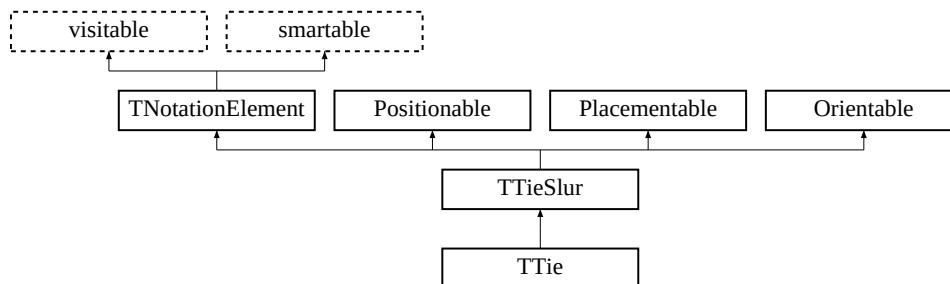
The technical element is a list of basic technical annotations.

9.156 TTie Class Reference

Ties representation.

```
#include <TNotation.h>
```

Inheritance diagram for TTie::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Protected Member Functions

- **TTie** (StartStop::type type)

Friends

- EXP friend SMARTP< TTie > **newTie** (StartStop::type type=StartStop::undefined)

9.156.1 Detailed Description

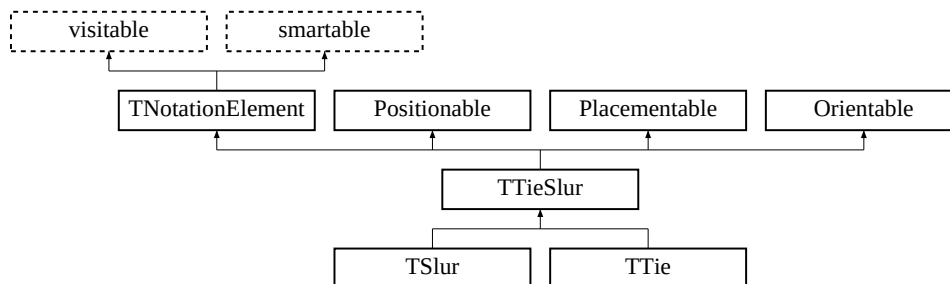
MusicXML *ties* are defined in note.dtd.

9.157 TTieSlur Class Reference

base class for ties and slurs

```
#include <TNotation.h>
```

Inheritance diagram for TTieSlur::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)=0
- StartStop::type **getType** () const
- int **getNumber** () const
- LineType::type **getLineType** () const
- void **setType** (StartStop::type type)
- void **setNumber** (int num)
- void **setLineType** (LineType::type type)
- const string & **getElement** () const

Protected Member Functions

- **TTieSlur** (string elt, StartStop::type type)

9.157.1 Detailed Description

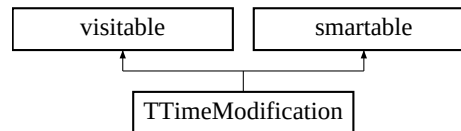
Ties and slurs share common attributes and are represented by a single object derived into **TTie** (p. 231) and **TSlur** (p. 218).

9.158 TTimeModification Class Reference

Represents tuplets and other durational changes.

```
#include <TTimeModification.h>
```

Inheritance diagram for TTimeModification::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setActualNotes** (long v)
- void **setNormalNotes** (long v)
- void **setNormalType** (NoteType::type v)
- void **setNormalDots** (long v)
- long **getActualNotes** () const
- long **getNormalNotes** () const
- NoteType::type **getNormalType** () const
- long **getNormalDots** () const

Friends

- EXP friend SMARTP< TTimeModification > **newTimeModification** ()

9.158.1 Detailed Description

The actual-notes element describes how many notes are played in the time usually occupied by the number of normal-notes. If the normal-notes type is different than the current note type (e.g., a quarter note within an eighth note triplet), then the normal-notes type (e.g. eighth) is specified in the normal-type and normal-dot elements.

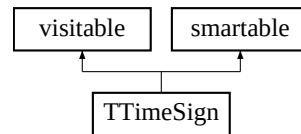
TTimeModification (p. 233) corresponds to the MusicXML *time-modification* as defined in note.dtd.

9.159 TTimeSign Class Reference

Represents time signatures.

```
#include <TTimeSign.h>
```

Inheritance diagram for TTimeSign::



Public Types

- enum {
 undefined = -1, **groupbeats** = -1, **common** = 1, **cut**,
 single_number, **normal**, **last** = normal }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setSymbol** (int symbol)
- int **getSymbol** () const
- long **add** (long beat, long type)
 beat type must be groupbeat for the first elements of a composite like 3+2/8
- bool **measured** () const
 senza measures are implicitly denoted by an empty beat list
- TRational **rational** ()
 returns the time signature as a rational number (senza measure is -1/1)
- vector< long > & **getBeats** ()
- vector< long > & **getBeatTypes** ()

Static Public Member Functions

- static const string **xmlsymbol** (int d)
 convert an integer symbol to a MusicXML string
- static int **xmlsymbol** (const string str)
 convert an MusicXML string to an integer symbol

Protected Member Functions

- **TTimeSign ()**
empty constructor denotes unmeasured time

Friends

- EXP friend **SMARTP< TTimeSign > newTimeSign ()**

9.159.1 Detailed Description

Time signatures are represented by two elements. The beats element indicates the number of beats, as found in the numerator of a time signature. The beat-type element indicates the beat unit, as found in the denominator of a time signature. The symbol attribute is used to indicate another notation beyond a fraction: the common and cut time symbols, as well as a single number with an implied denominator. Normal (a fraction) is the implied symbol type if none is specified. Multiple pairs of beat and beat-type elements are used for composite time signatures with multiple denominators, such as 2/4 + 3/8. A composite such as 3+2/8 requires only one beat/beat-type pair. A senza-misura element explicitly indicates that no time signature is present.

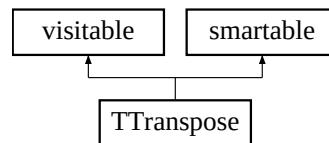
The MusicXML *time* element is defined in attributes.dtd.

9.160 TTranspose Class Reference

used for transposing instruments.

```
#include <TTranspose.h>
```

Inheritance diagram for TTranspose::



Public Types

- enum { **undefined** = 0xffff }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setDiatonic** (long diat)
- void **setChromatic** (long chrom)
- void **setOctaveChge** (long oct)
- void **setDouble** (long dbl)
- long **getDiatonic** () const
- long **getChromatic** () const
- long **getOctaveChge** () const
- long **getDouble** () const

Friends

- EXP friend SMARTP< TTranspose > newTranspose ()

9.160.1 Detailed Description

If the part is being encoded for a transposing instrument in written vs. concert pitch, the transposition must be encoded in the transpose element. The transposition is represented by chromatic steps (required) and three optional elements: diatonic pitch steps, octave changes, and doubling an octave down. The chromatic and octave-change elements are numeric values added to the encoded pitch data to create the sounding pitch. The diatonic element is also numeric and allows for correct spelling of enharmonic transpositions.

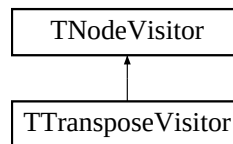
The MusicXML *transpose* element is defined in attributes.dtd.

9.161 TTransposeVisitor Class Reference

A visitor that operates a diatonic transposition of a score.

```
#include <TTransposeVisitor.h>
```

Inheritance diagram for TTransposeVisitor::



Public Member Functions

- **TTransposeVisitor** (int steps, bool generateTransposeElement)
- virtual void **visiteEnter** (SKey &elt)
- virtual void **visiteEnter** (SNote &elt)
- virtual void **visiteEnter** (SOornament &elt)
- virtual void **visiteEnter** (SWavyLine &elt)
- virtual void **visiteEnter** (SMordentOrnament &elt)
- virtual void **visiteEnter** (STrillOrnament &elt)
- virtual void **visiteEnter** (SOtherOrnament &elt)
- virtual void **visiteEnter** (STranspose &elt)
- virtual void **visiteEnter** (SPWMeasure &elt)
- virtual void **visiteEnter** (SPWPart &elt)
- virtual void **visiteLeave** (SAttributes &elt)

Static Public Member Functions

- static int **getOctave** (Chromatic steps)
- static Chromatic **getOctaveStep** (Chromatic steps)
- static int **getKey** (Chromatic steps)
- static Diatonic **getDiatonics** (Chromatic steps, int key)
- static int **getAccidental** (Diatonic pitch, int key)
- static int **incPitch** (Diatonic &pitch, int &octave)
- static int **decPitch** (Diatonic &pitch, int &octave)
- static int **transposePitch** (Diatonic &pitch, float &alter, int &octave, Chromatic steps)

Protected Member Functions

- virtual void **changePitch** (SPitch &elt)
- virtual void **setTranspose** (STranspose &dst)
- virtual void **setAccident** (SAccidental &acc, float alter)

Protected Attributes

- Chromatic **fChromaticSteps**
- Chromatic **fCurrentSteps**
- Diatonic **fDiatonicSteps**
- Diatonic **fCurrentDiatonics**
- int **fOctaveChange**
- int **fCurrentOctaveChange**
- int **fKeySign**
- int **fCurrentKeySign**
- int **fTransposeKey**
- bool **fGenerateTransposeElement**
- int **fMeasureNum**
- int **fTransposeVisited**

9.161.1 Detailed Description

The **TTransposeVisitor** (p. 237) computes a diatonic transposition of a score. The transposition interval is specified as a number of chromatic steps, the simplest enharmonic diatonic transposition is automatically selected.

The **TTransposeVisitor** (p. 237) can operate in 2 modes:

- a standard transposing mode: where all the parts are transposed according to the specified interval
- a transposing instrument mode: where the transposition is applied to a normalized score, i.e. a score as written for a non-transposing instrument. It means that a 3 interval applied to a score already transposed with a 2 interval will only apply an additional 1 interval.

Warning:

this visitor is intended to operate on partwise scores only

Todo

transposing ornaments elements

9.161.2 Member Function Documentation

9.161.2.1 static int getOctave (Chromatic steps) [inline, static]

computes an octaves count from a chromatic interval

Parameters:

steps a chromatic steps count

Returns:

the count of octaves

9.161.2.2 static Chromatic getOctaveStep (Chromatic *steps*) [inline, static]

computes a step number within an octave

Parameters:

steps a chromatic steps count

Returns:

the step number within an octave

9.161.2.3 int getKey (Chromatic *steps*) [static]

computes a key sign corresponding to a chromatic degree

Parameters:

steps a chromatic step

Returns:

a numeric key signature representing a number of sharps when positive and a number of flats when negative

9.161.2.4 Diatonic getDiatonics (Chromatic *steps*, int *key*) [static]

computes a diatonic interval corresponding to a chromatic interval and a key signature

Parameters:

steps a chromatic steps count

key a key signature

Returns:

the count of corresponding diatonic steps

9.161.2.5 int getAccidental (Diatonic *pitch*, int *key*) [static]

gives a note accidental according to a key signature

Parameters:

pitch a diatonic pitch (where C=1)

key the key signature expressed as a number of sharps when > 0 or flats when < 0

Returns:

an accidental as a positive value for sharps and negative for flats

9.161.2.6 int incPitch (Diatonic & *pitch*, int & *octave*) [static]

increment a diatonic pitch and returns the corresponding number of chromatic steps

Parameters:

pitch a diatonic pitch value (where C=1)

octave the octave number on input and the possible new octave number on output

Returns:

the corresponding chromatic steps count

9.161.2.7 int decPitch (Diatonic & *pitch*, int & *octave*) [static]

decrement a diatonic pitch and returns the corresponding number of chromatic steps

Parameters:

pitch a diatonic pitch value (where C=1)

octave the octave number on input and the possible new octave number on output

Returns:

the corresponding chromatic steps count

9.161.2.8 int transposePitch (Diatonic & *pitch*, float & *alter*, int & *octave*, Chromatic steps)
[static]

transpose a pitch expressed as a diatonic value + alteration + octave

Parameters:

pitch on input a diatonic pitch value (where C=1), on output the new pitch value

alter on input the accidental value, on output the new accidental value

octave on input the octave number, on output the new octave number

steps the transposing interval expressed in chromatic steps

Returns:

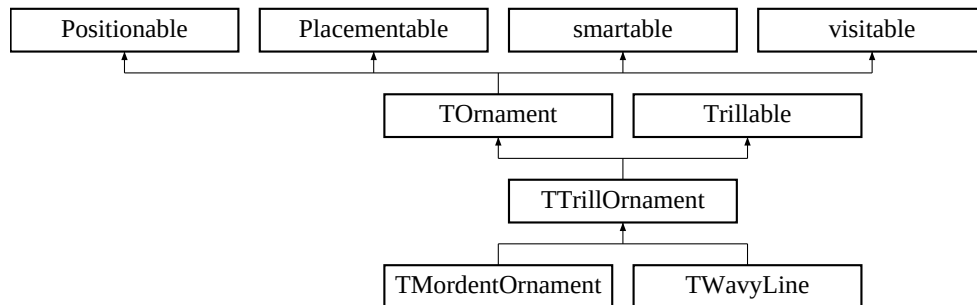
the corresponding diatonic steps count

9.162 TTrillOrnament Class Reference

ornaments that carry a trill-sound entity

```
#include <TOrnaments.h>
```

Inheritance diagram for TTrillOrnament::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Protected Member Functions

- **TTrillOrnament** (type t)

Friends

- EXP friend SMARTP< TTrillOrnament > newTrillOrnament (type t)

9.162.1 Detailed Description

Trill-mark, turn, delayed-turn, shake, mordent, inverted-mordent and wavy-line are ornaments that carry the trill-sound entity.

9.163 TTrillSound Class Reference

Represents the trill-sound entity.

```
#include <common.h>
```

Public Types

- enum { **undefined**, **upright**, **inverted**, **last** = inverted }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setStartNote** (TrillStart::type start)
- void **setTrillStep** (TrillStep::type step)
- void **setTwoNoteTurn** (TrillStep::type tnt)
- void **setAccelerate** (YesNo::type yn)
- void **setBeat** (int beat)
- void **setSecondBeat** (int beat)
- void **setLastBeat** (int beat)
- TrillStart::type **getStartNote** () const
- TrillStep::type **getTrillStep** () const
- TrillStep::type **getTwoNoteTurn** () const
- YesNo::type **getAccelerate** () const
- int **getBeat** () const
- int **getSecondBeat** () const
- int **getLastBeat** () const

9.163.1 Detailed Description

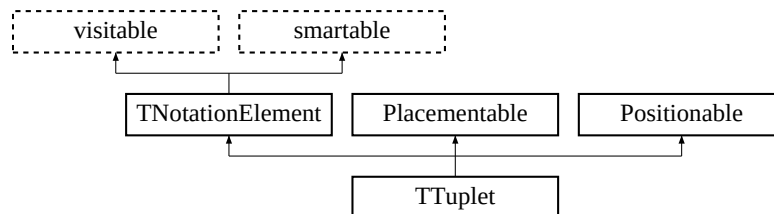
The MusicXML *trill-sound* entity is defined in common.dtd

9.164 TTuplet Class Reference

Graphical representation of a tuplet.

```
#include <TNotation.h>
```

Inheritance diagram for TTuplet::



Public Types

- enum {
undefined = -1, **actual** = 1, **both**, **none**,
last = none }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type type)
the required type attribute
- void **setNumber** (int num)
optional number for nested tuplets
- void **setBracket** (YesNo::type bracket)
optional bracket attribute
- void **setShowNum** (int sn)
optional display control
- void **setShowType** (int st)
optional display control
- StartStop::type **getType** () const
- int **getNumber** () const
- YesNo::type **getBracket** () const
- int **getShowNum** () const
- int **getShowType** () const
- STupletDesc & **actualDesc** ()
- STupletDesc & **normalDesc** ()

Static Public Member Functions

- static const string **xmlshow** (int d)
convert a numeric show type to a MusicXML string
- static int **xmlshow** (const string str)
convert an MusicXML string to a numeric show type

Protected Member Functions

- **TTuplet** (StartStop::type type)

Friends

- EXP friend **SMARTP< TTuplet > newTuplet** (StartStop::type type=StartStop::undefined)

9.164.1 Detailed Description

A tuplet element is present when a tuplet is to be displayed graphically, in addition to the sound data provided by the time-modification elements. The number attribute is used to distinguish nested tuplets. The bracket attribute is used to indicate the presence of a bracket. If unspecified, the results are implementation-dependent.

The tuplet-actual and tuplet-normal elements provide optional full control over tuplet specifications. Each allows the number and note type (including dots) describing a single tuplet. If any of these elements are absent, their values are based on the time-modification element.

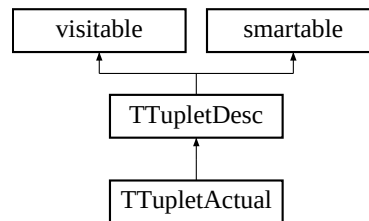
The show-number attribute is used to display either the number of actual notes, the number of both actual and normal notes, or neither. It is actual by default. The show-type attribute is used to display either the actual type, both the actual and normal types, or neither. It is none by default.

9.165 TTupletActual Class Reference

an actual tuplet description.

```
#include <TNotation.h>
```

Inheritance diagram for TTupletActual::



Friends

- EXP friend STupletDesc newTupletActual ()

9.165.1 Detailed Description

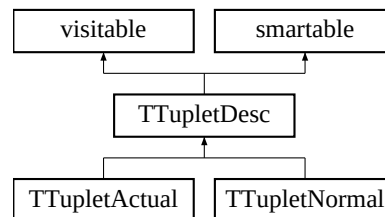
It correspond to the *tuplet-actual* MusicXML elements as defined in note.dtd.

9.166 TTupletDesc Class Reference

a tuplet description.

```
#include <TNotation.h>
```

Inheritance diagram for TTupletDesc::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setNumber** (int num)
- void **setType** (int type)
- void **setDots** (int dots)
- int **getNumber** () const
- int **getType** () const
- int **getDots** () const
- const string **getName** () const

Protected Member Functions

- **TTupletDesc** (string suffix)

9.166.1 Detailed Description

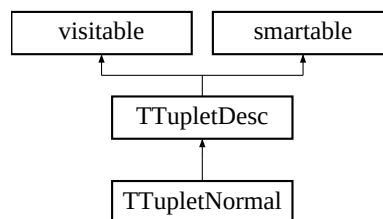
It correspond to the *tuplet-actual* and *tuplet-normal* MusicXML elements as defined in note.dtd.

9.167 TTupletNormal Class Reference

a normal tuplet description.

```
#include <TNotation.h>
```

Inheritance diagram for TTupletNormal::



Friends

- EXP friend STupletDesc newTupletNormal ()

9.167.1 Detailed Description

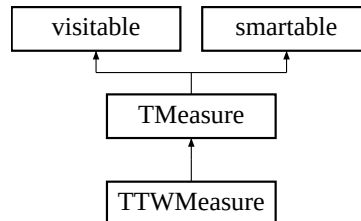
It correspond to the *tuplet-normal* MusicXML elements as defined in note.dtd.

9.168 TTWMeasure Class Reference

The MusicXML timewise *measure* element.

```
#include <TScoreTimewise.h>
```

Inheritance diagram for TTWMeasure::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SMARTP< TTWPart > &part)
- vvector< SMARTP< TTWPart > > & **parts** ()

Protected Member Functions

- **TTWMeasure** (string num)
- **TTWMeasure** (long num)

Friends

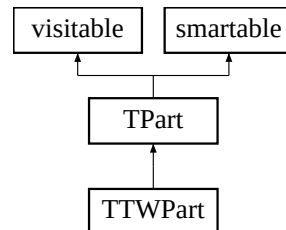
- EXP friend SMARTP< TTWMeasure > **newTWMeasure** (string num)
- EXP friend SMARTP< TTWMeasure > **newTWMeasure** (long num)

9.169 TTWPart Class Reference

The MusicXML timewise *part* element.

```
#include <TScoreTimewise.h>
```

Inheritance diagram for TTWPart::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- long **add** (const SMusicData &data)
- vvector< SMusicData > & **data** ()

Protected Member Functions

- **TTWPart** (string id)

Friends

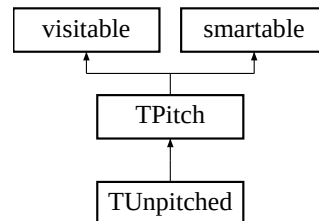
- EXP friend SMARTP< TTWPart > **newTWPart** (string id)

9.170 TUnpitched Class Reference

The MusicXML representation of an unpitched note.

```
#include <TPitch.h>
```

Inheritance diagram for TUnpitched::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

- EXP friend SMARTP< TUnpitched > newUnpitched ()

9.170.1 Detailed Description

The unpitched element indicates musical elements that are notated on the staff but lack definite pitch, such as unpitched percussion and speaking voice. Like notes, it uses step and octave elements to indicate placement on the staff, following the current clef. If percussion clef is used, the display-step and display-octave are interpreted as if in treble clef, with a G in octave 4 on line 2. If not present, the note is placed on the middle line of the staff, generally used for one-line staves.

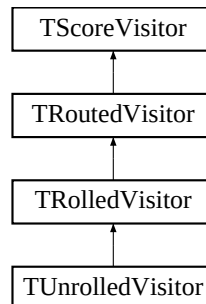
The MusicXML *unpitched* element is defined in note.dtd.

9.171 TUnrolledVisitor Class Reference

A visitor that "unroll" the score.

```
#include <TUnrolledVisitor.h>
```

Inheritance diagram for TUnrolledVisitor::



Public Member Functions

- **TUnrolledVisitor** (**TNodeVisitor** *visitor)
- virtual void **visite** (**SPWMeasure** &elt)
- virtual void **visite** (**SPWPart** &elt)
- virtual void **visite** (**SSound** &elt)
- const list< TSection > & **GetSectionList** ()

9.171.1 Detailed Description

To "unroll" the score, structural information denoted by **repeat**, **ending**, **coda** and **segno** signs is interpreted and the score is visited similarly to a musician that would play the score ie: for example a section repeated twice is visited twice.

Todo

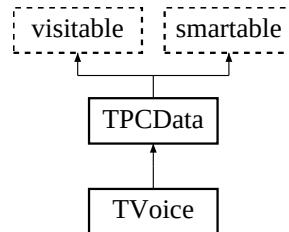
Management of multiple jump and sound **ForwardRepeat** attribute.

9.172 TVoice Class Reference

the voice elements.

```
#include <common.h>
```

Inheritance diagram for TVoice::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)

Friends

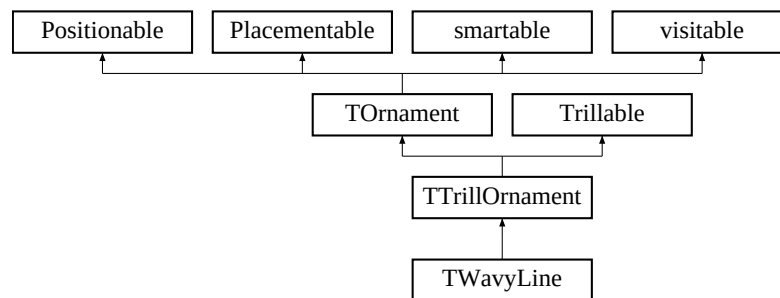
- EXP friend SMARTP< TVoice > newVoice ()

9.173 TWavyLine Class Reference

Represents a MusicXML *wavy-line* element.

```
#include <TOrnaments.h>
```

Inheritance diagram for TWavyLine::



Public Types

- enum { **undefined** = -1 }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (StartStop::type t)
- StartStop::type **getType** () const
- void **setNumber** (int num)
- int **getNumber** () const

Protected Member Functions

- TWavyLine (StartStop::type type)

Friends

- EXP friend SMARTP< TWavyLine > **newWavyLine** (StartStop::type t=StartStop::undefined)

9.173.1 Detailed Description

wavy-line elements can be applied both to notes and to measures. Wavy lines are one way to indicate trills; when used with a measure element, they should always have type="continue" set.

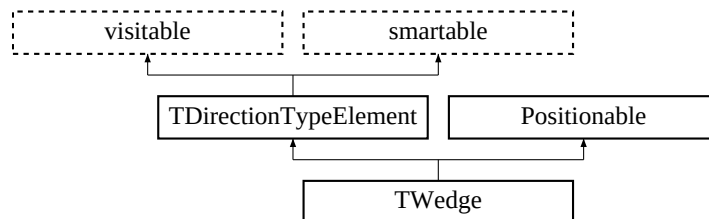
The MusicXML *wavy-line* element is defined in common.dtd

9.174 TWedge Class Reference

the MusicXML *wedge* element.

```
#include <TDirection.h>
```

Inheritance diagram for TWedge::



Public Types

- enum {
 undefined = -1, **crescendo** = 1, **diminuendo**, **stop**,
 last = stop }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setType** (int type)
- void **setNumber** (int num)
- void **setSpread** (int sp)
- int **getType** () const
- int **getNumber** () const
- int **getSpread** () const

Static Public Member Functions

- static const string **xmltype** (int d)
 convert an numeric wedge type to a MusicXML string
- static int **xmltype** (const string str)
 convert an MusicXML string to a numeric wedge type

Protected Member Functions

- TWedge** (int type)

Friends

- EXP friend SMARTP< TWedge > **newWedge** (int type=TOctaveShift::undefined)

9.174.1 Detailed Description

Wedge spread is measured in tenths of staff line space. The type is crescendo for the start of a wedge that is closed at the left side, and diminuendo for the start of a wedge that is closed on the right side. Spread values at the start of a crescendo wedge or end of a diminuendo wedge are ignored.

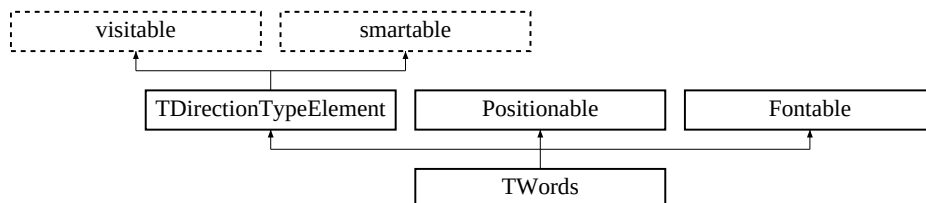
The MusicXML *wedge* element is defined in *direction.dtd*.

9.175 TWords Class Reference

The MusicXml *words* element.

```
#include <TDirection.h>
```

Inheritance diagram for TWords::



Public Types

- enum {
 undefined = -1, **left** = 1, **center**, **right**,
 last = right }

Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setJustify** (int just)
- void **setLang** (const string &lang)
- void **setValue** (const string &value)
- int **getJustify** () const
- const string & **getLang** () const
- const string & **getValue** () const

Static Public Member Functions

- static const string **xmljustify** (int d)
 convert an numeric justification to a MusicXML string
- static int **xmljustify** (const string str)
 convert an MusicXML string to a numeric justification

Friends

- EXP friend SMARTP< TWords > **newWords** ()

9.175.1 Detailed Description

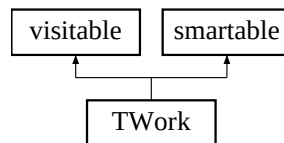
Left justification is assumed if not specified. Language is Italian ("it") by default. Defined in direction.dtd.

9.176 TWork Class Reference

Represents works and movements.

```
#include <TWork.h>
```

Inheritance diagram for TWork::



Public Member Functions

- virtual void **accept** (TScoreVisitor &visitor)
- void **setTitle** (const std::string &title)
the optionnal work title
- void **setNum** (const std::string &num)
the optionnal work number
- const std::string & **getTitle** () const
- const std::string & **getNum** () const

Friends

- EXP friend SMARTP< TWork > newWork ()

9.176.1 Detailed Description

Works and movements are optionally identified by number and title as in MuseData. The work element also may indicate a link to the opus document that composes multiple movements into a collection.

Todo

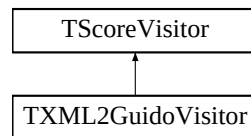
opus support is to be implemented

9.177 TXML2GuidoVisitor Class Reference

A score visitor to produce a generic Guido representation.

```
#include <TXML2GuidoVisitor.h>
```

Inheritance diagram for TXML2GuidoVisitor::



Public Member Functions

- **TXML2GuidoVisitor** (bool comments=true, bool checkStem=true, bool bar=false)
- void **visite** (SAccidental &elt)
- void **visite** (SArticulationElement &elt)
- void **visite** (SArticulations &elt)
- void **visite** (SAttributes &elt)
- void **visite** (SBackup &elt)
- void **visite** (SBarline &elt)
- void **visite** (SBeam &elt)
- void **visite** (SBracket &elt)
- void **visite** (SChord &elt)
- void **visite** (SClef &elt)
- void **visite** (SCoda &elt)
- void **visite** (SCreator &elt)
- void **visite** (SDirection &elt)
- void **visite** (SDirectionType &elt)
- void **visite** (SDirective &elt)
- void **visite** (SDynamic &elt)
- void **visite** (SEncoding &elt)
- void **visite** (SEnding &elt)
- void **visite** (SFermata &elt)
- void **visite** (SForward &elt)
- void **visite** (SGraceNote &elt)
- void **visite** (SGraphNode &elt)
- void **visite** (SIdentification &elt)
- void **visite** (SKey &elt)
- void **visite** (SMetronome &elt)
- void **visite** (SMidiInstrument &elt)
- void **visite** (SNotation &elt)
- void **visite** (SNote &elt)
- void **visite** (SNoteHead &elt)
- void **visite** (SOctaveShift &elt)
- void **visite** (SPWMeasure &elt)
- void **visite** (SPWPart &elt)
- void **visite** (SPartGroup &elt)

- void **visite** (SPartList &elt)
- void **visite** (SPitch &elt)
- void **visite** (SRepeat &elt)
- void **visite** (SRest &elt)
- void **visite** (SScoreInstrument &elt)
- void **visite** (SScorePart &elt)
- void **visite** (SScorePartwise &elt)
- void **visite** (SSegno &elt)
- void **visite** (SSLur &elt)
- void **visite** (SSound &elt)
- void **visite** (SStrongAccent &elt)
- void **visite** (STie &elt)
- void **visite** (STimeModification &elt)
- void **visite** (STimeSign &elt)
- void **visite** (STranspose &elt)
- void **visite** (STuplet &elt)
- void **visite** (STupletDesc &elt)
- void **visite** (SUnpitched &elt)
- void **visite** (SWavyLine &elt)
- void **visite** (SWedge &elt)
- void **visite** (SWords &elt)
- **Sguidoelement** & **current** ()
- void **generatePositions** (bool state)

Protected Member Functions

- void **add** (Sguidoelement &elt)
- void **push** (Sguidoelement &elt)
- void **pop** ()

Protected Attributes

- long **fCurrentVoice**
- long **fCurrentDivision**
- long **fCurrentOffset**
- bool **fMeasureEmpty**

Classes

- struct **delayedElement**
- class **SlursVisitor**

9.177.1 Constructor & Destructor Documentation

9.177.1.1 TXML2GuidoVisitor (bool *comments* = true, bool *stem* = true, bool *bar* = false)

Todo

support of IDs for tags other than slurs

9.177.2 Member Function Documentation

9.177.2.1 void visite (SBarline & *elt*) [virtual]

visit of a barline may add several guido elements: `\bar`, `\fermata`, `\repeatBegin` `\repeatEnd` ...

Reimplemented from **TScoreVisitor** (p. 212).

9.177.2.2 void visite (SClef & *elt*) [virtual]

Todo

translation of none and TAB clefs

Reimplemented from **TScoreVisitor** (p. 212).

9.177.2.3 void visite (SCreator & *elt*) [virtual]

Todo

handling the different creator types as defined in MusicXML. There is actually no corresponding tag apart the `composer` tag in guido.

Reimplemented from **TScoreVisitor** (p. 212).

9.177.2.4 void visite (SMetronome & *elt*) [virtual]

Todo

complete translation of TMetronome

Reimplemented from **TScoreVisitor** (p. 212).

9.177.2.5 void visite (SScoreInstrument & *elt*) [virtual]

Todo

support of the MIDI prog change

Reimplemented from **TScoreVisitor** (p. 212).

9.177.2.6 void visite (STimeSign & *elt*) [virtual]

Todo

check the way unmeasured time is handled
check the symbol translation and handle the single number symbol

Reimplemented from **TScoreVisitor** (p. 212).

interface for visitable objects.

Inheritance diagram for visitable::



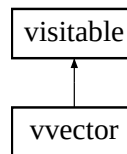
- virtual void **accept** (TScoreVisitor &visitor)=0

9.179 vvector Class Template Reference

A visitable vector.

```
#include <TScoreVisitor.h>
```

Inheritance diagram for vvector::



Public Member Functions

- void **accept** (TScoreVisitor &visitor)

9.179.1 Detailed Description

```
template<typename T> class MusicXML::vvector< T >
```

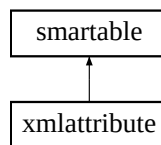
relay the accept method to all the vector elements.

9.180 xmlattribute Class Reference

A generic xml attribute representation.

```
#include <xml.h>
```

Inheritance diagram for xmlattribute::



Public Member Functions

- void **setName** (const string &name)
- void **setValue** (const string &value)
- void **setValue** (long value)
- void **setValue** (float value)
- const string & **getName** () const
- const string & **getValue** () const
- void **print** (std::ostream &os) const

Protected Member Functions

- **xmlattribute** (string name, string value)
- **xmlattribute** (string name, long value)
- **xmlattribute** (string name, float value)

Friends

- EXP friend SMARTP< **xmlattribute** > **new_xmlattribute** (string name, string value)
- EXP friend SMARTP< **xmlattribute** > **new_xmlattribute** (string name, long value)
- EXP friend SMARTP< **xmlattribute** > **new_xmlattribute** (string name, float value)
- EXP friend SMARTP< **xmlattribute** > **new_xmlattribute** (string name, int value)

9.180.1 Detailed Description

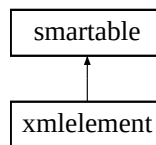
An attribute is represented by its name and its value.

9.181 xmlelement Class Reference

A generic xml element representation.

```
#include <xml.h>
```

Inheritance diagram for xmlelement::



Public Member Functions

- void **setValue** (unsigned long value)
- void **setValue** (long value)
- void **setValue** (int value)
- void **setValue** (float value)
- void **setValue** (const string &value)
- void **setName** (const string &name)
- const string & **getName** () const
- const string & **getValue** () const
- long **add** (const **Sxmlattribute** &attr)
- long **add** (const **Sxmlelement** &elt)
- const vector< **Sxmlelement** > & **elements** () const
- const vector< **Sxmlattribute** > & **attributes** () const
- bool **empty** () const
- void **print** (std::ostream &os) const

Protected Member Functions

- **xmlelement** (string name)
- **xmlelement** (string name, unsigned long value)
- **xmlelement** (string name, long value)
- **xmlelement** (string name, int value)
- **xmlelement** (string name, float value)
- **xmlelement** (string name, string value)

Friends

- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name)
- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name, unsigned long val)
- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name, long val)
- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name, int val)
- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name, float val)
- EXP friend **SMARTP**< **xmlelement** > **new_xmlelement** (string name, string val)

9.181.1 Detailed Description

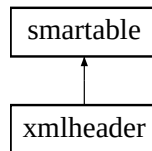
An element is represented by its name, its value, the lists of its attributes and its enclosed elements. Attributes and elements must be added in the required order.

9.182 xmlheader Class Reference

a class for generating the MusicXML header.

```
#include <xml.h>
```

Inheritance diagram for xmlheader::



Public Types

- enum { **partwise** = 1, **timewise**, **last** = timewise }

Public Member Functions

- void **setMode** (int mode)
sets the MusicXML score mode
- void **setLocation** (const string &loc)
sets the MusicXML dtd location
- int **getMode** () const
returns the MusicXML score mode
- const string & **getLocation** () const
returns the MusicXML dtd location
- const string **MusicXMLVersion** () const
returns the current supported MusicXML version
- void **print** (std::ostream &os) const

Static Public Member Functions

- static const string **scoreMode** (int mode)
provides conversion from numeric to string modes
- static int **scoreMode** (string mode)
provides conversion from string to numeric modes

Protected Member Functions

- **xmlheader** (int mode, string loc)

Friends

- EXP friend **SMARTP**< **xmlheader** > **new_xmlheader** (int mode=xmlheader::partwise, string loc="http://www.musicxml.org/dtds/")

9.183 YesNo Class Reference

provides conversions between numeric yes-no types and strings

```
#include <conversions.h>
```

Public Types

- enum **type** { **undefined**, **yes**, **no**, **last** = no }

Static Public Member Functions

- static const string **xml** (type d)
convert a numeric yes-no value to a MusicXML string
- static type **xml** (const string str)
convert a MusicXML string to a numeric yes-no value

Chapter 10

LibMusicXML Page Documentation

10.1 The MusicXML format

The MusicXML is a xml format that organizes the music into a header followed by the core music data.

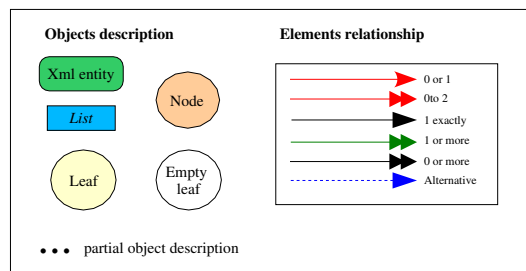


Figure 10.1: MusicXML elements relationship

The core music data may be viewed as *partwise* or *timewise* data:

- *partwise* data are organized into parts containing measures
- *timewise* data are organized into measures containing parts.

10.1.1 MusicXML music data

A MusicXML `part` element may be viewed as a music part, assigned to a given instrument. Therefore a part generally contains a single staff but in case of instruments like the piano or the organ, it includes several staves.

A `measure` contains elements grouped under the `music-data` entity. These elements cover the following purposes:

- music score description. Most of the elements are intended to enumerate the graphic components of a music score. The `note` element is the most important one but the measure contains also measure specific attributes like `clef`, `key` or `time` signatures, `transpose` indications or `barline` description, as well as `direction` elements attached to a part or the overall score.

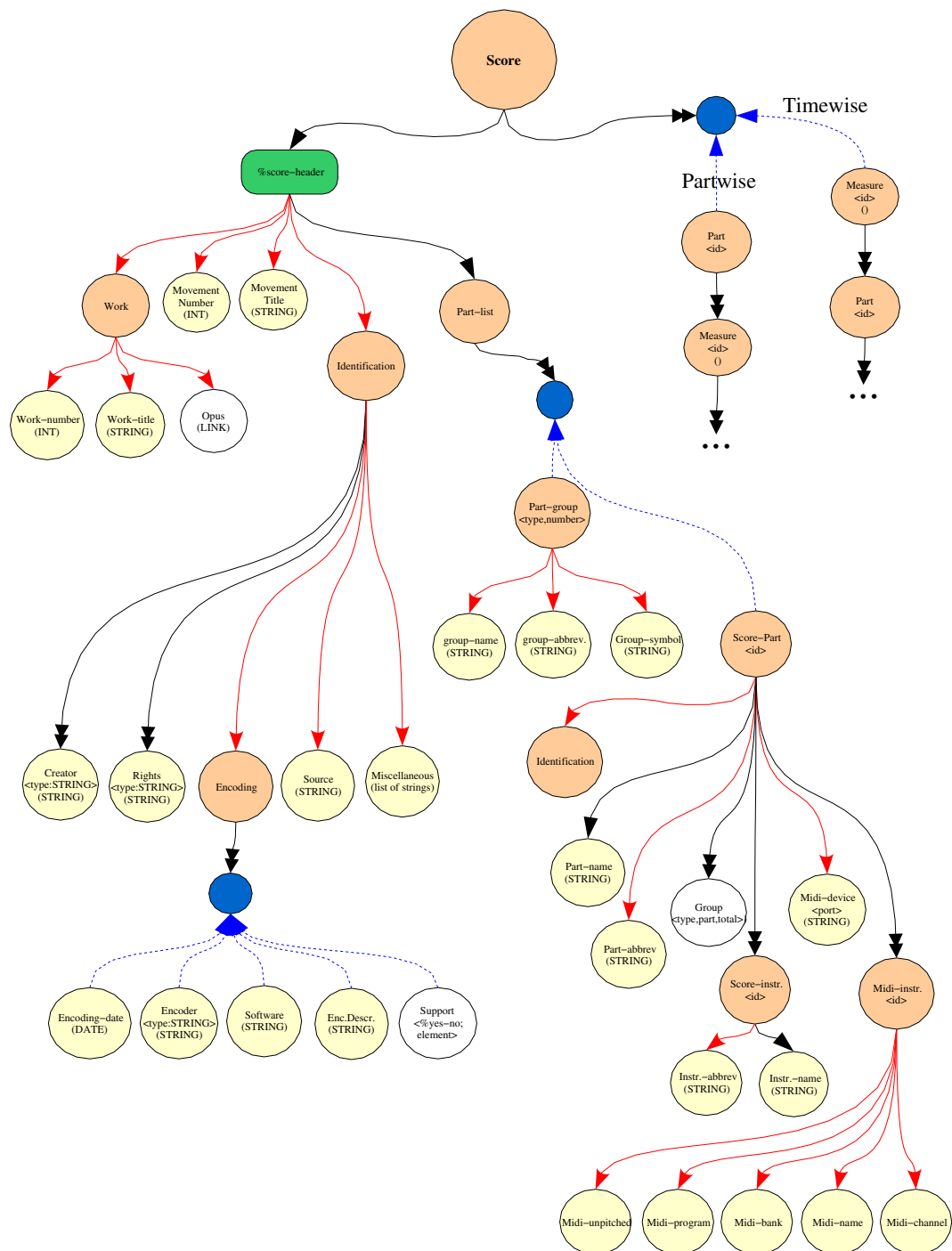


Figure 10.2: The MusicXML Score format

- time description using elements to move the time backward (backup element) or forward (forward element).

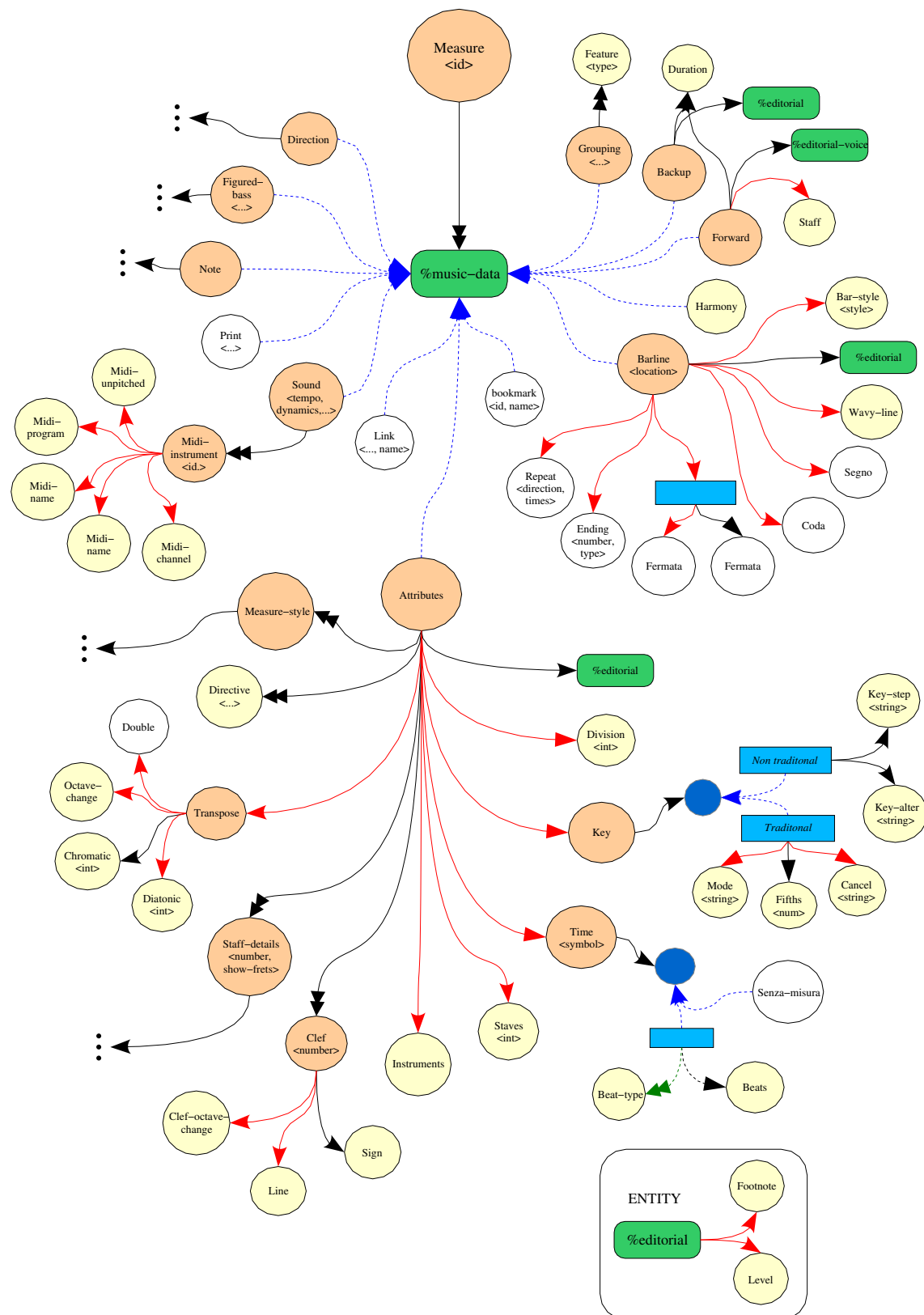
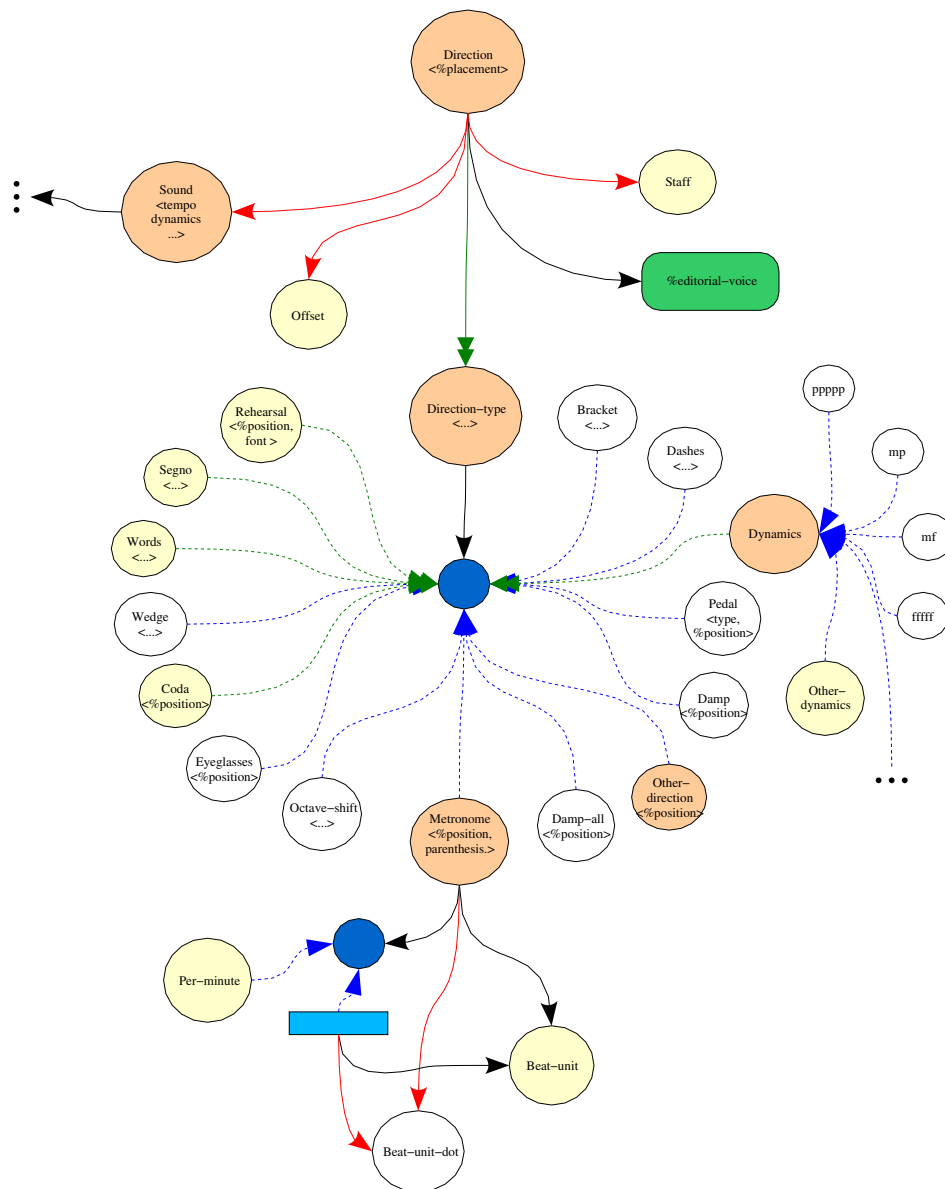


Figure 10.3: The MusicXML `measure` element

- music analysis using elements like `harmony` or `grouping`.
- playback parameters: the `sound` element allows for tempo, dynamics description, but also for sound control including MIDI instrument assignment, and for structural description (`dacapo`, `segno`, `dalsegno`, `coda`, `tocoda`).
- XLink support: `link` and `bookmark` elements,
- printing parameters: `print` element.

Figure 10.4: The MusicXML `direction` element

The `direction` element is used for musical indications that are not attached to a specific note. Two or more may be combined to indicate starts and stops of wedges, dashes, etc. It supports also dynamics, metronome indications and may include `sound` elements.

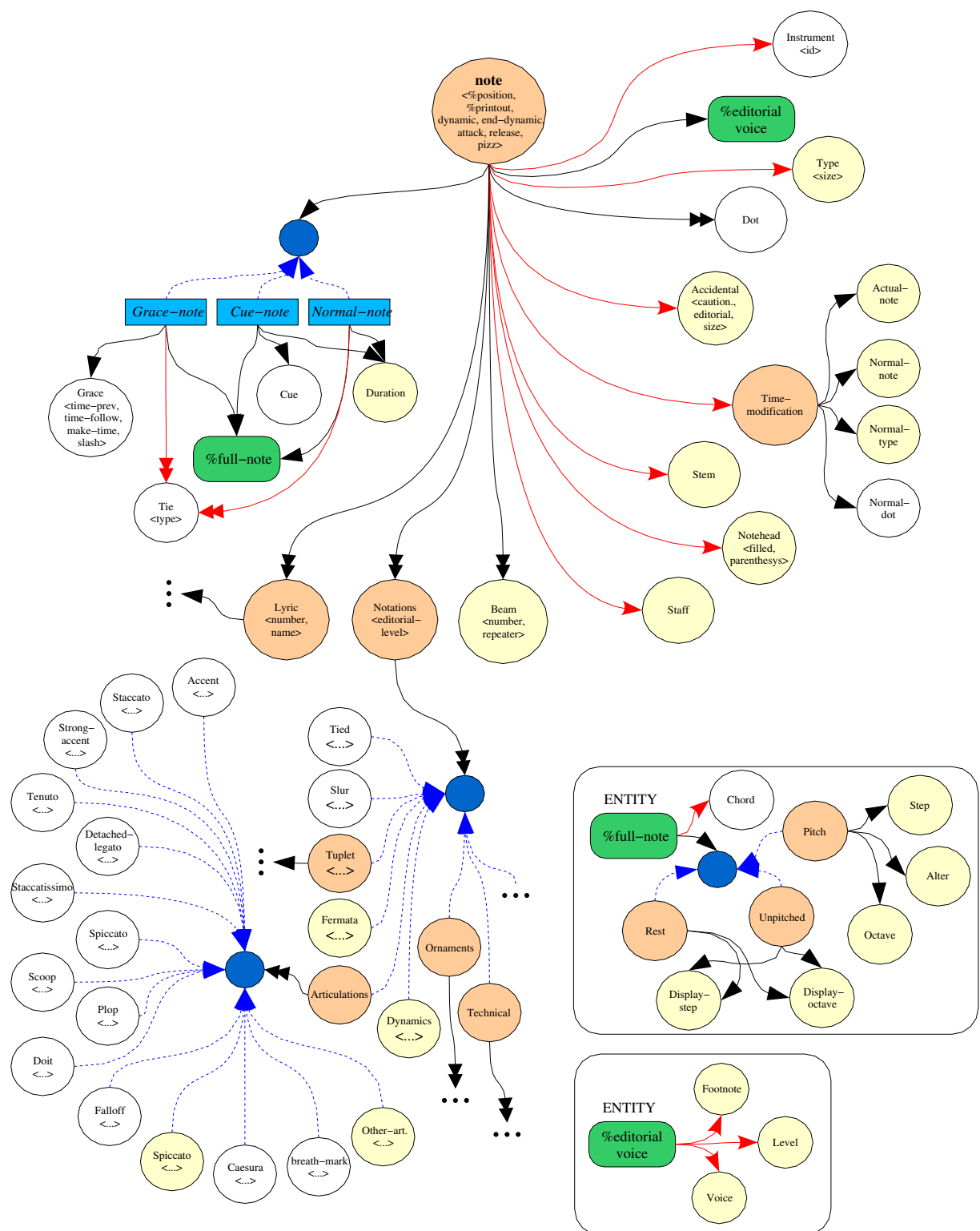


Figure 10.5: The MusicXML note element

The `note` element is central to the music description. A note may be a *cue* note, a *grace* note or a *regular*

note. All of them share common elements which are pitch, chord, and rest information. Unpitched elements are used for unpitched percussion, speaking voice, and other musical elements lacking determinate pitch. A `note` element includes all the necessary for an accurate graphic rendering of all the signs attached to it. It covers:

- the graphic `type` corresponding to the symbolic note duration: whole, half, quarter note...
- possible `accidental` and `dot`,
- `stem` and `beaming` information,
- the graphic `notehead` shape: triangle, diamond, square...
- `staff` assignment
- `notation` elements attached to the note such as `articulation`, `ornament`, `slur`, `tied` ...
- `lyrics`

Note that redundant graphic and sound information may live together, for example:

- a `tuplet` element is present when a tuplet is to be displayed graphically, in addition to the sound data provided by the `time-modification` element.
- a `tie` element indicates sound; the `tied` element indicates notation.
- a `duration` element indicates the note duration in division units. It is the sounding duration vs. notated duration.

A note has also attributes intended for a more accurate graphic placement or to describe differences in duration or dynamics specific to an interpretation or performance.

10.1.2 MusicXML time

MusicXML doesn't include explicit time information in that sense that music events like notes don't carry a date. The time is implicitly moved each time an element that has a duration is encountered. Exception to this rule is the `chord` element: a `chord` element indicates that the note carrying the `chord` element is an additional chord tone with the preceding note.

The duration of a note marked with a `chord` element can be no longer than the preceding note. Although a note that is part of a chord has a duration, the time doesn't move when it is encountered.

Additional elements are provided to explicitly move the time: `backup` and `forward` are intended to coordinate multiple voices in one part, including music on multiple staves. `Forward` is generally used to move forward within voices and staves, while `backup` to move backward between voices and staves.

10.2 Sample code

This section provides examples of the MusicXML library use.

10.2.1 Reading and writing MusicXML files

```
/*

Copyright (C) 2003 Grame
Grame Research Laboratory, 9 rue du Garet, 69001 Lyon - France
research@grame.fr

This file is provided as an example of the MusicXML Library use.
It shows how to read and write xml files.

*/

#include <iostream>
#include <string>

#include "TMusicXMLFile.h"

using namespace std;
using namespace MusicXML;

int main (int argc, char * argv[])
{
    // use of a TMusicXMLFile object to read and write xml files
    TMusicXMLFile file;
    for (int i=1; i<argc; i++) {
        // reads the file (file name is readed from the command line)
        SScore score = file.read(argv[i]);
        if (score) {
            // and writes the readed score to the standard output
            // note that 'cout' could be replaced by a file name as well
            file.write (score, cout);
        }
        else {
            cerr << "cannot read file \"" << argv[i] << "\"" << endl;
        }
    }
    return 0;
}
```

10.2.2 Building a score from scratch

```
/*

Copyright (C) 2003 Grame
Grame Research Laboratory, 9 rue du Garet, 69001 Lyon - France
research@grame.fr

This file is provided as an example of the MusicXML Library use.
It shows how to build a memory representation of a score from scratch.

*/

#include <iostream>
#include <sstream>
#include <string>
```

```

#include <cstdlib>
#include <ctime>
#include <math.h>

#ifdef WIN32
#include <windows.h>
#endif

#include "LibMusicXML.h"

using namespace MusicXML;

const string kPartID = "P1";
#define kDivision 4

// a simple function that return random numbers in the given range
int getrandom(int range);

// the 'scorepart' function builds a score-part MusicXML element
SScorePart scorepart();

// the 'makeheader' function builds the score header
void makeheader(SScoreHeader& header);

// creates a measure containing random notes
SPWMeasure makemeasure(unsigned long num);

// creates specific elements of the score first measure
void firstmeasure(SPWMeasure& m);

// creates a part containing 'count' measures
SPWPart makepart(int count);

// the function that creates and writes the score
void randomMusic(int measuresCount);

//-----
// a simple function that return random numbers in the given range
//-----
int getrandom(int range) {
    float f = (float)rand() / RAND_MAX;
    return (int)(f * range);
}

//-----
// the 'scorepart' function builds a score-part MusicXML element
//-----
SScorePart scorepart() {
    SScorePart part = newScorePart(kPartID); // creates the score-part element
    part->setPartName ("Recorder");           // sets the part name

    // creates a score instrument
    SScoreInstrument instrument = newScoreInstrument ("I1");
    instrument->setName ("Recorder");          // sets the instrument name
    part->add(instrument);                     // and adds the instrument to the part

    // creates a MIDI instrument
    SMidiInstrument midi = newMidiInstrument("I1");
    midi->setProgram (75); // sets the MIDI program to the General MIDI program for the recorder
    part->add(midi);       // and adds the MIDI instrument to the part

    return part;
}

//-----

```

```

// the 'makeheader' function builds the score header
// the score header contains various information, including identification,
// parts list etc...
//-----
void makeheader(SScoreHeader& header) {
    // creates an identification element
    SIdentification id = newIdentification();

    SCreator creator = newCreator ();           // creates a creator element
    creator->setType("Composer");                // sets the creator type
    creator->setName("Georg Chance");            // sets the creator name
    id->add(creator);                            // adds the creator to the identification

    SEncoding ec = newEncoding();               // creates an encoding element
    ec->setSoftware ("MusicXML Library");        // sets the software name
    id->encoding() = ec;                        // adds the encoding to the identification

    header->identification() = id;              // adds the identification to the header

    header->setMovementTitle ("Random Music");   // sets the movement title
    header->partList() = newPartList();         // creates a part list
    header->partList()->add (scorepart());      // and adds a new part to the part list
}

//-----
// creates specific elements of the score first measure
// the first measure is generally a special measure since it contains
// elements like the clef, the time and key signatures etc...
//-----
void firstmeasure(SPWMeasure& m) {
    SAttributes attributes = newAttributes();    // creates new attributes

    SClef clef = newClef();                     // creates a new clef
    clef->setSign(TClef::G);                    // sets the clef sign
    clef->setLine(2);                           // sets the clef line number
    attributes->add(clef);                      // and adds the clef to the attributes

    STimeSign ts = newTimeSign();               // creates a new time signature
    ts->add (4, 4);                             // sets the time signature to 4/4
    attributes->timeSign() = ts;                // and drop the time sign into the attributes

    // sets the attributes division: division is the MusicXML way to indicates how many
    // divisions per quarter note are used to indicate a note's duration.
    attributes->setDivisions(kDivision);

    m->add (attributes);                        // and adds the attributes to the measure

    SDirection dir = newDirection();            // creates a direction element
    SSound sound = newSound();                  // creates a sound element
    long tempo = 120 + getrandom(40)-20;        // computes a random tempo value
    sound->setTempo (tempo);                    // and sets the sound tempo
    dir->sound() = sound;                      // and drop the sound element into the direction element

    SDirectionType dt = newDirectionType();    // creates a direction-type element
    SMetronome metro = newMetronome();         // creates a metronome element
    metro->setBeat (NoteType::quarter);         // and sets the metronome to
    metro->setPerMinute (tempo);                // quarter note = tempo
    dt->add(metro);                            // adds the metronome to the direction-type element
    dir->add(dt);                              // adds the direction-type to the direction element
    m->add (dir);                              // and finally adds the direction to the measure
}

//-----
// creates a measure containing random notes
// the function takes the measure number as an argument

```

```
//-----
SPWMeasure makemeasure(unsigned long num) {
    // creates a new partwise measure
    SPWMeasure measure = newPWMeasure(num);

    if (num==1) {                // if it's the first measure
        firstmeasure(measure);    // creates specific elements of the first measure
    }
    for (int i=0; i<4; i++) {      // next adds 4 quarter notes
        SNote note = newNote();    // creates the note
        SPitch pitch = newPitch(); // creates a pitch
        pitch->setStep(getrandom(7)+1); // sets the pitch to a random value
        pitch->setOctave (4 + getrandom(2)); // sets the octave to a random value
        note->pitch() = pitch;      // and drops the pitch to the note
        note->setDuration (kDivision); // sets the note duration to a quarter note
        note->graphics() = newGraphNode(); // creates the graphic elements of the note
        note->graphics()->setType (NoteType::quarter); // and sets the graphic note type to quarter note
        measure->add (note);        // and finally adds the note to the measure
    }
    return measure;
}

//-----
// creates a part containing 'count' measures
//-----
SPWPart makepart(int count) {
    SPWPart part = newPWPart(kPartID); // creates a new part
    for (int i=1; i<=count; i++)       // and 'count' times
        part->add (makemeasure(i));    // adds a new measure to the part
    return part;
}

//-----
// the function that creates and writes the score
//-----
void randomMusic(int measuresCount) {
    SScorePartwise score = newScorePartwise(); // creates a partwise score
    score->scoreHeader() = newScoreHeader();   // creates a score header and drops it to the score
    makeheader(score->scoreHeader());          // fills the score header
    score->add(makepart(measuresCount));        // adds a part to the score

    TMusicXMLFile f;
    f.write(score, cout); // and finally writes the score to the standard output
}

int main (int argc, char * argv[]) {
    // if present, reads the measures count from the command line
    int count = (argc == 2) ? atoi(argv[1]) : 20;
    // sets the random numbers seed
    srand((unsigned)time(0));
    // computes and writes the score
    randomMusic(count);
    return 0;
}
```

10.2.3 Visiting a score

/*

Copyright (C) 2003 Grame
 Grame Research Laboratory, 9 rue du Garet, 69001 Lyon - France
 research@grame.fr

This file is provided as an example of the MusicXML Library use.

It shows how to read and write xml files. It also gives a concrete example of how to browse and modify the memory representation of a score.

```

*/

#include <iostream>

#include "LibMusicXML.h"

using namespace std;
using namespace MusicXML;

//-----
//
// This is an example of a visitor implementation. Visitors allows
// to browse the memory representation of a score. Two basic visitors
// are provided by the library:
// - a TRoutedVisitor which is a visitor that provides a path for
//   traversing the score tree (see the library documentation for the
//   path description).
// - a TScoreVisitor which is a visitor that does nothing: objects
//   that derive from TScoreVisitor must implement a path for traversing
//   the score tree. This may be necessary only for non-standard way
//   of browsing the score.
// The RawTransposer visitor has no special requirement concerning the path
// for traversing the tree and therefore derives from TRoutedVisitor.
// It is only interested in TNote objects and therefore, only overrides
// the corresponding method.
//
//-----
class RawTransposer : public TRoutedVisitor {

public:
    // constructor is initialized with the transposing interval
    RawTransposer(int interval) : fInterval(interval) {}
    virtual ~RawTransposer() {}
    void visite ( SNote& elt );

private:
    int fInterval;      // the transposing interval
};

//-----
// The raw transposition implementation.
//-----
void RawTransposer::visite (SNote& note) {
    // first get the note pitch element (it's a MusicXML pitch)
    SPitch pitch = note->pitch();
    // and next adds the transposing interval
    // the += operator affects the step and octave only (no accidental change)
    *pitch += fInterval;
}

//-----
int main (int argc, char * argv[])
{
    if (argc != 3) {
        cerr << "usage: " << argv[0] << " <interval> <file>" << endl;
        exit(1);
    }

    // transposing interval is readed from the command line
    int interval = atoi(argv[1]);
    // use of a TMusicXMLFile object to read and write xml files
    TMusicXMLFile file;

```

```
// reads the file (file name is readed from the command line)
SScore score = file.read(argv[2]);
if (score) {
    if (interval) {
        // declares a RawTransposer object
        RawTransposer transposer(interval);
        // ask the score to accept the transposer
        score->accept(transposer);
    }
    // and finally writes the result on the standard output
    // 'cout' could be replaced by a file as well
    file.write (score, cout);
}
else cerr << "cannot read file \"" << argv[2] << "\"" << endl;
return 0;
}
```

10.3 Todo List

Class `guidonotestatus` (p. 48) handling the current beat value for **num* duration form.

Class `TArticulationElement` (p. 69) support of the line-shape and line-type attributes

Class `TAttributes` (p. 72) clarify the Instrument type and semantic

Class `TEnding` (p. 100) support of multiple numbers

Class `TFrameNote` (p. 112) : use the `TString` and `TFret` classes (**`TTechnical.h`** (p. ??))

Class `TGraphNode` (p. 117) the MusicXML stem definition includes now a position attribute and should be implemented as a separate element

Class `TMeasure` (p. 131) support of the *implicit* and *non-controlling* attributes.

Class `TMidiContextVisitor` (p. 136) Handling of grace notes. Handling of divisions attribute in coda or segno sound attribute. Handling of actual duration in fine sound element.

Member `MusicXML::TRoutedVisitor::visite` (p. 204)(**`SLyric &elt`**) route to the font associated to the text elements

Class `TTransposeVisitor` (p. 237) transposing ornaments elements

Class `TUnrolledVisitor` (p. 251) Management of multiple jump and sound **`ForwardRepeat`** attribute.

Class `TWork` (p. 257) opus support is to be implemented

Member `TXML2GuidoVisitor::TXML2GuidoVisitor` (p. 259)(**`bool comments=true, bool checkStem=true, bool bar=false`**) support of IDs for tags other than slurs

Member `TXML2GuidoVisitor::visite` (p. 260)(**`SClef &elt`**) translation of none and TAB clefs

Member `TXML2GuidoVisitor::visite` (p. 260)(**`SCreator &elt`**) handling the different creator types as defined in MusicXML. There is actually no corresponding tag apart the `composer` tag in guido.

Member `TXML2GuidoVisitor::visite` (p. 260)(**`SMetronome &elt`**) complete translation of `TMetronome`

Member `TXML2GuidoVisitor::visite` (p. 260)(**`SScoreInstrument &elt`**) support of the MIDI prog change

Member `TXML2GuidoVisitor::visite` (p. 260)(**`STimeSign &elt`**) check the way unmeasured time is handled
check the symbol translation and handle the single number symbol

Index

- [/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/s25/](#)
 - Directory Reference, 34
 - [/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/s25/](#)
 - Directory Reference, 29
 - [/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/s25/](#)
 - Directory Reference, 32
 - [/Users/fober/src/libmusicxml.svn/branches/dev/libmusicxml/s25/](#)
 - Directory Reference, 35
- bimap, 38
- decPitch
 - MusicXML::TTransposeVisitor, 240
- equal
 - MusicXML::TMetronome, 135
- getAccidental
 - MusicXML::TTransposeVisitor, 239
- getChan
 - TMidiContextVisitor, 137
- getDate
 - TMidiContextVisitor, 138
- getDiatonics
 - MusicXML::TTransposeVisitor, 239
- getDuration
 - TMidiContextVisitor, 138
- getDynamics
 - TMidiContextVisitor, 138
- getFullDuration
 - TMidiContextVisitor, 137
- getKey
 - MusicXML::TTransposeVisitor, 239
- getOctave
 - MusicXML::TTransposeVisitor, 238
- getOctaveStep
 - MusicXML::TTransposeVisitor, 238
- getPart
 - TMidiContextVisitor, 137, 138
- getPitch
 - TMidiContextVisitor, 137
- getTempo
 - TMidiContextVisitor, 138
- getTPQ
 - TMidiContextVisitor, 138
- global/ Directory Reference, 31
- guido
 - guidochecker, 42
 - guidochord, 43
 - guidoelement, 44
 - guidoknote, 46
 - guidonoteduration, 47
 - guidonotesustains, 48
 - guidoparam, 49
 - guidoseq, 50
 - guidotag, 51
 - guidovisitable, 52
 - guidovisitor, 53
- incPitch
 - MusicXML::TTransposeVisitor, 239
- MusicXML::Bendable, 37
- MusicXML::Editorialable, 39
- MusicXML::Fontable, 40
- MusicXML::FullCue, 41
- MusicXML::LineType, 54
- MusicXML::NoteType, 55
- MusicXML::Orientable, 56
- MusicXML::Placemetable, 58
- MusicXML::Positionable, 59
- MusicXML::StartStop, 64
- MusicXML::TAccidental, 65
- MusicXML::TAccidentalMark, 67
- MusicXML::TArpeggiate, 68
- MusicXML::TArticulationElement, 69
- MusicXML::TArticulations, 71
- MusicXML::TAttributes, 72
- MusicXML::TBackup, 73
- MusicXML::TBarline, 74
- MusicXML::TBass, 76
- MusicXML::TBeam, 77
- MusicXML::TBeatRepeat, 79
- MusicXML::TBend, 80
- MusicXML::TBendSound, 81
- MusicXML::TBendWithBar, 82
- MusicXML::TBezier, 83
- MusicXML::TBracket, 84
- MusicXML::TChord, 86
- MusicXML::TClef, 87
- MusicXML::TCoda, 89

- MusicXML::TCreator, 90
- MusicXML::TDashes, 91
- MusicXML::TDegree, 92
- MusicXML::TDirection, 93
- MusicXML::TDirectionType, 94
- MusicXML::TDirectionTypeElement, 95
- MusicXML::TDirective, 96
- MusicXML::TDynamic, 97
- MusicXML::TEditorial, 98
- MusicXML::TEncoding, 99
- MusicXML::TEnding, 100
- MusicXML::TExpression, 102
- MusicXML::TFeature, 103
- MusicXML::TFermata, 104
- MusicXML::TFigure, 105
- MusicXML::TFiguredBass, 106
- MusicXML::TFingering, 107
- MusicXML::TFont, 108
- MusicXML::TFootnote, 109
- MusicXML::TForward, 110
- MusicXML::TFrame, 111
- MusicXML::TFrameNote, 112
- MusicXML::TFunction, 113
- MusicXML::TGlissando, 114
- MusicXML::TGraceNote, 116
- MusicXML::TGraphNode, 117
- MusicXML::TGrouping, 119
- MusicXML::THammerPull, 120
- MusicXML::THarmonic, 121
- MusicXML::THarmony, 123
- MusicXML::THarmonyChord, 124
- MusicXML::TIdentification, 125
- MusicXML::TKey, 126
- MusicXML::TLevel, 128
- MusicXML::TLyric, 129
- MusicXML::TMeasure, 131
- MusicXML::TMeasureRepeat, 132
- MusicXML::TMeasureStyle, 133
- MusicXML::TMeasureStyleElement, 134
- MusicXML::TMetronome, 135
- MusicXML::TMetronome
 - equal, 135
- MusicXML::TMidiDevice, 139
- MusicXML::TMidiInstrument, 140
- MusicXML::TMidiInstrument
 - setUnpitched, 141
- MusicXML::TMiscField, 142
- MusicXML::TMiscId, 143
- MusicXML::TMordentOrnament, 144
- MusicXML::TMultipleLyricPart, 145
- MusicXML::TMultipleRest, 146
- MusicXML::TMusicData, 147
- MusicXML::TMusicXMLFile, 148
- MusicXML::TNodeVisitor, 149
- MusicXML::TNonArpeggiate, 154
- MusicXML::TNotation, 155
- MusicXML::TNotationElement, 156
- MusicXML::TNote, 157
- MusicXML::TNoteHead, 159
- MusicXML::TOctaveShift, 161
- MusicXML::TOrientation, 163
- MusicXML::TOrnament, 164
- MusicXML::TOrnaments, 166
- MusicXML::TOtherArticulation, 167
- MusicXML::TOtherDirection, 168
- MusicXML::TOtherDirections, 169
- MusicXML::TOtherNotation, 170
- MusicXML::TOtherOrnament, 171
- MusicXML::TPart, 172
- MusicXML::TPartGroup, 173
- MusicXML::TPartList, 175
- MusicXML::TPartListElement, 176
- MusicXML::TPCData, 179
- MusicXML::TPedal, 180
- MusicXML::TPitch, 181
- MusicXML::TPlacement, 183
- MusicXML::TPosition, 184
- MusicXML::TPrint, 185
- MusicXML::TPrintout, 186
- MusicXML::TPWMMeasure, 187
- MusicXML::TPWPart, 188
- MusicXML::TRehearsal, 191
- MusicXML::TRepeat, 192
- MusicXML::TRest, 193
- MusicXML::TRights, 194
- MusicXML::Trillable, 195
- MusicXML::TrillStart, 196
- MusicXML::TrillStep, 197
- MusicXML::TRolledVisitor, 198
- MusicXML::TRoot, 201
- MusicXML::TRootFunction, 202
- MusicXML::TRoutedVisitor, 203
- MusicXML::TRoutedVisitor
 - visite, 204
- MusicXML::TScore, 206
- MusicXML::TScoreHeader, 207
- MusicXML::TScoreInstrument, 208
- MusicXML::TScorePart, 209
- MusicXML::TScorePartwise, 210
- MusicXML::TScoreTimewise, 211
- MusicXML::TScoreVisitor, 212
- MusicXML::TSegno, 215
- MusicXML::TSlash, 216
- MusicXML::TSlide, 217
- MusicXML::TSlur, 218
- MusicXML::TSound, 219
- MusicXML::TSound
 - setDamperPedal, 221

- setPan, 221
- MusicXML::TStaffDetails, 222
- MusicXML::TStaffTuning, 224
- MusicXML::TStrongAccent, 225
- MusicXML::TSupport, 226
- MusicXML::TSymbolicNoteDuration, 227
- MusicXML::TTechnical, 228
- MusicXML::TTechnicals, 230
- MusicXML::TTie, 231
- MusicXML::TTieSlur, 232
- MusicXML::TTimeModification, 233
- MusicXML::TTimeSign, 234
- MusicXML::TTranspose, 236
- MusicXML::TTransposeVisitor, 237
- MusicXML::TTransposeVisitor
 - decPitch, 240
 - getAccidental, 239
 - getDiatonics, 239
 - getKey, 239
 - getOctave, 238
 - getOctaveStep, 238
 - incPitch, 239
 - transposePitch, 240
- MusicXML::TTrillOrnament, 241
- MusicXML::TTrillSound, 242
- MusicXML::TTuplet, 243
- MusicXML::TTupletActual, 245
- MusicXML::TTupletDesc, 246
- MusicXML::TTupletNormal, 247
- MusicXML::TTWMeasure, 248
- MusicXML::TTWPart, 249
- MusicXML::TUnpitched, 250
- MusicXML::TUnrolledVisitor, 251
- MusicXML::TVoice, 252
- MusicXML::TWavyLine, 253
- MusicXML::TWedge, 254
- MusicXML::TWords, 256
- MusicXML::TWork, 257
- MusicXML::visitable, 261
- MusicXML::vvector, 262
- MusicXML::xmlattribute, 263
- MusicXML::xmlelement, 264
- MusicXML::xmlheader, 266
- MusicXML::YesNo, 268

pairmap, 57

- setDamperPedal
 - MusicXML::TSound, 221
- setPan
 - MusicXML::TSound, 221
- setUnpitched
 - MusicXML::TMidiInstrument, 141
- smartable, 60

SMARTP, 62

TMidiContextVisitor, 136

TMidiContextVisitor

- getChan, 137
- getDate, 138
- getDuration, 138
- getDynamics, 138
- getFullDuration, 137
- getPart, 137, 138
- getPitch, 137
- getTempo, 138
- getTPQ, 138

TPartSummary, 177

transposePitch

- MusicXML::TTransposeVisitor, 240

TRational, 189

TScanVisitor, 205

TXML2GuidoVisitor, 258

- TXML2GuidoVisitor, 259

TXML2GuidoVisitor

- TXML2GuidoVisitor, 259

- visite, 260

visite

- MusicXML::TRoutedVisitor, 204

- TXML2GuidoVisitor, 260

Visitors, 27